

单位代码： 10293 密 级：

南京邮电大学

专 业 学 位 硕 士 论 文



论文题目： Swift: 启发式多信号融合自适应网络拥塞控制算法

学	号	<u>1224045520</u>
姓	名	<u>杭天铖</u>
导	师	<u>王俊昌教授</u>
专业	学位类别	<u>电子信息</u>
类	型	<u>全日制</u>
专业	(领域)	<u>计算机技术</u>
论文	提交日期	<u>2026 年 3 月</u>

摘要

随着云计算、移动互联网、人工智能和物联网技术的发展，端到端网络传输正在从单一固定终端和局域链路，扩展到跨地域远距离传输、蜂窝/无线/卫星等多接入网络，以及手机、笔记本、台式机和边缘设备等异构终端并存的场景。传统基于丢包的拥塞控制算法（如 TCP NewReno、CUBIC）存在拥塞信号滞后、高带宽延迟积（Bandwidth-Delay Product, BDP）网络收敛缓慢、难以区分拥塞程度等局限；基于时延的 TCP Vegas 和基于带宽/最小 RTT 模型的 BBR 则依赖易受路径变化影响的测量量，其与丢包算法共存时的公平性也取决于具体配置。本文提出 Swift 算法，一种启发式多信号融合自适应网络拥塞控制方法。Swift 以确定性规则在确认或拥塞事件上于协议栈之外计算控制决策，不依赖训练样本与神经网络推理，其行为可按规则分支解释，并可在给定配置和随机种子下重复执行。

Swift 算法的核心创新收敛为三点：（1）面向远距离与异构终端传输的多信号拥塞状态感知，将 ECN 状态、拥塞控制状态机状态、拥塞事件类型、RTT 和在途字节等信息组织为 11 维有效观测子空间，并据此区分超时、ECN 驱动的窗口缩减与普通丢包；该子空间嵌入 15 元素状态传输容器，另含 4 项跨进程路由元数据。（2）启发式反馈自适应的融合控制，根据最小 RTT 感知的 RTT 膨胀阈值、性能反馈值相对慢速基线的偏移以及连续增长趋势调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$ ，并以滑动时间窗累计确认量和窗口化最大值滤波估计带宽延迟积（Bandwidth-Delay Product, BDP），有界跟踪 $\alpha \times BDP$ 目标窗口。（3）面向不确定拥塞信号的稳定性与安全约束，对普通丢包、ECN 和超时分别采用 0.70、0.75 和 0.50 的窗口保留因子，并设置最多 3 次连续缩减、降窗后 4 个 ACK 事件冻结、窗口上下界和 CA_LOSS 下的陈旧动作作废机制。

本文基于 ns-3.40 网络仿真平台实现了 Swift 算法原型，仿真进程与决策进程经 ZeroMQ 同步请求一应答信道交互（由 ns3-gym/OpenGym 框架承载 15 元素状态容器的跨进程传输，仅作为消息通道使用，不含学习逻辑）。实验矩阵覆盖近端高速、共享收敛链路、无线局域网接入、蜂窝移动接入、城域与长途/跨域广域网及 LEO/GEO 卫星通信共 36 个场景，并通过 enable_udp_burst 开关构建“纯 TCP”与“TCP + 平均负载为瓶颈速率 32%（峰值 64%，50% 占空比）的 UDP Burst”两类 A/B 对照，共 288 次仿真。全部指标由 ns-3 FlowMonitor 产物重新导出并仅在前向数据流上定义；瓶颈队列统一采用启用 ECN 标记的 RED（ $MinTh = 0.3Q$ 、 $MaxTh = 0.9Q$ ），四种协议对称开启 ECN 协商。288 次仿真的指标均由 ns-3 FlowMonitor 前向 TCP 数据流统计得到，并通过流数完整性、物理取值范围和配置一致性检查。**稳态实验结果表明：**纯 TCP 设置下 Swift 在所考察场景中整体呈现较高的聚合吞吐量，相对逐场景最强基线的增益为 +2.1% ~ +5.8%（均值 +4.1%），相关结果覆盖近端高速、无线/蜂窝接入与远距离链路；平均单向时延相对 NewReno/CUBIC 均值平均低约 2.3%（36 个场景中 24 个更低），

相对 BBR 平均高约 0.8%，个别场景最高约 10.4%；Swift 的丢包率与 NewReno 相当，Jain 公平性指数接近 1 且总体与基线相当。**跨流量鲁棒性实验**表明：UDP 突发下 Swift 在多数场景中仍呈现吞吐优势（相对最强基线 +2.2% ~ +5.9%），吞吐量保持率均值 68.6% 与基线基本一致，未观察到明显吞吐塌缩；GEO、LEO 卫星和长途 WAN 等代表性远距离场景也呈现吞吐优势。上述结果来自每个 (场景, 协议, 设置) 的一次确定性运行，不附带跨随机种子置信区间；时延、丢包与公平性存在场景相关的混合结果，不能据此主张全面优于基线。

关键词：拥塞控制，启发式自适应，多信号融合，显式拥塞通知，自适应参数调优，远距离传输，异构终端

Abstract

End-to-end network transmission now spans long-distance cross-region paths, cellular, wireless, and satellite access networks, and heterogeneous endpoint environments involving phones, laptops, desktops, edge devices, and other terminal classes. Traditional loss-based congestion control algorithms such as TCP NewReno and CUBIC react only after loss, converge slowly in high Bandwidth-Delay Product (BDP) networks, and do not distinguish congestion severity. Delay-based TCP Vegas and model-based BBR rely on delay and bandwidth measurements that can be affected by path variation; their fairness when competing with loss-based algorithms also depends on the network configuration. This paper proposes Swift, a *heuristic* multi-signal adaptive congestion-control algorithm. Swift computes control decisions outside the protocol stack through deterministic rules on acknowledgement or congestion events, without training data or neural-network inference. Its behavior can be inspected branch by branch and repeated under a fixed configuration and random seed.

The core innovations of Swift are consolidated into three aspects: (1) multi-signal congestion-state perception for long-distance and heterogeneous endpoint transmission, using an 11-dimensional effective observation subspace to combine ECN status, Congestion Algorithm (CA) state, congestion events, RTT, and bytes in flight and to classify timeout, ECN-driven reduction, and ordinary loss; the subspace is carried in a 15-element container with four routing metadata fields; (2) heuristic feedback-adaptive fusion control that estimates the Bandwidth-Delay Product (BDP) from cumulative acknowledged bytes over a sliding time window and a windowed max-filter, adjusts per-flow $\alpha \in [0.85, 1.30]$ from min-RTT-aware RTT inflation, baseline-relative feedback, and growth trends, and tracks the $\alpha \times BDP$ target with bounded steps; and (3) stability and safety constraints for uncertain congestion signals, including retention factors of 0.70, 0.75, and 0.50 for ordinary loss, ECN, and timeout, at most three consecutive reductions, a four-ACK post-decrease freeze, window bounds, and invalidation of stale pending actions in CA_LOSS.

We implemented the Swift prototype on the ns-3.40 simulation platform; the simulator and decision processes interact over a synchronous ZeroMQ request-reply channel (the 15-element state container is carried by the ns3-gym/OpenGym framework, used purely as a message channel and containing no learning logic). The experiment matrix spans 36 scenarios covering near-end high-speed links, shared oversubscribed links, wireless LAN access, cellular mobile access, metropolitan/long-haul/inter-region wide area networks, and LEO/GEO satellite links. An `enable_udp_burst` switch constructs paired “pure TCP” and “TCP + On-Off UDP burst” settings, where the burst has an average offered load of 32% of the bottleneck rate (64% peak, 50% duty cycle), for 288 runs in total.

All metrics were re-derived from the ns-3 FlowMonitor artifacts and are defined *on the forward data flows only*; the bottleneck queue is RED with ECN marking ($MinTh = 0.3Q$, $MaxTh = 0.9Q$), and ECN negotiation is enabled symmetrically for all four protocols. Metrics for the 288 runs were computed from forward TCP flows reported by ns-3 FlowMonitor and checked for flow-count completeness, physical value ranges, and configuration consistency. **Steady-state results** show that Swift generally provides higher aggregate throughput across the evaluated scenarios, with gains of +2.1%–+5.8% (mean +4.1%) over the strongest per-scenario baseline. Its mean one-way delay is on average about 2.3% lower than the NewReno/CUBIC mean but 0.8% higher than BBR, with increases of up to 10.4% in individual scenarios. Swift’s loss rate is comparable to NewReno, while its Jain fairness index remains close to one and broadly comparable with the baselines. **Cross-traffic experiments** show a similar throughput pattern in most scenarios under UDP bursts (+2.2%–+5.9% versus the strongest baseline), while the mean throughput-retention ratio of 68.6% is comparable with the baselines and no marked throughput collapse is observed. Representative GEO and LEO satellite and long-haul WAN cases also show throughput gains. Each (scenario, protocol, setting) triple corresponds to one deterministic run, so the results have no cross-seed confidence intervals. Delay, loss, and fairness are mixed across scenarios and do not support claims of uniform superiority.

Key Words: Congestion Control, Heuristic Adaptation, Multi-signal Fusion, Explicit Congestion Notification, Adaptive Parameter Tuning, Long-distance Transmission, Heterogeneous Endpoints

目 录

摘要	I
Abstract	III
插图索引	X
表格索引	XI
专用术语注释表	XII
第一章 绪论	1
1.1 研究背景与动机	1
1.2 研究目标与主要贡献	2
1.3 论文组织结构	3
第二章 相关工作	4
2.1 传统拥塞控制算法	4
2.1.1 基于丢包的拥塞控制算法	4
2.1.2 基于延迟的拥塞控制算法	6
2.1.3 混合拥塞控制算法	7
2.2 显式拥塞通知机制	8
2.2.1 ECN 工作原理	8
2.2.2 ECN 相关拥塞控制算法	8
2.3 基于数据驱动的拥塞控制方法	10
2.3.1 离线策略生成方法	10
2.3.2 强化学习方法	10
2.4 远距离与异构终端网络特性分析	11
2.4.1 路径与接入特性	11
2.4.2 流量特性分析	11
2.4.3 应用延迟需求分析	12
2.4.4 网络设备与反馈机制特性	12
2.5 拥塞控制的理论基础	13
2.5.1 网络流量模型	13
2.5.2 排队论分析	13
2.5.3 控制论视角	14
2.6 学习型智能决策方法在网络中的应用综述	15
2.6.1 路由优化	15
2.6.2 资源分配	16
2.6.3 安全防护	16
2.6.4 与 Swift 的关系	16
2.7 本章小结	16
第三章 Swift 算法设计与实现	18
3.1 系统架构概述	18
3.2 决策问题建模与设计原则	19
3.2.1 状态向量设计原则	20
3.2.2 决策输出设计	20
3.2.3 性能反馈值设计	20
3.2.4 规则策略表示	21

3.3	11 维观测空间详细设计	21
3.3.1	连接标识参数 (索引 0、3)	22
3.3.2	时间信息参数 (索引 1、2)	22
3.3.3	窗口状态参数 (索引 4、5)	22
3.3.4	传输性能参数 (索引 6、7、8)	23
3.3.5	延迟测量参数 (索引 9、10)	23
3.3.6	回调类型参数 (索引 11)	24
3.3.7	拥塞控制状态参数 (索引 12、13、14)	24
3.4	多信号融合拥塞检测算法	25
3.4.1	拥塞信号分类	25
3.4.2	分层优先级机制	26
3.4.3	信号优先级与误判抑制策略	27
3.4.4	完整拥塞检测算法	28
3.5	差异化拥塞响应机制	28
3.5.1	窗口保留因子设计	28
3.5.2	差异化响应的理论依据	29
3.5.3	慢启动阈值更新	30
3.5.4	差异化拥塞响应算法	31
3.6	启发式反馈自适应的参数调优	31
3.6.1	乘性增加因子的作用	31
3.6.2	多因素调整机制	32
3.6.3	参数边界约束	33
3.6.4	自适应参数调优算法	35
3.7	融合控制策略	35
3.7.1	带宽延迟积估计	35
3.7.2	慢启动阶段策略	36
3.7.3	拥塞避免阶段策略	37
3.7.4	窗口边界约束	38
3.7.5	融合控制策略算法	38
3.8	每流独立状态管理	38
3.8.1	状态数据结构	38
3.8.2	状态查找与更新	40
3.8.3	状态生命周期管理	40
3.9	实现细节与优化技术	41
3.9.1	系统架构设计	41
3.9.2	核心算法伪代码	42
3.9.3	数据结构设计	42
3.9.4	性能优化技术	42
3.9.5	状态—决策同步交互路径	43
3.9.6	正确性保证机制	43
3.10	本章小结	44

第四章 实验评估	45
4.1 实验环境配置	45
4.1.1 软件与运行环境	45
4.1.2 网络拓扑与队列配置	45
4.1.3 仿真参数	46
4.1.4 随机种子与重复性说明	47
4.1.5 UDP Burst 干扰流量与实验矩阵	47
4.2 评测指标	48
4.2.1 指标的流向界定	48
4.2.2 吞吐量 (Throughput)	48
4.2.3 延迟 (Latency)	49
4.2.4 丢包率 (Loss Rate)	49
4.2.5 公平性 (Fairness)	49
4.3 对比算法	49
4.3.1 TCP NewReno	50
4.3.2 TCP CUBIC	50
4.3.3 TCP BBR	50
4.4 实验场景设计	50
4.4.1 场景矩阵与覆盖范围	50
4.4.2 数据质量审计与使用范围	51
4.5 实验方法论	52
4.5.1 实验设计原则	52
4.5.2 统计分析方法	52
4.6 实验结果分析	52
4.6.1 吞吐量对比	52
4.6.2 延迟对比	54
4.6.3 丢包率与公平性对比	55
4.6.4 延迟抖动对比	56
4.7 深入分析	56
4.7.1 场景分组讨论	56
4.7.2 机制有效性分析	57
4.7.3 关键参数的作用边界	58
4.8 UDP Burst 跨流量鲁棒性评估	58
4.8.1 聚合鲁棒性指标	58
4.8.2 配对场景逐一验证	59
4.8.3 鲁棒性机理分析	60
4.9 算法优势与适用边界总结	60
4.10 本章小结	61
第五章 讨论	62
5.1 设计考量	62
5.1.1 11 维观测空间的设计原理	62
5.1.2 差异化窗口保留因子的选择	62
5.1.3 自适应参数调优的设计	62

5.2	理论分析	63
5.2.1	收敛性分析	63
5.2.2	公平性分析	63
5.2.3	复杂度分析	63
5.3	适用边界与改进方向	63
5.3.1	在途水位与延迟的剩余代价	63
5.3.2	长 RTT 路径的验证与残余问题	64
5.3.3	实验证据的范围限制	64
5.3.4	参数自动调优	65
5.3.5	学习型策略增强	65
5.3.6	真实部署验证	65
5.4	与相关工作的比较	65
5.4.1	与 Aurora 的比较	65
5.4.2	与 DCTCP 的比较	66
5.4.3	与 BBR 的比较	66
5.4.4	与 PCC 的比较	66
5.5	敏感性分析	66
5.5.1	窗口保留因子的理论依据	67
5.5.2	连续递减保护阈值的数学分析	67
5.5.3	RTT 膨胀比阈值的设计考量	67
5.6	计算开销分析	67
5.6.1	时间复杂度	68
5.6.2	空间复杂度	68
5.6.3	工程开销来源	68
5.7	理论分析深入探讨	68
5.7.1	多信号融合的信息论基础	68
5.7.2	差异化响应的最优控制分析	69
5.7.3	自适应参数调优的规则化描述	69
5.7.4	收敛性的 Lyapunov 分析	69
5.7.5	公平性的定性分析	70
5.8	本章小结	70
第六章	结论与展望	71
6.1	研究总结	71
6.2	研究展望	73
6.2.1	参数自适应与窗口控制的细化	73
6.2.2	策略表达能力扩展	73
6.2.3	多目标优化扩展	74
6.2.4	真实网络部署与验证	74
6.2.5	实验方法的扩展	74
6.2.6	安全性、鲁棒性与跨层协同	74
6.2.7	异构网络与新兴传输协议适配	74
6.3	应用场景展望	75
6.4	研究意义与社会价值	75

参考文献 76

附录 A 程序清单 78

附录 B 攻读专业学位硕士期间撰写的论文 79

附录 C 攻读专业学位硕士期间申请的专利 80

附录 D 攻读专业学位硕士期间参加的科研项目 81

致谢 82

插图索引

图 1.1	Swift 算法核心贡献概览	2
图 2.1	拥塞控制算法分类体系	4
图 2.2	ECN 信号传递工作流程	9
图 3.1	Swift 系统架构	18
图 4.1	哑铃型 (Dumbbell) 实验网络拓扑	46
图 4.2	19 个代表性场景的聚合前向吞吐量 (对数坐标)	53
图 4.3	19 个场景的平均单向前向时延 (对数坐标)。黑色短划线为各场景的基线单向传播时延，柱 顶与短划线之间的距离即排队分量	55
图 4.4	15 个代表性配对场景的跨流量鲁棒性：UDP Burst 引起的吞吐量变化 (上) 与新增丢包 (下， 符号对数坐标)	59

表格索引

表 2.1	代表性拥塞控制算法综合对比	16
表 3.1	Swift 11 维观测空间参数定义	22
表 3.2	差异化窗口保留因子	29
表 4.1	默认仿真参数	46
表 4.2	代表性 19 个对照场景配置 (S1-S19; 另有 17 个补充场景见正文)	51
表 4.3	聚合前向吞吐量对比 (Mbps)	53
表 4.4	平均单向前向时延对比 (ms)	54
表 4.5	丢包率 (%) 与 Jain 公平性指数对比 (S1-S19)	56
表 4.6	平均延迟抖动对比 (ms, 代表性场景)	57
表 4.7	UDP Burst 跨流量鲁棒性聚合指标 (配对场景, $n = 36$)	58
表 4.8	UDP Burst 设置下 15 个代表性场景的吞吐量与延迟 (前向流聚合)	59
表 4.9	Swift 算法优势与适用边界 (基于全矩阵 36 场景)	60
表 5.1	Swift 与 Aurora 的详细对比	65
表 5.2	Swift 与 DCTCP 的详细对比	66
表 5.3	Swift 与 BBR 的详细对比	66

专用术语注释表

符号说明

$cwnd$	拥塞窗口 (Congestion Window), TCP 发送端在未收到确认的情况下最多可以发送的数据量 (字节)
$ssthresh$	慢启动阈值 (Slow Start Threshold), 区分慢启动和拥塞避免阶段的临界值 (字节)
α	乘性增加因子 (Multiplicative Increase Factor), 控制拥塞避免阶段窗口增长速度, 取值范围 $[0.85, 1.30]$, 基准值 1.10
γ	加性增加因子 (Additive Increase Factor), 控制窗口的加性增量, 默认值为 1.0
ρ	窗口保留因子 (Window Retention Factor), 拥塞响应后新窗口与原窗口的比值
ρ_{loss}	丢包保留因子, 默认值 0.70
ρ_{ecn}	ECN 保留因子, 默认值 0.75
$\rho_{timeout}$	超时保留因子, 默认值 0.50
η_f, η_b	性能反馈值快线/慢线指数移动平均 (EMA) 平滑系数, 默认值 0.15/0.02
BDP	带宽延迟积 (Bandwidth-Delay Product), 网络链路能够容纳的最大在途数据量 (字节)
MSS	最大报文段长度 (Maximum Segment Size), 本文取 1440 字节
RTT	往返时延 (Round-Trip Time), 数据包从发送端到接收端再返回的总时间
$W(t)$	时刻 t 的拥塞窗口大小
W_{max}	上次拥塞事件前的最大窗口大小
K	CUBIC 算法中恢复到 W_{max} 所需的时间
C	链路容量或 CUBIC 缩放因子
β	窗口缩减因子
δ	延迟惩罚权重参数
λ	丢包惩罚权重参数

H	Hurst 参数, 描述流量自相似性
$\bar{Q}$	平均队列长度
$\bar{W}$	平均等待时间
τ	反馈延迟 (约等于 RTT)

缩略词

TCP	传输控制协议 (Transmission Control Protocol)
ECN	显式拥塞通知 (Explicit Congestion Notification)
RTT	往返时延 (Round-Trip Time)
BDP	带宽延迟积 (Bandwidth-Delay Product)
AIMD	加性增乘性减 (Additive Increase Multiplicative Decrease)
MSS	最大报文段长度 (Maximum Segment Size)
DCTCP	数据中心 TCP (Data Center TCP)
BBR	瓶颈带宽和往返传播时间 (Bottleneck Bandwidth and Round-trip propagation time)
CUBIC	三次函数拥塞控制算法
ACK	确认 (Acknowledgment)
SACK	选择性确认 (Selective Acknowledgment)
CE	拥塞经历 (Congestion Experienced)
ECE	ECN 回显 (ECN-Echo)
CWR	拥塞窗口缩减 (Congestion Window Reduced)
RED	随机早期检测 (Random Early Detection)
CA	拥塞算法 (Congestion Algorithm)
PPO	近端策略优化 (Proximal Policy Optimization)
DQN	深度 Q 网络 (Deep Q-Network)
PCC	面向性能的拥塞控制 (Performance-oriented Congestion Control)
ZMQ	ZeroMQ 消息队列
SDN	软件定义网络 (Software Defined Networking)
QoS	服务质量 (Quality of Service)
QUIC	快速 UDP 互联网连接 (Quick UDP Internet Connections)

RDMA	远程直接内存访问 (Remote Direct Memory Access)
PFC	优先级流量控制 (Priority Flow Control)
INT	带内网络遥测 (In-band Network Telemetry)
HPCC	高精度拥塞控制 (High Precision Congestion Control)
WAN	广域网 (Wide Area Network)
Pod	交换机组 (Point of Delivery)
EMA	指数移动平均 (Exponential Moving Average)
IPC	进程间通信 (Inter-Process Communication)

第一章 绪论

1.1 研究背景与动机

随着云计算、大数据、人工智能、移动互联网和物联网技术的蓬勃发展，端到端网络传输的服务对象正在从固定主机扩展到手机、笔记本、台式机、边缘节点和传感设备等多模态终端，传输路径也从近端局域链路延伸到跨城、跨域、蜂窝接入、无线局域网和卫星通信等远距离复杂网络。传输控制协议（Transmission Control Protocol, TCP）作为互联网传输层的核心协议，承载了大量关键业务流量，其拥塞控制机制的性能直接影响网络资源利用效率、应用响应延迟和用户服务质量。网络拥塞控制是计算机网络领域最具挑战性的研究课题之一，自互联网诞生以来一直是学术界和工业界持续关注的焦点问题。

在现代端到端网络中，链路与终端条件呈现出更强的异构性。一方面，城域/广域网、蜂窝网络、WiFi 与卫星链路具有不同的 RTT 量级、丢包背景、可用带宽和链路抖动；另一方面，手机、电脑、边缘设备等终端在处理能力、无线接入方式、移动性和业务负载上差异显著。这种长距离、多接入、多终端并存的网络环境对传统拥塞控制算法提出了前所未有的严峻挑战。

现代网络应用承载着多种类型的业务负载，这些业务对网络性能有着截然不同的需求。移动视频、实时会议和云游戏需要稳定的低延迟与低抖动；分布式存储、内容分发和大文件同步需要高吞吐量以支持大规模数据传输；边缘计算、工业物联网和车联网则同时关注可靠性、实时性和终端资源约束。单一的拥塞控制策略难以同时满足这些多样化的性能需求。

更为复杂的是，远距离和多接入传输中的拥塞信号更难解释。丢包可能来自真实队列溢出，也可能来自无线误码、链路切换或终端侧调度抖动；RTT 升高既可能表示排队，也可能来自路径变化、接入链路波动或终端处理延迟；移动终端后台同步、视频传输、RPC 突发、备份任务和多租户共享链路还会造成短时间带宽挤占。传统单一信号驱动的拥塞控制方法难以在这些场景中稳定区分拥塞程度。

传统的 TCP 拥塞控制算法在这种复杂的网络环境中显露出诸多不足。基于丢包的算法（如 TCP NewReno^[1]、TCP CUBIC^[2]）将数据包丢失作为拥塞信号，但丢包是拥塞的滞后指示器，通常意味着网络缓冲区已经溢出，排队延迟已经积累到较高水平。基于时延的算法（如 TCP Vegas^[3]）和基于带宽/最小 RTT 模型的 TCP BBR^[4] 尝试在丢包前调节发送行为，但 RTT 与带宽估计易受路径变化影响，与基于丢包算法共存时的公平性也取决于具体配置。

显式拥塞通知（Explicit Congestion Notification, ECN）机制为解决上述问题提供了新的思路。ECN 允许网络设备在队列长度超过阈值时标记数据包而非丢弃，从而实现更早、更精确的拥塞信号传递。数据中心 TCP（DCTCP^[5]）验证了 ECN 细粒度反馈在特定低时延网络中的有效性，但其适用前提较强；在远距离、多接入和多模态终端并存的环境中，现有算法对

ECN、RTT、丢包和协议栈状态的联合利用仍不充分，大多采用静态参数配置，缺乏对动态网络环境的自适应能力。

近年来，机器学习特别是强化学习技术为网络拥塞控制提供了新的研究思路^[6]：Aurora、Orca 等方案通过训练神经网络策略或在线学习调整发送行为，具有适应不同网络条件的潜力。然而，学习型方案通常需要训练与模型推理设施，其训练分布外泛化能力仍需专门验证，决策过程也较难逐事件审计。部分方案主要依赖窗口、时延和吞吐统计，未直接利用 ECN 与拥塞控制状态机等协议栈内部信号。

本文因此选择白盒启发式的设计路线：不引入学习型策略，而是在协议栈回调点采集包含 ECN、拥塞状态机与窗口/RTT 在内的多维状态，以确定性规则在确认或拥塞事件上输出拥塞窗口与慢启动阈值决策，仅在参数层使用基线相对的轻量反馈自适应。该路线侧重可解释性和可重复性，并通过多信号融合与参数自适应处理不同路径条件。

1.2 研究目标与主要贡献

针对上述问题，本文提出 Swift 算法，一种面向远距离传输、多接入链路和异构终端场景的启发式多信号融合自适应网络拥塞控制方法。其设计目标是在给定安全边界内提高链路利用率，并控制排队时延和窗口振荡。

图1.1展示了 Swift 算法的三项核心贡献及各组件之间的逻辑关系。

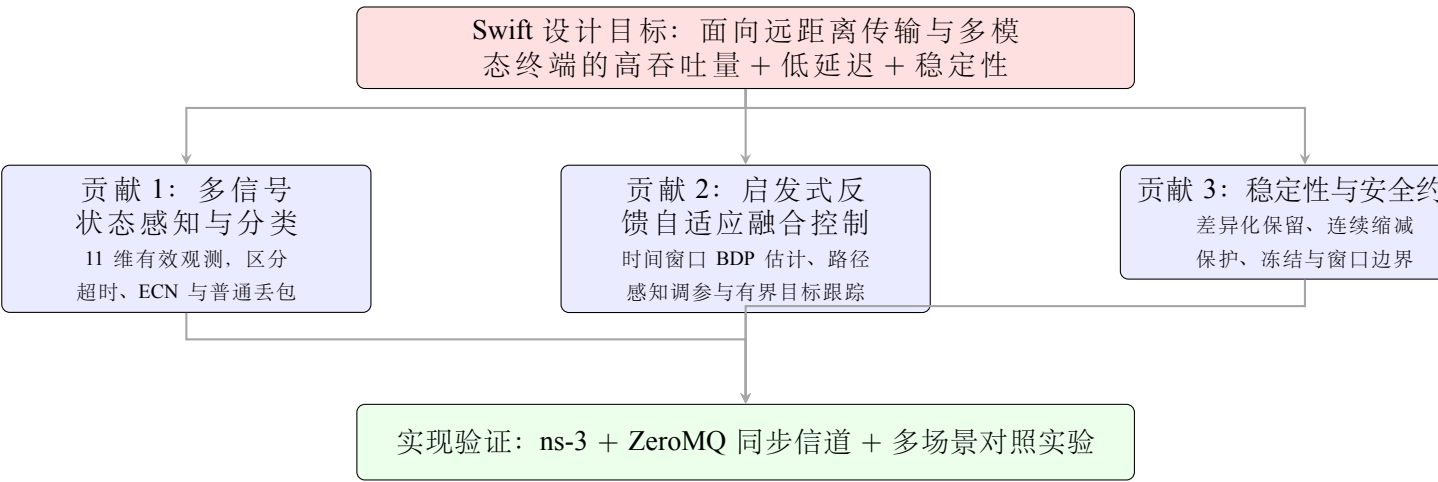


图 1.1 Swift 算法核心贡献概览

本文的主要研究贡献如下：

(1) 面向远距离与异构终端的多信号拥塞状态感知

本文设计了一个 11 维有效观测子空间（嵌入于 15 元素状态传输容器，另含套接字 UUID、环境类型、仿真时间、节点 ID 四项跨进程路由元数据），将 ECN 状态、拥塞控制状态机状态、拥塞事件类型、RTT 和在途字节等信息纳入状态表示，并据此区分超时、ECN 驱动的窗口缩

减与普通丢包。该观测子空间为不同接入与路径条件下的拥塞判定提供协议语义；本文的仿真场景用于抽象这些链路条件，不等同于具体终端硬件测试。

(2) 启发式反馈自适应的融合控制

本文采用滑动时间窗口内的累计确认量估计交付速率，经窗口化最大值滤波并结合最小 RTT 得到 BDP 估计；控制器根据最小 RTT 感知的 RTT 膨胀阈值、性能反馈值相对慢速基线的偏移和连续增长趋势调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$ ，并以有界步长跟踪 $\alpha \times BDP$ 目标窗口。该过程由确定性规则执行，不包含策略训练或神经网络推理。

(3) 面向不确定拥塞信号的稳定性与安全约束

Swift 对普通丢包、ECN 和超时分别采用 0.70、0.75 和 0.50 的窗口保留因子，并设置最多 3 次连续缩减、降窗后 4 个 ACK 事件冻结、窗口上下界和 CA_LOSS 下的陈旧动作作废。上述机制用于限制连续过度缩减、快速反弹与陈旧决策覆盖，具体独立贡献仍需消融实验验证。

1.3 论文组织结构

本文的其余部分组织如下：

第 2 章介绍相关工作，包括传统拥塞控制算法、基于机器学习的拥塞控制方法、显式拥塞通知机制、智能决策方法在网络领域的应用综述等。

第 3 章详细描述 Swift 算法的设计与实现，包括系统架构、观测空间设计、多信号融合拥塞检测、差异化拥塞响应、自适应参数调优、融合控制策略等核心组件。

第 4 章介绍实验评估，包括实验环境配置、评测指标、对比算法、实验场景设计以及详细的实验结果分析。

第 5 章讨论 Swift 算法的适用边界和未来研究方向。

第 6 章总结全文并展望未来工作。

第二章 相关工作

网络拥塞控制是计算机网络领域的核心研究课题之一，自互联网诞生以来一直是学术界和工业界关注的焦点。本章从传统拥塞控制算法、显式拥塞通知机制、基于机器学习的拥塞控制方法三个方面综述相关研究工作，并分析各方法的优缺点及其对 **Swift** 算法设计的启示。

图2.1展示了拥塞控制算法的分类体系及其演进关系。

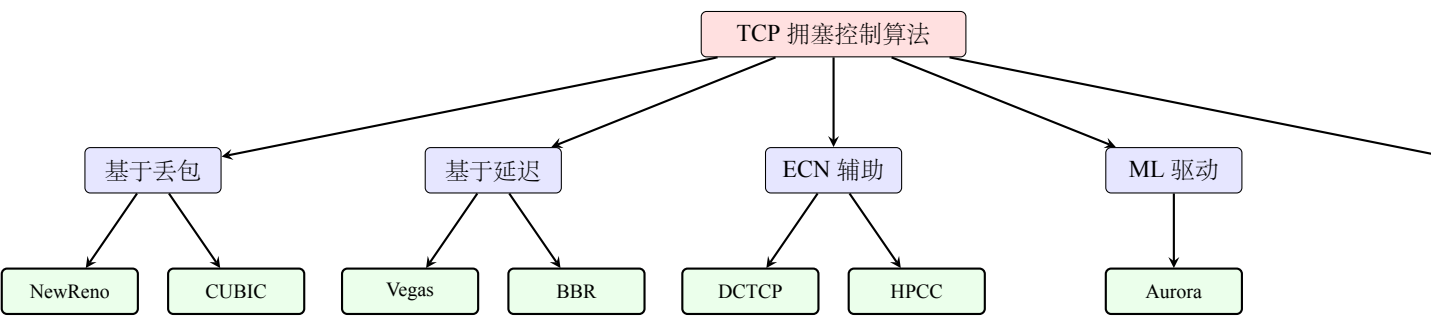


图 2.1 拥塞控制算法分类体系

2.1 传统拥塞控制算法

传统的 TCP 拥塞控制算法根据其使用的拥塞信号类型，可以分为基于丢包的算法、基于延迟的算法和混合算法三大类。

2.1.1 基于丢包的拥塞控制算法

基于丢包的拥塞控制算法将数据包丢失作为网络拥塞的主要判断信号。这类算法的核心思想是：当发送端检测到数据包丢失（通过超时或重复确认判断）时，认为网络发生了拥塞，需要降低发送速率以缓解拥塞。

TCP Tahoe 和 **TCP Reno** 是最早的拥塞控制算法，由 Van Jacobson 于 1988 年提出^[7]。TCP Tahoe 引入了慢启动(Slow Start)、拥塞避免(Congestion Avoidance)和快速重传(Fast Retransmit)三个核心机制。慢启动机制在连接建立初期指数级增加拥塞窗口，快速探测可用带宽；拥塞避免机制在窗口达到阈值后线性增加窗口，谨慎探测额外带宽；快速重传机制在收到三个重复确认时立即重传丢失的数据包，而无需等待超时。TCP Reno 在 Tahoe 的基础上增加了快速恢复(Fast Recovery)机制，在快速重传后将窗口减半而非重置为 1，提高了丢包后的恢复效率。

TCP NewReno^[1] 是对 TCP Reno 的改进，主要解决了 Reno 在多个数据包丢失时性能下降的问题。NewReno 修改了快速恢复机制的行为：在快速恢复期间，如果收到部分确认(Partial

ACK，确认了部分但非全部已发送数据），则继续保持在快速恢复状态，直到所有丢失的数据包都被成功重传。这种改进使得 NewReno 能够更高效地处理突发丢包情况，成为 RFC 6582 中定义的标准 TCP 拥塞控制算法之一。

TCP CUBIC^[2] 是目前 Linux 操作系统的默认拥塞控制算法，自 Linux 2.6.19 版本开始采用。CUBIC 的核心创新是采用三次函数（Cubic Function）模型调整拥塞窗口大小：

$$W(t) = C \cdot (t - K)^3 + W_{max} \quad (2.1)$$

其中， $W(t)$ 是时刻 t 的窗口大小， C 是缩放因子， $K = \sqrt[3]{\frac{W_{max} \cdot \beta}{C}}$ 是从上次拥塞事件恢复到 W_{max} 所需的时间， W_{max} 是上次拥塞事件前的窗口大小， β 是窗口缩减因子。

CUBIC 的三次函数模型具有以下特点：（1）在远离 W_{max} 时快速增长，在接近 W_{max} 时缓慢增长，实现了对可用带宽的稳定探测；（2）窗口增长速度与 RTT 无关，避免了传统 AIMD 算法在高 RTT 链路上收敛缓慢的问题；（3）在 W_{max} 附近有一个“平台期”，减少了在最优工作点附近的窗口振荡。

TCP BIC^[8] 是 CUBIC 的前身，采用二分搜索（Binary Search）算法调整拥塞窗口。BIC 的基本思想是：上次拥塞发生时的窗口 W_{max} 是带宽上限的估计，当前窗口是带宽下限的估计，通过二分搜索在两者之间寻找最优窗口。BIC 算法的窗口增长函数如下：

$$W_{new} = W_{current} + \frac{W_{max} - W_{current}}{2} \quad (2.2)$$

当窗口接近 W_{max} 时，BIC 采用加性增加策略以避免过冲。BIC 算法在高带宽延迟积网络中表现良好，但其窗口增长行为对 RTT 不敏感，在与其他 TCP 变体共存时可能产生公平性问题。

基于丢包的算法存在以下共同缺陷：

（1）**拥塞信号滞后**：丢包是网络拥塞的滞后指示器。网络设备的缓冲区在溢出丢包前已经积累了大量排队数据，导致排队延迟显著增加。在深缓冲区（Deep Buffer）网络环境中，这种延迟可能达到数十甚至数百毫秒级别，严重影响延迟敏感型应用的性能。

（2）**缓冲区膨胀**：基于丢包算法的典型行为是持续增加发送速率直到发生丢包，这会导致网络缓冲区长期处于高占用状态，产生“Bufferbloat”（缓冲区膨胀）问题^[9]。缓冲区膨胀不仅增加了网络延迟，还降低了网络的整体吞吐量，因为大量带宽被用于传输最终会被丢弃的数据包。

（3）**无法区分拥塞程度**：基于丢包算法只能检测“有丢包”或“无丢包”两种状态，无法区分轻度拥塞和严重拥塞。因此，无论拥塞程度如何，算法都采用统一的窗口缩减策略，在轻度拥塞时可能响应过度，在严重拥塞时又可能响应不足。

(4) **高 BDP 网络收敛缓慢**: 在高带宽延迟积网络中, 基于丢包算法的收敛速度极为缓慢。以 10Gbps 带宽、100 微秒 RTT 的链路为例, BDP 约为 125KB, 需要约 86 个 MSS 的窗口才能充分利用带宽。从初始窗口增长到 86 个 MSS 需要较长时间, 丢包后的恢复更是如此。

2.1.2 基于延迟的拥塞控制算法

基于延迟的拥塞控制算法通过监测往返时延 (RTT) 的变化来推断网络拥塞状态, 试图在丢包发生前就感知到拥塞并采取措施。这类算法的核心假设是: 网络拥塞会导致队列积累, 进而导致 RTT 增加。

TCP Vegas^[3] 是最早的基于延迟拥塞控制算法之一, 由 Lawrence Brakmo 和 Larry Peterson 于 1994 年提出。Vegas 通过比较期望吞吐量和实际吞吐量来估计网络中的排队数据量。期望吞吐量定义为:

$$Expected = \frac{cwnd}{BaseRTT} \quad (2.3)$$

实际吞吐量定义为:

$$Actual = \frac{cwnd}{RTT} \quad (2.4)$$

其中, $BaseRTT$ 是测量到的最小 RTT, 被认为是无排队情况下的传播延迟。差值 $\Delta = Expected - Actual$ 表示网络中排队的的数据量。Vegas 根据 Δ 的大小调整拥塞窗口:

$$cwnd = \begin{cases} cwnd + 1 & \text{如果 } \Delta < \alpha \\ cwnd - 1 & \text{如果 } \Delta > \beta \\ cwnd & \text{否则} \end{cases} \quad (2.5)$$

其中, α 和 β 是预设的阈值参数, 通常设置为 $\alpha = 1$ 、 $\beta = 3$ 。

TCP BBR^[4] (Bottleneck Bandwidth and Round-trip propagation time) 是 Google 于 2016 年提出的基于延迟算法, 是近年来最具影响力的拥塞控制算法之一。BBR 的核心思想是通过显式估计瓶颈带宽 (Bottleneck Bandwidth, BtlBw) 和最小 RTT (RTT_{prop}), 以带宽延迟积为目标调整发送速率。

BBR 通过维护一个滑动窗口来跟踪最大投递率 (作为 BtlBw 的估计) 和最小 RTT (作为 RTT_{prop} 的估计)。发送速率和拥塞窗口按以下方式计算:

$$pacing_rate = \frac{pacing_gain \times BtlBw}{6} \quad (2.6)$$

$$cwnd = cwnd_gain \times BDP = cwnd_gain \times BtlBw \times RTT_{prop} \quad (2.7)$$

BBR 采用周期性的增益调整策略，在不同阶段采用不同的 $pacing_gain$ 和 $cwnd_gain$ 值，以实现带宽探测和 RTT 测量。

TCP Copa^[10] 是 MIT 于 2018 年提出的基于延迟算法，采用了与 Vegas 类似但更精细的 RTT 监测机制。Copa 将目标排队延迟设定为 $RTT_{standing} - RTT_{min}$ 的函数，其中 $RTT_{standing}$ 是最近一个 RTT 窗口内的最小 RTT。Copa 采用以下窗口调整规则：

$$cwnd = cwnd + \frac{1}{cwnd \cdot \delta} \quad \text{或} \quad cwnd = cwnd - \frac{1}{cwnd \cdot \delta} \quad (2.8)$$

其中， δ 是目标延迟参数，默认值为 0.5。Copa 还引入了“竞争模式”检测机制，当检测到与基于丢包算法的流竞争时，会切换到更激进的发送策略。

基于延迟的算法存在以下共同问题：

(1) **RTT 测量不稳定**：RTT 测量受多种因素影响，包括操作系统调度延迟、网络路径抖动、网络拓扑变化、对端主机处理延迟等。在高速网络中，这些干扰因素可能与实际排队延迟处于同一数量级，导致基于 RTT 的拥塞推断不可靠。

(2) **公平性问题**：基于延迟算法在与基于丢包算法共存时存在严重的公平性问题。由于基于延迟算法对拥塞信号更敏感，会在 RTT 增加时提前降低发送速率，而基于丢包算法继续增加窗口直到丢包。研究表明，在与 CUBIC 流共存时，BBR 流的吞吐量可能下降 30% 以上^[11]。

(3) **参数敏感性**：基于延迟算法的性能对参数设置较为敏感。例如，Vegas 的 α 和 β 参数、Copa 的 δ 参数都需要根据具体网络环境进行调优，而静态参数难以适应动态变化的网络条件。

2.1.3 混合拥塞控制算法

混合拥塞控制算法综合利用丢包和延迟两种信号进行拥塞控制，试图结合两类算法的优点。

Compound TCP^[12] 是 Microsoft 开发的混合算法，在 Windows Vista 及后续版本中使用。Compound TCP 在传统的基于丢包的窗口增长之上叠加了一个基于延迟的“快速”窗口分量，在网络空闲时快速增长，在队列积累时减速增长。

TCP Illinois^[13] 根据 RTT 变化调整 AIMD 参数，在低延迟时采用更激进的增加策略，在高延迟时采用更保守的增加策略和更激进的减少策略。

TCP Westwood^[14] 通过测量 ACK 到达率来估计可用带宽，在丢包时根据估计的带宽而非固定比例调整慢启动阈值。

混合算法在一定程度上缓解了单一信号类型的局限性，但也增加了算法复杂性，且参数调优更加困难。

2.2 显式拥塞通知机制

显式拥塞通知（Explicit Congestion Notification, ECN）是由 IETF 在 RFC 3168^[15] 中定义的一种网络层辅助拥塞控制机制。ECN 通过修改 IP 报头中的服务类型字段（Type of Service, ToS）来传递拥塞信息，实现了比隐式丢包信号更早、更精确的拥塞通知。

2.2.1 ECN 工作原理

ECN 在 IP 报头中使用两个比特（ECN 字段），定义了四种码点：

- 00: Non-ECT（非 ECN 能力传输），表示端点不支持 ECN
- 01: ECT(1)（ECN 能力传输 1），表示端点支持 ECN
- 10: ECT(0)（ECN 能力传输 0），表示端点支持 ECN
- 11: CE（Congestion Experienced，拥塞经历），由路由器标记，表示经历了拥塞

ECN 的工作流程如下：发送端在发送数据包时将 ECN 字段设置为 ECT(0) 或 ECT(1)，表明该数据包支持 ECN。当网络设备（如路由器）检测到队列长度超过预设阈值时，将 ECN 字段从 ECT(0/1) 修改为 CE，而不是丢弃数据包。接收端收到 CE 标记的数据包后，在 TCP 报头中设置 ECE（ECN-Echo）标志，将拥塞信息反馈给发送端。发送端收到 ECE 标志后，采取拥塞响应措施（如降低发送速率），并在随后的数据包中设置 CWR（Congestion Window Reduced）标志，通知接收端已经响应了拥塞通知。图2.2展示了 ECN 信号传递的完整工作流程。

2.2.2 ECN 相关拥塞控制算法

DCTCP^[5]（Data Center TCP）是 Microsoft 和 Stanford 于 2010 年提出的面向数据中心环境的拥塞控制算法，是 ECN 应用的典范。DCTCP 的核心创新是利用 ECN 标记的**比例**（而非仅仅是有无）来进行细粒度的窗口调整。DCTCP 维护一个指数加权移动平均（EWMA）的 ECN 标记比例 α ：

$$\alpha = (1 - g) \cdot \alpha + g \cdot F \quad (2.9)$$

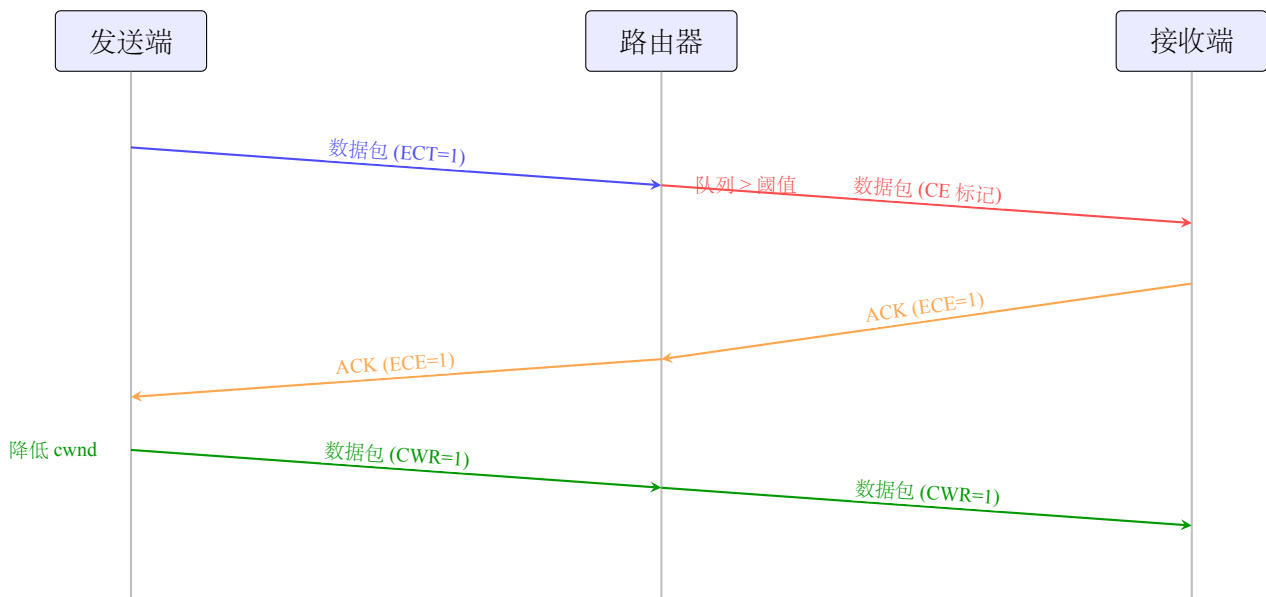


图 2.2 ECN 信号传递工作流程

其中, g 是平滑因子 (通常为 $1/16$), F 是最近一个 RTT 内被标记的数据包比例。拥塞窗口根据 α 进行调整:

$$cwnd = cwnd \times \left(1 - \frac{\alpha}{2}\right) \quad (2.10)$$

这种设计使得 DCTCP 能够根据拥塞程度进行比例响应: 轻度拥塞时 (α 较小) 轻微降低窗口, 严重拥塞时 (α 较大) 大幅降低窗口。

DCTCP 要求网络交换机配置适当的 ECN 标记阈值, 通常使用 RED (Random Early Detection) 或其变体。推荐的配置是将标记阈值设置为较低的值 (如 20 个数据包或 BDP 的 10%), 以实现早期拥塞通知。

ECN++^[16] 是对 DCTCP 的改进, 引入了更精细的 ECN 标记策略和响应机制。ECN++ 在接收端累积 ECN 标记计数, 而非仅使用二元 ECE 标志, 从而传递更丰富的拥塞信息。

HPCC^[17] (High Precision Congestion Control) 是阿里巴巴于 2019 年提出的高精度拥塞控制算法, 利用网络设备提供的精确拥塞信息 (如队列长度、带宽利用率) 进行拥塞控制。HPCC 要求网络设备支持 INT (In-band Network Telemetry) 功能。

ECN 机制的主要优势在于: (1) 能够在丢包发生前提供明确的拥塞信号; (2) 避免了因丢包导致的重传开销; (3) 信号来自网络设备的显式标记, 比推测性的 RTT 测量更可靠。然而, ECN 的有效使用需要网络设备的支持和正确配置, 且现有算法对 ECN 信号的利用还不够充分。

2.3 基于数据驱动的拥塞控制方法

数据驱动方法通过离线搜索、在线试验或强化学习等方式生成或调整拥塞控制策略，其效果依赖训练或优化所采用的网络模型、目标函数与评价环境。

2.3.1 离线策略生成方法

Remy^[18] 采用离线优化方法生成拥塞控制规则。Remy 在给定网络模型和目标函数的条件下搜索窗口调整规则；规则依据 RTT 测量、ACK 间隔等特征选择窗口增量和发送间隔。其结果针对预设模型优化，对模型偏差和状态划分粒度较为敏感，应用到不同网络条件时需要重新评估。

2.3.2 强化学习方法

强化学习方法通过与网络环境交互来优化拥塞控制策略，可能适应训练分布覆盖的网络变化，但不能据此保证获得全局最优策略或适用于未见条件。

Aurora^[19] 是 CMU 于 2019 年提出的基于深度强化学习的拥塞控制算法。Aurora 采用 PPO (Proximal Policy Optimization) 算法训练神经网络策略，输入网络状态特征（延迟梯度、延迟比率、发送比率等），输出发送速率调整因子。Aurora 的状态空间包含 10 个特征，涵盖延迟、吞吐量和丢包率的历史统计信息。

Aurora 的奖励函数设计为吞吐量和延迟的加权组合：

$$reward = throughput - \delta \cdot delay - \lambda \cdot loss \quad (2.11)$$

其中， δ 和 λ 是权重参数，控制延迟和丢包的惩罚程度。

Orca^[20] 是 UC Berkeley 于 2020 年提出的强化学习拥塞控制算法，采用经典的 AIMD 框架，但使用强化学习来动态调整 AIMD 参数。Orca 的核心思想是：AI 参数（增加因子 α 和减少因子 β ）不应是固定值，而应根据网络状态动态调整。

PCC^[21] (Performance-oriented Congestion Control) 是 UIUC 提出的基于在线学习的拥塞控制算法。PCC 采用实用函数 (Utility Function) 指导发送速率调整，通过在线实验探索最优速率。

$$U(x) = T \cdot x^\alpha - \delta \cdot L \cdot x - \beta \cdot D \cdot x \quad (2.12)$$

其中， x 是发送速率， T 是吞吐量， L 是丢包率， D 是延迟， α 、 δ 、 β 是参数。PCC 通过梯度上升方法最大化实用函数。

强化学习拥塞控制面临若干需要分别评估的问题：状态表示未必包含 ECN 与拥塞状态机等协议栈内部语义；奖励函数需要处理吞吐、延迟、丢包和公平性的权衡；训练与推理会引入额外资源需求；策略在训练分布之外的表现需要独立验证。不同方法对这些问题的处理并不相同，因此本文不作统一的维度或开销量化。

本文提出的 Swift 不属于学习型拥塞控制：它不训练策略、不依赖神经网络推理，而是以确定性规则映射协议栈状态到窗口决策，仅借用“多维状态描述与反馈驱动参数调整”的组织方式。Swift 同样需要验证参数在未覆盖网络条件下的适用性，只是不涉及训练分布和模型推理问题。

2.4 远距离与异构终端网络特性分析

为了更好地理解 Swift 算法的设计动机，本节分析远距离传输、多接入链路与异构终端并存环境中的网络特性。此类环境通常同时包含手机、笔记本电脑、台式机、边缘节点和云端服务等端系统，端到端路径可能跨越无线接入、城域网、广域网和卫星链路，网络条件随终端能力、接入方式和路径拥塞状态动态变化。

2.4.1 路径与接入特性

远距离与异构终端传输场景具有以下特点：

(1) 往返时延跨度大：不同路径的 RTT 可能从毫秒级扩展到数百毫秒级。较大的带宽时延积使发送端需要更准确地估计可容纳的在途数据量，否则容易出现带宽利用不足或队列持续驻留。

(2) 接入链路异构：终端可能通过有线以太网、Wi-Fi、蜂窝网络或卫星链路接入网络。不同接入链路在带宽、误码、调度粒度和队列管理方式上差异显著，使固定参数的拥塞控制难以同时适配多类路径。

(3) 终端能力差异明显：手机、笔记本电脑、桌面终端和边缘设备在处理能力、网卡能力、缓冲区规模和操作系统协议栈配置方面存在差异，这会影响 ACK 反馈节奏、突发发送行为和拥塞窗口恢复速度。

(4) 路径状态动态变化：无线信道波动、移动性、跨流量突发和路由变化会导致 RTT、吞吐量和丢包风险快速变化。拥塞控制算法需要利用多维状态信号进行及时感知，而不能仅依赖单一丢包反馈。

2.4.2 流量特性分析

远距离与多终端网络中的流量负载具有明显的多样性：

(1) **业务类型多样**: 文件同步、视频会议、网页访问、远程桌面、云游戏、在线备份和模型数据传输等业务可能同时存在, 对吞吐量、时延和稳定性的要求不同。

(2) **突发跨流量普遍**: 非响应式 UDP 流、周期性上报、后台同步和多媒体码流可能在短时间内占用瓶颈队列, 放大 RTT 抖动并干扰 TCP 长流的窗口控制。

(3) **长短流混合**: 短流关注完成时间, 长流关注持续吞吐量和稳定性。若拥塞控制响应过慢, 短流可能受到排队延迟影响; 若响应过激, 长流又可能出现带宽利用不足。

(4) **反馈信号不完全一致**: 丢包、ECN、RTT 膨胀和协议栈状态分别反映不同层面的网络变化。多信号融合能够降低单一信号误判带来的风险。

2.4.3 应用延迟需求分析

不同应用对网络延迟和吞吐量的需求并不相同:

(1) **实时交互业务**: 视频会议、云游戏和远程协作要求持续低延迟, 拥塞控制需要避免队列长期驻留。

(2) **大文件与数据同步业务**: 云盘同步、备份和数据分发更关注带宽利用率, 需要在不引发持续拥塞的前提下充分填充高 BDP 路径。

(3) **移动终端业务**: 移动设备受无线链路波动和终端能耗约束影响, 更需要平滑、稳定且可自适应的窗口调节策略。

(4) **边缘与云服务访问**: 边缘计算、远程推理和跨区域服务调用同时关注时延、吞吐量和鲁棒性, 需要算法在多目标之间动态权衡。

Swift 算法的“高吞吐、低排队、强鲁棒”设计目标正是为了适应上述复杂场景: 在路径条件良好时保持较高带宽利用率, 在拥塞或突发跨流量出现时及时降低排队和丢包风险。

2.4.4 网络设备与反馈机制特性

网络设备和协议栈反馈机制对拥塞控制算法有重要影响:

(1) **缓冲区规模差异**: 不同接入设备和中间节点的缓冲区规模差异较大, 浅缓冲链路更容易丢包, 深缓冲链路更容易产生排队时延。

(2) **ECN 支持程度不一**: 部分网络设备支持 ECN 标记, 可以在队列溢出前提供早期拥塞信号。算法需要在可获得 ECN 信号时充分利用该信号, 同时, 在无 ECN 场景下仍保持基于丢包和 RTT 的稳健响应。

(3) **终端协议栈可观测性增强**: 现代协议栈能够暴露拥塞状态、拥塞事件、RTT 样本和在途字节等信息, 为多维状态驱动的拥塞控制建模提供基础。

(4) **部署形态多样**: 拥塞控制逻辑既可以在内核 TCP 栈中实现, 也可以在用户态传输协议或仿真环境中验证。统一的状态感知和窗口控制框架有助于后续迁移与工程化。

2.5 拥塞控制的理论基础

本节介绍网络拥塞控制的理论基础, 包括流量模型、排队论分析和控制论视角, 为理解 Swift 算法的设计决策提供理论支撑。

2.5.1 网络流量模型

网络流量建模是拥塞控制研究的基础。常用的流量模型包括:

(1) **泊松模型**: 传统上假设数据包到达服从泊松过程, 即到达间隔服从指数分布。泊松模型的数学性质简单, 便于分析, 但实际网络流量通常表现出更强的突发性。

(2) **自相似模型**: 研究表明, 网络流量具有自相似 (Self-Similar) 特性, 即在不同时间尺度上呈现相似的统计特性。自相似流量可以用分数布朗运动 (Fractional Brownian Motion) 或 Pareto 分布建模。设流量过程为 $X(t)$, 其自相似性定义为:

$$X(t) \stackrel{d}{=} m^{-H} X(mt), \quad \forall m > 0 \quad (2.13)$$

其中 H 是 Hurst 参数 ($0.5 < H < 1$), $\stackrel{d}{=}$ 表示同分布。

(3) **ON-OFF 模型**: TCP 流量可以用 ON-OFF 模型描述, 其中 ON 状态对应数据传输, OFF 状态对应等待。如果 ON 和 OFF 的持续时间服从重尾分布, 聚合流量将表现出自相似特性。

(4) **流体模型**: 在分析大规模网络时, 可以将离散的数据包流近似为连续的流体流。设发送速率为 $x(t)$, 队列长度为 $q(t)$, 链路容量为 C , 则流体模型为:

$$\dot{q}(t) = x(t) - C, \quad \text{当 } q(t) > 0 \text{ 或 } x(t) > C \quad (2.14)$$

Swift 算法的设计考虑了流量的突发性特征。当前实现优先使用显式丢包回调和 ECN 状态作为拥塞判定信号, 并将 RTT 膨胀用于 α 参数自适应调节, 从而在突发跨流量和路径 RTT 波动下避免仅依赖单一信号作出窗口决策。

2.5.2 排队论分析

排队论为分析网络缓冲区行为提供了理论工具。考虑单个瓶颈链路, 可以建模为 M/M/1 队列 (泊松到达、指数服务时间、单服务器) 或更一般的 G/G/1 队列。

M/M/1 队列的稳态性质：设到达率为 λ ，服务率为 μ ，负载为 $\rho = \lambda/\mu$ ($\rho < 1$ 时队列稳定)，则：

平均队列长度：

$$\bar{Q} = \frac{\rho}{1 - \rho} \quad (2.15)$$

平均等待时间 (Little 公式)：

$$\bar{W} = \frac{\bar{Q}}{\lambda} = \frac{1}{\mu - \lambda} \quad (2.16)$$

队列长度分布：

$$P(Q = k) = (1 - \rho)\rho^k, \quad k = 0, 1, 2, \dots \quad (2.17)$$

从排队论角度，Swift 的低延迟特性源于其维持较低的平均负载 ρ 。通过对 ECN 信号的敏感响应，Swift 在 ρ 接近 1 之前就降低发送速率，从而保持 $\bar{Q}$ 和 $\bar{W}$ 在较低水平。

ECN 标记概率：设队列长度为 Q ，ECN 标记阈值为 K ，则在 RED (Random Early Detection) 策略下：

$$P(\text{标记}|Q) = \begin{cases} 0 & Q < \text{min_th} \\ \frac{Q - \text{min_th}}{\text{max_th} - \text{min_th}} & \text{min_th} \leq Q < \text{max_th} \\ 1 & Q \geq \text{max_th} \end{cases} \quad (2.18)$$

Swift 对 ECN 的利用采用直接状态检测机制：当 `ecnState` 为 `ECN_CE_RCVD` 或 `ECN_ECE_RCVD` 时，判定存在 ECN 拥塞信号，触发保留因子为 0.75 的早期预警窗口缩减。这种直接检测方式避免了基于事件率的间接测量，具有更低的响应延迟。

2.5.3 控制论视角

从控制论角度，TCP 拥塞控制可以建模为反馈控制系统。系统框图包括：

- 被控对象：网络（包括链路、路由器、队列）
- 控制器：拥塞控制算法
- 反馈信号：ACK、RTT、ECN 标记、丢包
- 控制输入：发送窗口或发送速率
- 干扰：跨流量、背景噪声

线性化分析：在稳态点附近，TCP 系统可以线性化。设稳态发送速率为 x^* ，稳态队列长度为 q^* ，令 $\delta x = x - x^*$ ， $\delta q = q - q^*$ ，则线性化系统为：

$$\dot{\delta q} = \delta x \quad (2.19)$$

$$\delta x(t) = -K \cdot \delta q(t - \tau) \quad (2.20)$$

其中 K 是控制增益， τ 是反馈延迟（约等于 RTT）。

系统的特征方程为：

$$s + K e^{-s\tau} = 0 \quad (2.21)$$

稳定性条件（小增益定理）：

$$K\tau < \frac{\pi}{2} \quad (2.22)$$

这意味着在高延迟网络中，控制增益 K 必须足够小以保证稳定性，但过小的 K 会导致收敛缓慢。Swift 的自适应参数调优机制可以理解为根据 τ （通过 RTT 测量）动态调整 K （通过 α 参数）。

时滞系统分析：网络的反馈延迟（RTT）使得拥塞控制成为时滞系统。时滞系统的稳定性分析比常微分方程系统更复杂。Swift 通过综合使用多种拥塞信号，减少了对单一信号（如 RTT）的依赖，提高了系统的鲁棒性。

2.6 学习型智能决策方法在网络中的应用综述

学习型智能决策方法（含强化学习）在网络领域的应用日益广泛。本节简要综述其在路由优化、资源分配、安全防护等方面的研究进展，用于界定学习型方法的一般能力边界，并与本文的白盒启发式设计形成对照。

2.6.1 路由优化

强化学习可用于路由策略搜索。Q-routing 将路由器建模为智能体，根据观测更新到各目的地的下一跳估计；DQN 等方法也被用于软件定义网络中的流量工程。所得策略是否最优取决于状态、动作、奖励和训练条件。

拥塞控制通常按每流、每确认或固定短周期更新，因而对决策时延和运行开销较敏感。Swift 据此选择规则控制，以便直接检查每次窗口更新；这一设计选择不意味着神经网络方法不能满足相同时间尺度。

2.6.2 资源分配

在云计算和边缘计算中，强化学习可用于虚拟机放置、容器调度和带宽分配。此类任务与拥塞控制的状态、动作和决策周期不同，具体时延和训练方式取决于实现，不能一概归为秒级到分钟级。

2.6.3 安全防护

强化学习在网络安全领域也有应用，如入侵检测、DDoS 防护等。与安全应用不同，拥塞控制需要处理合法流量，不能简单地丢弃可疑流量，需要更精细的控制策略。

2.6.4 与 Swift 的关系

Swift 不采用学习型决策闭环：策略本身是离线设计的确定性规则，不存在训练、在线探索或模型更新环节；学习型方法中以反馈信号引导参数调整的思路，在 Swift 中表现为基线相对的反馈自适应——决策主体始终是显式规则，反馈仅以快/慢指数移动平均之差微调乘性增加因子 α 。与之相比，学习型策略的可扩展表示能力是 Swift 不具备的，但其训练依赖、推理开销与不可审计性也正是 Swift 通过白盒规则设计所规避的问题。两类方法的设计取舍与对比详见第五章。

表2.1从多个维度对比了上述代表性拥塞控制算法与 Swift 的技术特性。

表 2.1 代表性拥塞控制算法综合对比

特性	NewReno	CUBIC	BBR	DCTCP	Swift
主要控制依据	丢包	丢包	带宽/最小 RTT 模型	ECN 比例	多信号状态
ECN 处理	TCP 通用机制	TCP 通用机制	TCP 通用机制	标记比例响应	状态直接判定
差异化响应	否	否	状态机增益循环	连续比例	三类保留因子
参数变化方式	固定规则	固定规则	固定状态机	固定规则	反馈驱动调节
BDP 估计	否	否	是	否	是
每流状态管理	是	是	是	是	是
决策表示	AIMD 规则	三次窗口函数	模型与增益循环	ECN 比例公式	显式启发式规则
训练需求	无	无	无	无	无

2.7 本章小结

本章综述了基于丢包、基于时延、模型驱动、ECN 辅助和数据驱动的拥塞控制方法。不同方法在信号时效性、参数适用范围、竞争公平性、训练需求和决策可解释性方面存在不同取舍，相关结论需结合具体实现与网络配置判断。

在此基础上，Swift 的技术主线归纳为三点：将 ECN 与拥塞状态机语义纳入多信号状态感知；以时间窗口交付率、最小 RTT 和基线相对反馈构成白盒启发式控制；通过差异化响应、窗口边界、连续缩减保护和降窗后冻结限制不稳定行为。其目标是在远距离、多接入与异构终端应用条件下改善吞吐，同时控制时延和窗口波动。

第三章 Swift 算法设计与实现

本章详细介绍 Swift 算法的设计与实现，包括系统架构概述、决策问题建模与设计原则、11 维观测空间设计、多信号融合拥塞检测算法、差异化拥塞响应机制、启发式反馈自适应的参数调优、融合控制策略以及每流独立状态管理机制。

3.1 系统架构概述

Swift 算法的系统架构如图3.1所示，由四个核心模块组成：网络仿真模块、拥塞控制环境模块、智能决策模块和进程间通信模块。

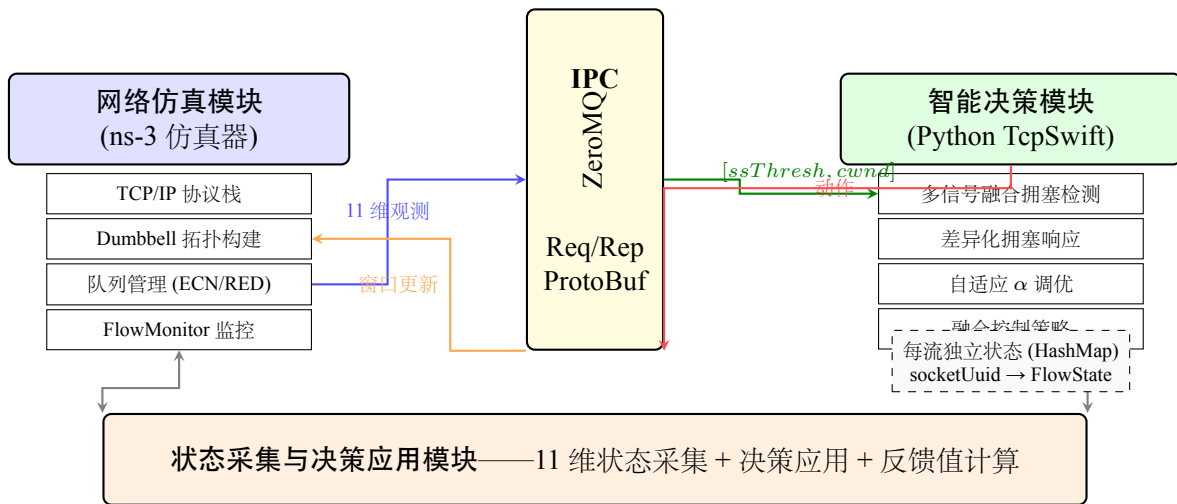


图 3.1 Swift 系统架构

网络仿真模块基于 ns-3 (Network Simulator 3) 离散事件仿真平台构建, 实现完整的 TCP/IP 协议栈。ns-3 是目前学术界和工业界广泛使用的网络仿真工具, 提供了丰富的网络协议模型和灵活的可扩展架构。网络仿真模块负责构建哑铃型等可控拓扑、模拟数据包传输过程、执行离散事件驱动的仿真调度, 并通过 FlowMonitor 等工具记录吞吐量、时延、丢包和公平性指标。

主动队列管理与 ECN 信号通路。仿真器 contrib/opengym/examples/swift-tcp/sim.cc 将瓶颈两侧网关的默认队列规则设为 ns3::RedQueueDisc, 并按队列容量的比例配置标记门限: $MinTh = 0.3 \times Q$ 、 $MaxTh = 0.9 \times Q$, 其中 $Q = \max(BDP \times n_{leaf} / MTU, 100)$ 为按带宽延迟积自动定尺的队列长度 (单位为数据包)。同时置 RedQueueDisc::UseEcn 为真, 使 RED 在队列长度进入 $[MinTh, MaxTh]$ 区间时对 ECT 数据包置 CE 码点而非直接丢弃。

若瓶颈使用不设置 CE 码点的 PfifoFastQueueDisc, 则协议栈不能从该队列获得

ECN 标记, 第 3.4 节所述 ECN 分支也无法由瓶颈标记触发。因此本文的参考配置选用 RED 并开启 ECN 标记; 第四章所引用的实验产物均在该配置下生成。`ns3::TcpSocketBase::UseEcn` 被统一置为 On 并作用于四种被比较协议, 以控制 ECN 协商能力这一实验变量。

Swift 决策进程与 ns-3 仿真进程经 ZeroMQ 同步请求—应答路径交互: 仿真进程传输包含 4 项元数据和 11 项有效状态的 15 元素容器, 决策进程返回慢启动阈值与拥塞窗口决策; 状态与动作的跨进程传输沿用 ns3-gym/OpenGym 框架提供的容器结构, 仅作为消息通道, 不含学习逻辑。由于该路径依赖同步消息交互, 本文未将其与 MTP 多线程并行仿真机制结合使用。

状态采集与决策应用模块 (环境层) 负责在 TCP 协议栈与决策进程之间建立桥梁: 五个 TCP 拥塞控制回调更新观测状态, 其中 `GetSsThresh()` 与 `IncreaseWindow()` 触发同步决策交互; 模块随后将返回的 `ssthresh` 与 `cwnd` 决策应用于 TCP 连接。

拥塞控制环境模块实现了以下回调接口:

- `GetSsThresh()`: 慢启动阈值获取回调, 在协议栈请求窗口缩减参数时触发同步决策, 具体原因需结合 CA 与 ECN 状态判定
- `IncreaseWindow()`: 窗口增加回调, 在收到正常确认时触发同步决策
- `PktsAcked()`: 数据包确认回调, 用于更新最近 RTT 与已确认报文段数量
- `CongestionStateSet()`: 拥塞状态设置回调, 用于处理状态迁移及超时时的陈旧动作作废
- `CwndEvent()`: 拥塞窗口事件回调, 用于更新 ECN 事件状态

智能决策模块采用 Python 实现, 负责执行 Swift 算法的核心逻辑: 多信号融合拥塞检测、差异化拥塞响应、自适应参数调优和融合控制策略计算。智能决策模块维护每个 TCP 连接的独立状态, 包括历史度量缓冲区、拥塞事件计数、性能反馈状态和自适应参数。

进程间通信模块采用 ZeroMQ (ZMQ) 消息队列实现仿真进程和决策进程之间的同步通信。通信协议采用请求-应答(Request-Reply)模式: 仿真进程在 `GetSsThresh()` 或 `IncreaseWindow` 回调中发送观测, 决策进程返回控制动作; 其余回调用于更新下一次观测所需的环境状态。消息格式采用 Protocol Buffers 序列化。

3.2 决策问题建模与设计原则

Swift 把网络拥塞控制刻画为逐事件决策问题: 在每个 ACK 确认或拥塞事件触发的回调上, 决策模块以当前状态向量 s 为输入, 按照确定性规则直接输出新的慢启动阈值与拥塞窗口决策 $\langle ssthresh, cwnd \rangle$, 并把由环境按传输进度与排队/丢包代价合成的性能反馈值 f_t 用于

参数层的慢速自适应。该”状态—决策—反馈”三元结构不涉及策略学习：规则是离线设计的，反馈仅用于乘性增加因子等参数的小步调整，不存在训练、探索或模型更新环节。决策结构的合理性由以下四组设计原则保证。

3.2.1 状态向量设计原则

状态向量的设计决定了一次回调能够携带多少决策所需信息。良好设计的判定标准包括：

状态信息覆盖：状态应包含区分不同网络状况所需的信息。信息不足会使决策模块难以区分需要不同响应的情形，例如”ECN 标记触发的收缩”与”普通丢包”。Swift 的 11 维有效状态纳入 ECN 状态、拥塞控制状态机状态、拥塞事件、RTT 与窗口度量，为三类拥塞语义判定提供输入，但不声称覆盖协议栈的全部可观测状态。

可观测性 (Observability)：状态应能够从环境中直接获得。Swift 的 15 个容器字段均由 ns-3 TCP 拥塞控制回调提供或由仿真环境生成，无需额外探测流量。

维度可控性 (Dimensional Controllability)：状态维度应保持在规则可维护的范围内。Swift 的 11 维有效状态配合基于规则的决策，单次处理涉及固定规模的字段读取、滑动窗口更新与比较；当前窗口容量固定，因此相对于仿真时长，其时间开销为常数级。

3.2.2 决策输出设计

决策输出决定了窗口控制的操作语义。Swift 直接输出新的 $ssthresh$ 与 $cwnd$ 绝对值而非窗口增量或离散档位，理由如下：

控制粒度 (Control Granularity)：直接输出窗口绝对值可将目标窗口（如 $\alpha \times BDP$ ）映射到协议状态，避免离散动作（如”增加 10%”）引入的额外量化。

绝对值语义 (Absolute Semantics)：每步决策都以协议栈当前状态为基准重新计算；若待应用窗口因协议栈进入 CA_LOSS 而失效，后续事件将基于更新后的状态重新计算。

边界约束 (Bound Constraints)：Python 控制器将拥塞窗口限制在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 区间；环境侧在应用动作前另以 $2 \times MSS$ 校验拥塞窗口和慢启动阈值的下界。二者共同限制当前实现中的窗口取值，但不替代对跨进程输入合法性的完整验证。

3.2.3 性能反馈值设计

环境侧在每个事件上合成一个标量性能反馈值 f_t ，供参数层的启发式自适应使用。其设计遵循三条原则：

即时性 (Immediacy)：每个确认事件与每个窗口收缩回调都产生反馈，不存在稀疏反馈造成的信号缺失问题。

多目标折中 (Multi-objective Trade-off)：反馈由传输进度激励与代价惩罚两部分合成：正常确认事件上的反馈为”与确认段数成正比且有上限的吞吐激励”减去”RTT 膨胀惩罚”；窗口收缩回调上按拥塞类型施加分级负向惩罚（ECN、丢包、超时的惩罚幅度递增），使反馈同时携带吞吐收益与排队/丢包代价信息。

相对性 (Relativity)：由于逐事件反馈几乎恒为正，绝对幅值不携带调节信息；Swift 只使用反馈相对其自身慢速基线的变化量驱动 α 微调（详见第3.6节），从而避免固定绝对阈值导致的单向漂移，并使同一组超参数可以跨带宽与 RTT 差异极大的路径复用。

3.2.4 规则策略表示

Swift 的决策策略采用确定性规则表示，而非随机策略或神经网络策略。这种选择的考量如下：

确定性：给定相同的状态向量，决策输出完全确定，行为可逐分支解释、可离线推演，实验可精确复现。

参数化 vs. 非参数化：神经网络策略是参数化策略，表示能力强但依赖训练与推理设施；基于规则的策略计算开销为常数级、无推理依赖，表示能力受人工规则结构限制。Swift 选择后者，以可解释性换取工程可控性，其策略表示能力的局限与可能的增强路线见第五章。

离线设计 + 在线参数自适应：规则主体离线设计，不经过在线学习；控制参数（乘性增加因子 α 等）在运行期由反馈与 RTT 信号小步在线调整，使算法在保持确定性主路径的同时获得对环境变化的适应能力。

3.3 11 维观测空间详细设计

本节详细描述 Swift 算法的 11 维观测空间设计，这是 Swift 区别于现有方案的核心创新之一。

记法约定：为与 ns-3 侧 OpenGym 实现 (contrib/opengym/examples/swift-tcp/tcp-swift-env.cc 中 parameterNum = 15) 保持一致，每次回调实际通过跨进程传输容器传递 15 个元素，其中前 4 个 (socketUuid、envType、simTime、nodeId) 为跨进程路由用的元数据通道，不参与 AIMD/ECN 响应/ α 自适应等窗口决策；后 11 个 (ssthresh、cwnd、segSize、segmentsAacked、bytesInFlight、lastRtt、minRtt、callbackType、caState、caEvent、ecnState) 构成参与决策的有效状态子空间。故下文所称”11 维观测空间”特指该有效子空间；表 3.1 同时列出元数据通道以便审阅者核对完整布局。

表 3.1 Swift 11 维观测空间参数定义

索引	参数名称	数据类型	说明
0	socketUuid	uint64	套接字唯一标识符
1	envType	uint64	环境类型 (0= 事件驱动, 1= 时间驱动)
2	simTime	uint64	仿真时间 (微秒)
3	nodeId	uint64	节点标识符
4	ssthresh	uint64	慢启动阈值 (字节)
5	cwnd	uint64	拥塞窗口大小 (字节)
6	segSize	uint64	TCP 报文段大小 (字节)
7	segmentsAcked	uint64	本次确认的报文段数量
8	bytesInFlight	uint64	在途字节数
9	lastRtt	uint64	最近往返时延 (微秒)
10	minRtt	uint64	最小往返时延 (微秒)
11	callbackType	uint64	回调函数类型
12	caState	uint64	拥塞算法状态
13	caEvent	uint64	拥塞算法事件
14	ecnState	uint64	ECN 状态

3.3.1 连接标识参数 (索引 0、3)

socketUuid (索引 0) 是套接字的唯一标识符, 用于区分不同的 TCP 连接。在多流并发场景下, 每个流需要维护独立的状态信息, **socketUuid** 是实现每流状态隔离的关键。**socketUuid** 在连接建立时由系统分配, 在连接生命周期内保持不变。

nodeId (索引 3) 是网络节点的标识符, 用于区分不同的发送端节点。在多节点仿真场景下, 不同节点可能运行不同的拥塞控制算法或具有不同的网络条件, **nodeId** 用于识别当前决策所属的节点。

3.3.2 时间信息参数 (索引 1、2)

envType (索引 1) 是环境类型标识符。当前事件驱动实现中该字段固定为 0, 用于标识状态采集来自 TCP 回调事件路径。

simTime (索引 2) 是当前仿真时间, 单位为微秒。仿真时间用于标识观测发生的事件顺序, 并为跨进程观测和动作交互提供时间上下文。

3.3.3 窗口状态参数 (索引 4、5)

ssthresh (索引 4) 是慢启动阈值 (Slow Start Threshold), 单位为字节。**ssthresh** 是 TCP 拥塞控制的核心状态变量之一, 用于区分慢启动阶段和拥塞避免阶段: 当 $cwnd < ssthresh$ 时处于慢启动阶段, 窗口指数增长; 当 $cwnd \geq ssthresh$ 时进入拥塞避免阶段, 窗口线性增长。

cwnd (索引 5) 是拥塞窗口 (Congestion Window) 大小, 单位为字节。**cwnd** 是 TCP 拥塞控

制的另一核心状态变量，决定了发送端在未收到确认的情况下最多可以发送的数据量。*cwnd* 的调整是拥塞控制的核心任务。

3.3.4 传输性能参数（索引 6、7、8）

segSize（索引 6）是 TCP 报文段大小（Maximum Segment Size, MSS），单位为字节。本文仿真器按 $mtu_bytes - 20 - (IPv4 \text{ 首部} + TCP \text{ 首部})$ 计算应用层数据单元，在 MTU 为 1500 字节、IPv4 与 TCP 首部各 20 字节的默认配置下得到 1440 字节，其中额外扣除的 20 字节为仿真器为 TCP 选项字段预留的余量。**segSize** 用于将以字节为单位的窗口度量换算为以报文段为单位的度量，并且是慢启动增量、加性递增步长以及窗口上下界的共同基准单位，因此该值的准确性直接决定全部窗口计算的量纲一致性。

segmentsAcked（索引 7）是本次确认的报文段数量。在收到累积确认（Cumulative ACK）或选择性确认（Selective ACK, SACK）时，**segmentsAcked** 表示新确认的报文段数量。**segmentsAcked** 用于计算窗口增长量，例如在慢启动阶段，窗口增长量为 $segmentsAcked \times MSS$ 。

bytesInFlight（索引 8）是在途字节数（Bytes in Flight），即已发送但尚未确认的数据量。**bytesInFlight** 用于计算管道利用率（Pipeline Utilization）：

$$utilization = \frac{bytesInFlight}{cwnd} \quad (3.1)$$

当 *utilization* 较低时，表明发送端未能充分利用拥塞窗口，可能是因为应用层数据不足或接收端通告窗口限制。

3.3.5 延迟测量参数（索引 9、10）

lastRtt（索引 9）是最近测量的往返时延（Round-Trip Time），单位为微秒。RTT 是端到端延迟的直接度量，包括传播延迟（Propagation Delay）、排队延迟（Queuing Delay）、处理延迟（Processing Delay）和传输延迟（Transmission Delay）四个组成部分。

minRtt（索引 10）是历史最小往返时延，单位为微秒。**minRtt** 作为路径传播时延的保守下界，用于估计带宽延迟积（BDP）和计算 RTT 膨胀程度。RTT 膨胀比定义为：

$$inflation_ratio = \frac{lastRtt}{minRtt} \quad (3.2)$$

当 *inflation_ratio* 接近 1 时，表明网络排队延迟较小；当 *inflation_ratio* 较大时，表明网络存在显著的排队延迟。

3.3.6 回调类型参数（索引 11）

callbackType（索引 11） 标识当前回调的类型，用于区分不同类型的事件。定义如下：

- 值为 0：GetSsThresh 回调，表示协议栈请求更新慢启动阈值，触发原因需结合 caState 与 ecnState 区分
- 值为 1：IncreaseWindow 回调，表示收到正常确认并进入窗口调整路径

callbackType 是多信号融合拥塞检测的关键输入之一。当 callbackType 为 0 时，可以确认协议栈正在执行窗口缩减流程，但不能仅凭该字段把原因判定为普通丢包。

3.3.7 拥塞控制状态参数（索引 12、13、14）

caState（索引 12） 是拥塞算法状态（Congestion Algorithm State），反映 TCP 拥塞控制状态机的当前状态。Linux TCP 实现定义了以下状态：

- CA_OPEN（值为 0）：开放状态，正常传输，无拥塞迹象
- CA_DISORDER（值为 1）：无序状态，收到重复确认或乱序确认
- CA_CWR（值为 2）：拥塞窗口缩减状态，响应 ECN 标记
- CA_RECOVERY（值为 3）：恢复状态，快速恢复过程中
- CA_LOSS（值为 4）：丢失状态，检测到超时丢包

caEvent（索引 13） 是拥塞算法事件（Congestion Algorithm Event），表示最近发生的拥塞相关事件。Linux TCP 实现定义了以下事件：

- CA_EVENT_TX_START（值为 0）：传输开始事件
- CA_EVENT_CWND_RESTART（值为 1）：拥塞窗口重启事件
- CA_EVENT_COMPLETE_CWR（值为 2）：CWR 完成事件
- CA_EVENT_LOSS（值为 3）：丢失事件
- CA_EVENT_ECN_NO_CE（值为 4）：无 CE 标记的 ECN 事件
- CA_EVENT_ECN_IS_CE（值为 5）：有 CE 标记的 ECN 事件

ecnState (索引 14) 是显式拥塞通知状态 (ECN State), 反映 ECN 协商和标记的状态。定义如下:

- ECN_DISABLED (值为 0): ECN 禁用
- ECN_IDLE (值为 1): ECN 空闲
- ECN_CE_RCVD (值为 2): 收到 CE 标记
- ECN_SENDING_ECE (值为 3): 正在发送 ECE 标志
- ECN_ECE_RCVD (值为 4): 收到 ECE 标志
- ECN_CWR_SENT (值为 5): 已发送 CWR 标志

caState、caEvent 与 ecnState 为拥塞类型判定提供协议栈语义。Swift 将这些字段与窗口、在途字节和 RTT 观测结合, 用于区分超时、ECN 驱动的窗口缩减与普通丢包; 该观测集合面向当前控制规则设计, 不代表覆盖 TCP 协议栈的全部状态。

3.4 多信号融合拥塞检测算法

Swift 算法的核心创新之一是多信号融合拥塞检测方法。该方法综合利用丢包回调、ECN 状态和 TCP 拥塞状态机信息, 采用显式信号优先级判断拥塞类型, 并将 RTT 膨胀保留给参数自适应调节, 而不把 RTT 膨胀作为独立降窗触发条件。

3.4.1 拥塞信号分类

Swift 将拥塞信号分为四个类别, 每一类信号在”发生时刻—可靠性—语义粒度”三个维度上具有不同特征, 这构成了后续差异化响应的判据基础。

(1) 显式降窗信号: 当 callbackType 为 0 (GetSsThresh 回调) 时, 表明 ns-3 的 TCP 协议栈已经自行判定需要缩减发送窗口并向拥塞控制算法索取新的慢启动阈值。该回调是协议栈内部唯一的”权威降窗入口”, 其触发条件包含快速重传检测到的丢包、重传超时以及 ECN 回显驱动的 CWR 进入三种情形, 因此不能笼统地等同于”丢包”。Swift 在该回调内部再依据 caState 和 ecnState 做二次判别, 这是本文与既有方案在信号解释粒度上的关键差异。

(2) ECN 拥塞经历信号: 当 ecnState 取值为 ECN_CE_RCVD (值为 2, 本端收到被标记 CE 的数据包) 或 ECN_ECE_RCVD (值为 4, 本端收到对端回显的 ECE 标志) 时, 表明路径上的队列长度已经越过 AQM 的标记门限, 但尚未溢出。ECN 是唯一由网络设备主动生成、且不依赖丢包或 RTT 推断的拥塞证据, 其时序显著早于丢包, 因此适合作为早期预警。Swift

直接读取协议栈的 ECN 状态字段，而不维护标记事件的时间戳队列与标记率统计，从而避免了标记率阈值这一额外的敏感参数。

(3) 拥塞控制状态机信号：caState 刻画了协议栈当前所处的恢复阶段。其中 CA_LOSS (值为 4) 指示重传超时，是严重程度最高的信号；CA_CWR (值为 2) 指示协议栈正因 ECN 回显而缩减窗口，在语义上属于 ECN 响应而非丢包响应；CA_DISORDER 与 CA_RECOVERY 则指示乱序或快速恢复过程，不构成独立的降窗触发条件。将 caState 纳入状态向量使决策模块能够在同一个 GetSsThresh 回调内区分”超时””ECN 驱动的 CWR”与”普通丢包”三条语义完全不同的路径。

(4) 延迟膨胀信号：RTT 膨胀比 (lastRtt/minRtt) 反映瓶颈队列的驻留程度。该信号连续可微、采样密度高，适合驱动参数的渐进调节；但它同时混杂了无线调度抖动、ACK 压缩、路径切换和终端侧处理延迟等非拥塞因素，在远距离与异构终端场景中误判概率明显高于前三类信号。因此 Swift 不将其用作独立的乘性递减触发条件，而仅用于 α 自适应与慢启动退出判定。

3.4.2 分层优先级机制

不同类型的拥塞信号具有不同的可靠性和紧迫程度，Swift 采用分层优先级机制进行处理：

第一优先级：显式降窗回调的三路语义判别

Algorithm 1 显式降窗回调的三路语义判别

Require: 回调类型 *callbackType*, 拥塞算法状态 *caState*, ECN 状态 *ecnState*

```

1: if callbackType = 0 then                                ▷ GetSsThresh 回调
2:   is_congested  $\leftarrow$  True
3:   if caState = CA_LOSS then
4:     congestion_type  $\leftarrow$  TIMEOUT                                ▷ 重传超时
5:   else if ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} or caState = CA_CWR
   then
6:     congestion_type  $\leftarrow$  ECN                                ▷ ECN 回显驱动的 CWR
7:   else
8:     congestion_type  $\leftarrow$  LOSS                                ▷ 快速重传丢包
9:   end if
10:  return (is_congested, congestion_type)
11: end if

```

GetSsThresh 回调本身只说明”协议栈决定降窗”，并不说明降窗的原因。ns-3 的 TcpSocketBase 在三种情形下都会进入该回调：快速重传确认丢包、重传定时器超时（此时 caState 被置为 CA_LOSS）、以及收到 ECE 回显后调用 EnterCwr（此时 caState 被置为 CA_CWR）。若不加区分地把整个回调映射为”丢包”，则 ECN 提供的早期预警在响应环节会被当作队列已

溢出处理，从而抵消 ECN 机制本应带来的时序优势——这正是本算法早期版本存在的语义缺陷。当前实现在回调内部按”超时优先、ECN 次之、普通丢包兜底”的顺序完成三路判别：CA_LOSS 对应最严重的路径级异常；ecnState 处于 CE_RCVD 或 ECE_RCVD、或 caState 已进入 CA_CWR 的情形归入 ECN 类；其余情形归入普通丢包类。三类判别结果分别驱动 0.50、0.75、0.70 三个窗口保留因子，使响应力度与信号严重程度保持单调一致。

第二优先级：ECN 拥塞经历信号检测

Algorithm 2 ECN 拥塞经历信号检测

Require: ECN 状态 *ecnState*

```

1: if ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} then
2:   is_congested  $\leftarrow$  True
3:   congestion_type  $\leftarrow$  ECN
4:   return (is_congested, congestion_type)
5: end if

```

当 ecnState 为 ECN_CE_RCVD（收到 CE 标记）或 ECN_ECE_RCVD（收到 ECE 回显）时，判定发生 ECN 拥塞。ECN 拥塞采用早期预警响应策略，当前实现的保留因子为 0.75；该值比普通丢包响应更温和，同时能够更及时释放队列压力。

3.4.3 信号优先级与误判抑制策略

Swift 的拥塞检测采用显式信号优先级，而不是将所有协议栈状态都作为独立拥塞触发条件。当前实现中，显式丢包回调具有最高优先级；ECN CE/ECE 状态是第二优先级的早期拥塞信号；RTT 膨胀、CA_CWR 和 CA_RECOVERY 等状态不直接触发降窗，而用于解释网络状态或影响后续参数调节。

(1) 显式丢包优先：当回调类型为 GetSsThresh 时，算法认为 TCP 协议栈已经进入丢包相关处理路径，并根据 CA 状态区分普通丢包和超时丢包。

(2) ECN 直接触发早期响应：当 ECN 状态为 ECN_CE_RCVD 或 ECN_ECE_RCVD 时，算法将其视为有效早期拥塞信号并触发 ECN 响应，同时累加 ECN 计数器用于后续状态分析。

(3) RTT 膨胀不单独触发降窗：RTT 膨胀容易受到无线调度、ACK 压缩和路径抖动影响。当前实现将其用于 α 自适应调节和慢启动退出判断，而不是作为独立拥塞事件直接执行乘性递减。

(4) 降窗后冻结抑制反弹：每次显式拥塞响应后，算法冻结后续若干 ACK 事件中的窗口增长，避免刚降窗后立即增窗造成二次排队。

3.4.4 完整拥塞检测算法

综合上述机制，Swift 的完整多信号融合拥塞检测算法如下：

Algorithm 3 多信号融合拥塞检测算法

Require: 观测状态 *obs*, 流状态 *state*

Ensure: 是否拥塞 *is_congested*, 拥塞类型 *type*

```

1:                                     ▷ 第一优先级：显式降窗回调的三路语义判别
2: if obs.callbackType = 0 then                                     ▷ GetSsThresh 回调
3:   if obs.caState = CA_LOSS then
4:     state.loss_count  $\leftarrow$  state.loss_count + 1
5:     return (True, TIMEOUT)                                     ▷ 重传超时
6:   end if
7:   if obs.ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} or obs.caState =
   CA_CWR then
8:     state.ecn_count  $\leftarrow$  state.ecn_count + 1
9:     return (True, ECN)                                     ▷ ECN 回显驱动的 CWR
10:  end if
11:   state.loss_count  $\leftarrow$  state.loss_count + 1
12:   return (True, LOSS)                                     ▷ 快速重传丢包
13: end if
14:                                     ▷ 第二优先级：IncreaseWindow 路径上的 ECN 状态巡检
15: if obs.ecnState  $\in$  {ECN_CE_RCVD, ECN_ECE_RCVD} then
16:   state.ecn_count  $\leftarrow$  state.ecn_count + 1
17:   return (True, ECN)
18: end if
19:                                     ▷ 未检测到拥塞
20: return (False, NONE)

```

该算法的时间复杂度为 $O(1)$ ，仅涉及常数次整型比较，不引入任何队列扫描或统计窗口维护开销，因此可以在每个 ACK 事件上无条件执行。算法的两级结构也决定了拥塞类型判定的互斥性：一次回调最多产生一个拥塞类型，不存在多个信号同时触发导致窗口被重复缩减的情形。

3.5 差异化拥塞响应机制

Swift 算法针对不同类型的拥塞信号采用差异化的响应策略，核心是差异化窗口保留因子 (Window Retention Factor) 机制。

3.5.1 窗口保留因子设计

窗口保留因子 ρ 定义为拥塞响应后新窗口与原窗口的比值：

$$new_wnd = \rho \cdot cwnd \quad (3.3)$$

传统算法（如 TCP Reno、CUBIC）对所有拥塞信号采用统一的窗口保留因子（通常为 0.5，即窗口减半）。这种”一刀切”的策略无法区分不同类型和程度的拥塞，在轻度拥塞时响应过度，在严重拥塞时响应不足。

Swift 根据拥塞类型设计了差异化的窗口保留因子：

表 3.2 差异化窗口保留因子

拥塞类型	保留因子	设计理由
丢包（Loss）	0.70	丢包是严重拥塞信号，需要较大幅度降低窗口
ECN 标记（ECN）	0.75	ECN 是早期信号，提前释放队列压力
超时丢失（Timeout）	0.50	超时表示最严重拥塞，需最大幅度窗口缩减

3.5.2 差异化响应的理论依据

差异化窗口保留因子的设计基于以下理论分析：

(1) 拥塞信号的时序特性

ECN 标记是最早的拥塞信号，通常在队列长度超过低阈值时触发，此时网络可能只是轻度拥塞。丢包是较晚的拥塞信号，通常在队列溢出时发生，表明网络已经严重拥塞。超时丢包是最晚也最严重的拥塞信号，通常意味着多个数据包连续丢失或网络路径严重中断，需要最大力度的窗口缩减。

根据时序关系和严重程度，Swift 设计了三级差异化响应：ECN 标记采用早期预警响应（保留因子 0.75），丢包采用较激进的响应（保留因子 0.70），超时采用最激进的响应（保留因子 0.50）。这种阶梯式的响应力度设计用于匹配不同严重程度的拥塞信号，同时通过降窗后冻结机制抑制窗口立即反弹。

(2) 信号可靠性分析

丢包通常表示报文未被成功交付，但在无线或切换场景中不一定由队列拥塞引起；ECN 标记的语义则取决于网络设备是否启用标记及其阈值配置。因此，当前实现对普通丢包采用 0.70 的保留因子，对 ECN 采用 0.75 的保留因子，并把超时作为更严重的恢复状态处理。这些取值属于人工设定，其独立效果仍需消融实验检验。

(3) 吞吐量-延迟权衡

较低的窗口保留因子能够更快地缓解拥塞、降低延迟，但可能降低后续吞吐量。较高的窗口保留因子有利于维持在途数据量，但可能延长拥塞持续时间。Swift 的差异化设计用于在这两类效应之间进行折中，其具体效果由第四章的整体实验结果讨论，不作最优性断言。

3.5.3 慢启动阈值更新

除了拥塞窗口，拥塞响应还需要更新慢启动阈值。当前实现首先以 $\rho \times cwnd$ 计算新的拥塞窗口，并设置最小窗口边界 $W_{min} = \max(4 \times MSS, 1)$ 。慢启动阈值采用 BDP 锚定但不迁就队列膨胀的策略：

$$ref = \min(cwnd, \max(BDP, W_{min})) \quad (3.4)$$

$$new_ssthresh = \max(\rho \times ref, W_{min}, new_cwnd) \quad (3.5)$$

该设计避免在队列已膨胀时把慢启动阈值设置到过高水平，同时保证阈值不低于新的拥塞窗口，防止拥塞恢复阶段出现窗口停滞。

此外，Swift 引入连续缩减保护和降窗后冻结两项稳定性机制，二者作用于不同的时间尺度且相互配合。

连续缩减保护针对的是“窗口死亡螺旋”：在突发拥塞或无线批量误码期间，协议栈可能在很短时间内连续触发多次降窗回调，若每次都执行乘性递减，窗口将以指数速度塌缩至下界，随后需要数十个 RTT 才能重新填满管道。算法为每条流维护连续缩减计数器 d ，每次执行拥塞响应时 d 自增；当 d 超过 $D_{max} = 3$ 时，响应退化为“保持当前窗口”而不再继续缩减。取 $D_{max} = 3$ 的依据是：连续三次丢包响应后窗口仅余 $0.70^3 \approx 34.3\%$ ，已经低于传统算法单次窗口减半的结果，继续缩减的边际收益迅速递减而恢复代价持续上升。

降窗后冻结针对的是“降窗即反弹”：乘性递减只减小了发送窗口，瓶颈队列中已经排队的数据仍需一段时间才能排空；若在此期间立即恢复窗口增长，新注入的数据会与残留队列叠加，导致排队延迟不降反升，并可能诱发第二次拥塞事件。因此任一显式降窗后，算法冻结后续 4 个 ACK 事件中的窗口增长，为队列排空留出时间窗口。

两项机制的交互存在一个容易被忽视的细节：连续缩减计数器 d 的清零时机。当前实现只在“窗口确实发生增长”时才把 d 置零，即清零动作位于冻结判断之后；若把清零动作前置到冻结判断之前，则冻结期内每个被冻结的 ACK 事件都会误将 d 清零， D_{max} 计数被反复复位，连续缩减保护实际失效——这恰是本算法早期版本存在的语义缺陷。将清零动作与真实的窗口增长绑定，保证了“只有当算法确认网络已经恢复到可以增窗的状态时，才认为连续拥塞序列已经结束”。

Algorithm 4 差异化拥塞响应算法**Require:** 当前窗口 $cwnd$, 当前阈值 $ssthresh$, 拥塞类型 $type$, 连续缩减计数 $consec_dec$ **Ensure:** 新窗口 new_cwnd , 新阈值 $new_ssthresh$

```

1:  $consec\_dec \leftarrow consec\_dec + 1$ 
2:                                     ▷ 连续缩减保护：超过 3 次后保持窗口
3: if  $consec\_dec > 3$  then
4:     return ( $cwnd, cwnd$ )                                     ▷ 保持当前窗口不再缩减
5: end if
6:                                     ▷ 根据拥塞类型选择窗口保留因子
7: if  $type = TIMEOUT$  then
8:      $\rho \leftarrow 0.50$                                          ▷ 超时：最严厉响应
9: else if  $type = LOSS$  then
10:     $\rho \leftarrow 0.70$                                          ▷ 丢包：较激进响应
11: else if  $type = ECN$  then
12:     $\rho \leftarrow 0.75$                                          ▷ ECN：早期预警响应
13: else
14:     $\rho \leftarrow 1.0$                                            ▷ 无拥塞，不调整
15: end if
16:                                     ▷ 计算新窗口
17:  $new\_cwnd \leftarrow \lfloor \rho \times cwnd \rfloor$ 
18:                                     ▷ 应用窗口下限约束
19:  $new\_cwnd \leftarrow \max(new\_cwnd, 4 \times MSS)$ 
20:                                     ▷ 计算新慢启动阈值：以 BDP 和当前窗口的较小有效参考为锚定
21:  $ref \leftarrow \min(cwnd, \max(bdp, 4 \times MSS))$ 
22:  $new\_ssthresh \leftarrow \max(\lfloor \rho \times ref \rfloor, new\_cwnd)$ 
23: return ( $new\_cwnd, new\_ssthresh$ )

```

3.5.4 差异化拥塞响应算法

3.6 启发式反馈自适应的参数调优

Swift 算法实现了启发式反馈驱动的参数自适应机制，根据多种因素动态调整乘性增加因子（Multiplicative Increase Factor, α ）。

3.6.1 乘性增加因子的作用

在 Swift 的融合控制策略中，乘性增加因子 α 控制拥塞避免阶段的窗口增长速度：

$$target_ref = \alpha \times BDP + \gamma \times MSS \quad (3.6)$$

α 的取值决定了窗口围绕 BDP 目标的激进程度：较大的 α 使目标窗口更接近或略高于估计 BDP，能够更快地利用可用带宽，但也更容易形成排队；较小的 α 使目标窗口低于或接近 BDP，有利于排队释放和延迟恢复。

3.6.2 多因素调整机制

Swift 根据以下因素动态调整 α :

(1) 基于 RTT 膨胀比的路径感知调整

RTT 膨胀比反映网络排队延迟程度:

$$inflation_ratio = \frac{lastRtt}{minRtt} \quad (3.7)$$

调整规则采用与最小 RTT 相关的动态阈值。首先根据最小 RTT 计算松弛量:

$$s = 0.3 + 0.15\sqrt{\max(0, RTT_{min,ms} - 1)} \quad (3.8)$$

并构造 $1 + s$ 、 $1 + 2s$ 和 $1 + 4s$ 三个路径感知阈值。短 RTT 路径对排队更敏感, 阈值更紧; 长 RTT 路径允许更大的正常抖动, 阈值更宽。具体调整规则为:

- 当 $inflation_ratio < 1 + s$ 时, 认为排队较小, 增大 α ; 若最小 RTT 超过 5 ms, 则增长步长为 0.02, 否则为 0.03。
- 当 $1 + s \leq inflation_ratio < 1 + 2s$ 时, 认为排队适中, 温和增大 α : $\alpha \leftarrow \alpha + 0.01$ 。
- 当 $inflation_ratio > 1 + 4s$ 时, 认为排队较重, 减小 α ; 若最小 RTT 超过 5 ms, 则减小步长为 0.03, 否则为 0.02。

这种路径感知响应机制能够避免固定阈值在远距离路径上过早收紧窗口, 同时保留低 RTT 路径中对排队增长的敏感性。

(2) 基于性能反馈值基线相对趋势的调整

Swift 利用环境合成的性能反馈值评估当前控制策略的相对优劣。反馈值在每个正常确认事件上由确认段数带来的吞吐收益减去 RTT 膨胀惩罚构成, 因此在绝大多数 ACK 事件上取正值。这一分布特征决定了不能用固定的绝对阈值判断“反馈是否变好”: 若采用形如 $R_{ema} > 0.5$ 的判据, 则该条件在几乎每个 ACK 事件上都成立, α 会被持续推向上界 α_{max} , 自适应机制退化为单向棘轮, 失去调节意义。

当前实现改为**基线相对判据**, 即同时维护快、慢两条指数移动平均, 用快线相对慢线的偏离量而非绝对水平来判断趋势:

$$R_{fast} \leftarrow (1 - \eta_f) \times R_{fast} + \eta_f \times reward, \quad \eta_f = 0.15 \quad (3.9)$$

$$R_{base} \leftarrow (1 - \eta_b) \times R_{base} + \eta_b \times reward, \quad \eta_b = 0.02 \quad (3.10)$$

其中 R_{fast} 为快线，反映最近若干次决策的即时效果； R_{base} 为慢线基线，反映当前网络条件下的长期反馈水平。两条 EMA 在收到首个反馈值时用同一数值初始化，避免冷启动阶段的虚假偏离。判定容差按基线幅值自适应缩放：

$$m = \max(0.25, 0.1 \times |R_{base}|) \quad (3.11)$$

调整规则为：

- 当 $R_{fast} > R_{base} + m$ 时，表明近期决策相对该路径的常态水平确有改善，适度提高激进度： $\alpha \leftarrow \alpha + 0.01$ 。
- 当 $R_{fast} < R_{base} - 4m$ 时，表明近期决策显著劣于常态水平，回退激进度： $\alpha \leftarrow \alpha - 0.01$ 。

下调阈值取 $4m$ 而上调阈值取 m ，构成有意的非对称设计：反馈序列的负向尖峰主要由单次丢包或超时惩罚引起，而这类事件已经由差异化拥塞响应机制单独处理，若在 α 层面再次同步收缩会造成响应叠加；采用较宽的下调死区可以把 α 的收缩限定在持续性劣化而非瞬时惩罚上。基线相对判据的另一优点是与路径无关：不同带宽和 RTT 的路径其反馈绝对幅值相差可达数个数量级，而相对偏离量是无量纲的，因此同一组超参数可以直接跨场景复用，无需按链路类型重新标定。

(3) 基于连续增长趋势的调整

当窗口连续增长超过 8 次时，表明网络状况持续良好，给予稳定增长的正向反馈：

$$\alpha \leftarrow \alpha + 0.01 \quad \text{如果连续增长次数} > 8 \quad (3.12)$$

3.6.3 参数边界约束

为保证控制策略的稳定性， α 的取值被限制在预设范围内：

$$\alpha \in [\alpha_{min}, \alpha_{max}] = [0.85, 1.30] \quad (3.13)$$

α 的默认初始值为 1.10。下界 0.85 为无线、蜂窝或浅缓冲路径提供更强排队释放空间；上界 1.30 限制窗口相对于 BDP 的长期扩张幅度，避免慢性队列积累。

Algorithm 5 自适应参数调优算法**Require:** 当前 α , 观测状态 obs , 性能反馈值 $feedback$, 历史状态 $history$ **Ensure:** 更新后的 α

```

1:                                     ▷ 因素 1: 基于 RTT 膨胀比的路径感知调整
2:  $inflation\_ratio \leftarrow obs.lastRtt / obs.minRtt$ 
3:  $min\_rtt\_ms \leftarrow obs.minRtt / 1000$ 
4:  $slack \leftarrow 0.3 + 0.15\sqrt{\max(0, min\_rtt\_ms - 1)}$ 
5:  $low \leftarrow 1 + slack, mid \leftarrow 1 + 2 \times slack, high \leftarrow 1 + 4 \times slack$ 
6: if  $inflation\_ratio < low$  then
7:    $ramp \leftarrow 0.02$  if  $min\_rtt\_ms > 5$  else  $0.03$ 
8:    $\alpha \leftarrow \alpha + ramp$ 
9:    $history.consecutive\_growth \leftarrow history.consecutive\_growth + 1$ 
10: else if  $inflation\_ratio < mid$  then
11:    $\alpha \leftarrow \alpha + 0.01$ 
12: else if  $inflation\_ratio > high$  then
13:    $step \leftarrow 0.03$  if  $min\_rtt\_ms > 5$  else  $0.02$ 
14:    $\alpha \leftarrow \alpha - step$ 
15:    $history.consecutive\_growth \leftarrow 0$ 
16: end if
17:                                     ▷ 因素 2: 基于反馈值基线相对偏离的趋势调整
18: if 首次收到反馈值 then
19:    $R_{fast} \leftarrow reward, R_{base} \leftarrow reward$ 
20: else
21:    $R_{fast} \leftarrow 0.85 \times R_{fast} + 0.15 \times reward$ 
22:    $R_{base} \leftarrow 0.98 \times R_{base} + 0.02 \times reward$ 
23: end if
24:  $m \leftarrow \max(0.25, 0.1 \times |R_{base}|)$ 
25: if  $R_{fast} > R_{base} + m$  then
26:    $\alpha \leftarrow \alpha + 0.01$ 
27: else if  $R_{fast} < R_{base} - 4 \times m$  then
28:    $\alpha \leftarrow \alpha - 0.01$ 
29: end if
30:                                     ▷ 因素 3: 基于连续稳定增长的反馈
31: if  $history.consecutive\_growth > 8$  then
32:    $\alpha \leftarrow \alpha + 0.01$ 
33: end if
34:                                     ▷ 应用边界约束
35:  $\alpha \leftarrow \max(\min(\alpha, 1.30), 0.85)$ 
36: return  $\alpha$ 

```

3.6.4 自适应参数调优算法

3.7 融合控制策略

Swift 采用融合控制策略计算目标拥塞窗口，将基于带宽延迟积（BDP）估计的速率控制与基于当前窗口的保守控制相结合。

3.7.1 带宽延迟积估计

带宽延迟积（Bandwidth-Delay Product, BDP）是网络链路能够容纳的最大在途数据量，是拥塞控制的重要参考指标：

$$BDP = Bandwidth \times RTT \quad (3.14)$$

Swift 采用”时间窗口投递速率采样 + 窗口化最大值过滤”两级结构估计瓶颈带宽，再与最小 RTT 相乘得到 BDP：

$$BDP = \max_{i \in W_{bw}} (DR_i) \times RTT_{min} \quad (3.15)$$

其中 W_{bw} 为容量 40 个样本的滑动窗口， RTT_{min} 为该流历史最小 RTT（同时接受协议栈上报的 $minRtt$ 与本地观测的 $lastRtt$ 作为候选下界）。

时间窗口投递速率采样。单个投递速率样本由累计确认字节数在一段时间窗口上的增量给出。算法为每条流维护累计确认字节数 $B(t)$ ，并在队列中记录二元组 $(t, B(t))$ ；每次收到确认后先更新 $B(t)$ ，再淘汰窗口左端超出时间跨度的样本，最后按下式计算样本：

$$DR = \frac{B(t) - B(t - \Delta)}{\Delta}, \quad \Delta = \text{clamp}(2 \times RTT_{min}, 5 \text{ ms}, 1 \text{ s}) \quad (3.16)$$

窗口跨度 Δ 取最小 RTT 的两倍，是为了让采样区间至少覆盖一个完整的 ACK 时钟周期：只有当窗口跨度不小于一个 RTT 时，窗口内的确认字节总量才等于瓶颈在该时段实际排空的数据量， DR 才具有”瓶颈排空速率”的物理含义。下界 5 ms 用于抑制微秒级 RTT 路径上因窗口过短而放大的采样噪声，上界 1 s 用于保证带宽下降时估计能够及时跟随。

为何不能按单次确认计算投递速率。本算法早期版本采用形如 $DR = segmentsAcked \times segSize / lastRtt$ 的单次确认公式。该式的分子只包含本次确认的字节数，而分母却是整个 RTT，因此它实际估计的是”若一个 RTT 内只到达这一个确认时的速率”，与真实瓶颈速率相差约等于每 RTT 的确认个数 $N_{ack} \approx cwnd / (segSize \times b)$ （其中 b 为延迟确认因子）。在高 BDP 路径上 N_{ack} 可达数百，估计值因而被系统性压低同一量级，式 (3.15) 给出的 BDP 随

之崩塌；此时拥塞避免阶段的目标窗口 $\alpha \times BDP$ 长期低于当前窗口，窗口只能停留在安全下界 $200 \times MSS$ 上，吞吐量被硬性锁定为 $200 \times MSS/RTT$ 。这一失效模式在第四章的跨域 WAN 场景中留下了可验证的痕迹，也是本文将该估计器整体替换为式 (3.16) 的直接原因。式 (3.16) 的分子与分母取自同一时间区间，量纲自洽，不再依赖每 RTT 确认个数。

窗口化最大值过滤。将 DR 样本压入容量 40 的滑动窗口并取窗口内最大值，其作用有三：(1) 投递速率样本天然瓶颈能力的下界估计——确认稀疏、应用层供数不足或跨流量抢占都只会使样本偏小而不会偏大，因此取最大值是对真实瓶颈的一致估计；(2) 滑动窗口限制了历史记忆长度，使估计能够跟随带宽的真实下降，避免全局最大值带来的永久性高估；(3) 窗口容量取 40 而非较小值，是为了让长 RTT 路径在窗口内容纳足够多的采样周期——GEO 卫星链路单个 RTT 可达 600 ms，过短的窗口会在瓶颈被充分采样之前就发生淘汰，导致估计值在低位振荡。

此外，流级吞吐量趋势由指数移动平均跟踪，用于状态观察与性能记录：

$$throughput_ema \leftarrow 0.9 \times throughput_ema + 0.1 \times DR \quad (3.17)$$

BDP 估计的完整流程如算法 6 所示。当 $\max(DR)$ 尚未建立或 RTT_{min} 无效时，BDP 回退为当前拥塞窗口，使算法在连接建立初期退化为常规的窗口驱动行为而非产生无意义的目标值。

3.7.2 慢启动阶段策略

在慢启动阶段 ($cwnd < ssthresh$)，Swift 采用指数增长策略以快速探测可用带宽：

$$target_cwnd = 2 \times BDP \quad (3.18)$$

$$increment = segmentsAacked \times MSS \quad (3.19)$$

当窗口远低于 BDP 时，采用加速增长：

$$\text{如果 } cwnd < 0.3 \times BDP, \quad increment = 2 \times segmentsAacked \times MSS \quad (3.20)$$

慢启动目标设为 $2 \times BDP$ 而非更大值（如 $3 \times BDP$ ），是为了避免慢启动阶段过度膨胀导致突发丢包。当窗口达到目标值后，自动退出慢启动进入拥塞避免阶段。

Algorithm 6 时间窗口投递速率与 BDP 估计**Require:** 流状态 *state*, 观测状态 *obs***Ensure:** BDP 估计值

```

1: if obs.lastRtt > 0 then
2:   state.min_rtt  $\leftarrow \min(\textit{state.min\_rtt}, \textit{obs.lastRtt})$ 
3: end if
4: if  $0 < \textit{obs.minRtt} < \textit{state.min\_rtt}$  then
5:   state.min_rtt  $\leftarrow \textit{obs.minRtt}$ 
6: end if
7: if obs.segmentsAked > 0 and obs.segSize > 0 then
8:   state.acked_total  $\leftarrow \textit{state.acked\_total} + \textit{obs.segmentsAked} \times \textit{obs.segSize}$ 
9:   state.samples.Append((obs.simTime, state.acked_total))
10:   $\Delta \leftarrow \text{clamp}(2 \times \textit{state.min\_rtt}, 5 \text{ ms}, 1 \text{ s})$ 
11:  while  $|\textit{state.samples}| \geq 2$  and  $\textit{obs.simTime} - \textit{state.samples}[0].t > \Delta$  do
12:    state.samples.PopFront()
13:  end while
14:  span  $\leftarrow \textit{obs.simTime} - \textit{state.samples}[0].t$ 
15:  delivered  $\leftarrow \textit{state.acked\_total} - \textit{state.samples}[0].B$ 
16:  if  $|\textit{state.samples}| \geq 2$  and span > 0 and delivered > 0 then
17:    DR  $\leftarrow \textit{delivered} / \textit{span}$ 
18:    state.bw_samples.Append(DR)
19:    state.max_bw  $\leftarrow \max(\textit{state.bw\_samples})$ 
20:  end if
21: end if
22: if state.max_bw > 0 and state.min_rtt <  $\infty$  then
23:   state.bdp  $\leftarrow \textit{state.max\_bw} \times \textit{state.min\_rtt}$ 
24: end if
25: return  $\textit{state.bdp} > 0 ? \textit{state.bdp} : \max(\textit{obs.cwnd}, 1)$ 

```

▷ 容量 40 的滑动窗口

3.7.3 拥塞避免阶段策略

在拥塞避免阶段 ($cwnd \geq ssthresh$), Swift 采用稳健的窗口增长策略:

$$\textit{target_ref} = \alpha \times BDP + \gamma \times MSS \quad (3.21)$$

其中, α 是乘性增加因子 (通过自适应调优动态调整, 基准值 1.10), $\gamma = 1.0$ 是加性增加因子, 表示每个 ACK 事件引入 1 个 MSS 的基础增量。拥塞避免阶段不再简单地以当前窗口作为下界持续取最大值, 而是围绕 $\alpha \times BDP$ 目标进行拉升或回落: 当目标高于当前窗口时, 以 $\max(\gamma \times MSS, MSS, \textit{gap}/16)$ 的步长向目标靠近; 当目标低于当前窗口时, 每个 ACK 事件按超出量的一半进行回落, 以更快释放浅缓冲路径中的排队。

融合控制策略还考虑管道利用率因素。管道利用率定义为:

$$\textit{utilization} = \frac{\textit{bytesInFlight}}{\textit{cwnd}} \quad (3.22)$$

调整规则：

- 仅当 RTT 未明显膨胀 ($lastRtt < 1.3 \times minRtt$) 时启用利用率补偿。
- 当 $utilization < 0.4$ 时, 短 RTT 路径额外增加 $2 \times MSS$, 长 RTT 路径额外增加 $1 \times MSS$ 。
- 当 $0.4 \leq utilization < 0.5$ 且路径不是长 RTT 路径时, 额外增加 $1 \times MSS$ 。

该设计只在管道确实未充分利用且队列未增长时加速, 避免旧式利用率补偿在无线、蜂窝或浅缓冲路径中造成持续排队。

3.7.4 窗口边界约束

为保证窗口大小的合理性, Swift 对计算结果应用边界约束:

$$cwnd \in [4 \times MSS, \max(4 \times BDP, 200 \times MSS)] \quad (3.23)$$

下界 $4 \times MSS$ 保证基本传输能力。上界设为 BDP 的 4 倍或 200 个 MSS 的较大值, 用于在 BDP 估计暂时偏大或路径缓冲较浅时限制窗口长期膨胀。

3.7.5 融合控制策略算法

3.8 每流独立状态管理

Swift 设计了每流独立的状态管理机制, 为每个 TCP 连接维护独立的状态数据结构, 支持多流并发场景下的差异化控制。

3.8.1 状态数据结构

每个 TCP 连接维护以下状态数据, 采用层次化组织方式:

(1) 历史度量缓冲区: 带宽估计维护两级队列。第一级是投递速率采样队列, 记录“仿真时刻—累计确认字节数”二元组, 容量上限 4096 项, 用于按时间窗口差分计算单个投递速率样本; 每次更新时淘汰左端超出窗口跨度 $\Delta = \text{clamp}(2 \times RTT_{min}, 5 \text{ ms}, 1 \text{ s})$ 的样本, 因此队列的实际长度随路径 RTT 与确认密度自适应变化。第二级是投递速率过滤队列, 容量 40 个样本, 取窗口内最大值作为瓶颈带宽估计。RTT 跟踪维护容量 40 个样本的最近 RTT 队列, 同时维护该流的全局最小 RTT 作为传播延迟估计。BDP 由最大带宽与最小 RTT 的乘积得到, 并在估计尚未建立时回退为当前拥塞窗口。

Algorithm 7 融合控制策略算法**Require:** 观测状态 obs , 自适应参数 α , $\gamma = 1.0$, BDP 估计 bdp **Ensure:** 新窗口 new_wnd

```

1:  $\triangleright$  降窗后冻结: 显式降窗后的若干 ACK 事件内保持窗口不增长
2: if  $state.freeze\_acks\_remaining > 0$  then
3:    $state.freeze\_acks\_remaining \leftarrow state.freeze\_acks\_remaining - 1$ 
4:   return  $obs.ssthresh, obs.cwnd$ 
5: end if
6:  $\triangleright$  计算管道利用率
7:  $utilization \leftarrow obs.bytesInFlight / obs.cwnd$ 
8:  $\triangleright$  判断当前阶段
9: if  $obs.cwnd < obs.ssthresh$  and  $in\_slow\_start$  then
10:    $\triangleright$  HyStart 风格慢启动退出
11:   if  $obs.lastRtt > HystartThreshold(state) \times state.ss\_min\_rtt\_sample$  then
12:      $state.in\_slow\_start \leftarrow False$ 
13:     return  $\max(obs.ssthresh, obs.cwnd), obs.cwnd$ 
14:   end if
15:    $target \leftarrow \max(2 \times bdp, 10 \times obs.segSize)$ 
16:    $increment \leftarrow obs.segmentsAked \times obs.segSize$ 
17:   if  $obs.cwnd < 0.3 \times bdp$  and  $obs.lastRtt < 1.2 \times state.min\_rtt$  then
18:      $increment \leftarrow 2 \times obs.segmentsAked \times obs.segSize$ 
19:   end if
20:    $new\_wnd \leftarrow \min(obs.cwnd + increment, target)$ 
21: else
22:    $\triangleright$  拥塞避免阶段: 围绕  $\alpha \times BDP$  目标进行有界步长拉升或回落
23:    $state.in\_slow\_start \leftarrow False$ 
24:    $target\_rate \leftarrow \lfloor \alpha \times bdp \rfloor$ 
25:    $gamma\_bytes \leftarrow \gamma \times obs.segSize$ 
26:   if  $target\_rate > obs.cwnd$  then
27:      $gap \leftarrow target\_rate - obs.cwnd$ 
28:      $step \leftarrow \max(gamma\_bytes, obs.segSize, gap/16)$ 
29:      $new\_wnd \leftarrow \min(obs.cwnd + step, target\_rate + gamma\_bytes)$ 
30:   else if  $target\_rate < obs.cwnd$  then
31:      $excess \leftarrow obs.cwnd - target\_rate$ 
32:      $new\_wnd \leftarrow \max(obs.cwnd - \max(excess/2, obs.segSize), target\_rate)$ 
33:   else
34:      $new\_wnd \leftarrow obs.cwnd + gamma\_bytes$ 
35:   end if
36:    $\triangleright$  仅在 RTT 未膨胀时启用管道利用率补偿; 长 RTT 路径降低补偿强度
37:   if  $obs.lastRtt < 1.3 \times state.min\_rtt$  and  $utilization < 0.4$  then
38:      $new\_wnd \leftarrow new\_wnd + (state.min\_rtt > 5ms ? obs.segSize : 2 \times obs.segSize)$ 
39:   else if  $obs.lastRtt < 1.3 \times state.min\_rtt$  and  $utilization < 0.5$  and  $state.min\_rtt \leq 5ms$ 
then
40:      $new\_wnd \leftarrow new\_wnd + obs.segSize$ 
41:   end if
42: end if
43: return  $obs.ssthresh, new\_wnd$ 

```

(2) **拥塞事件计数数据**：状态结构维护丢包事件计数和 ECN 标记事件计数，用于记录每条流在仿真过程中的拥塞事件分布。

(3) **性能指标数据**：性能指标模块维护吞吐量的指数移动平均值（EMA，平滑因子 0.1），用于跟踪流级别的性能变化趋势。

(4) **自适应参数数据**：自适应参数模块维护乘性增加因子 α （取值范围 [0.85, 1.30]，默认值 1.10）、加性增加因子 γ （默认值 1.0）、连续缩减计数器（用于连续缩减保护）、连续增长计数器（用于稳定增长反馈）、降窗后冻结 ACK 计数器（默认 4 个 ACK 事件）和慢启动阶段标记。

3.8.2 状态查找与更新

Swift 使用 `socketUuid` 作为键，通过哈希表实现 $O(1)$ 时间复杂度的状态查找和更新。哈希表的选择基于查找效率、动态扩展和内存效率三方面考量。

Algorithm 8 流状态查找与更新

```

1: function LookupFlowState(manager, socket_uuid)
2:   hash  $\leftarrow$  ComputeHash(socket_uuid)
3:   bucket  $\leftarrow$  manager.buckets[hash mod manager.num_buckets]
4:   for each entry in bucket do
5:     if entry.key = socket_uuid then
6:       return entry.value
7:     end if
8:   end for
9:   return null                                     ▷ 未找到
10: end function
11: function UpdateFlowState(manager, socket_uuid, obs)
12:   state  $\leftarrow$  GetOrCreateFlowState(manager, socket_uuid)
13:   state.last_update_time  $\leftarrow$  obs[2]
14:   UpdateMetrics(state.metrics_buffer, obs)
15:   if IsECNEvent(obs) then
16:     state.ecn_tracker.RecordEvent(obs[2])
17:   end if
18:   UpdatePerformanceMetrics(state.performance, obs)
19: end function

```

3.8.3 状态生命周期管理

TCP 连接的生命周期包括建立、传输和关闭三个阶段。Swift 的状态管理遵循延迟初始化、增量更新、垃圾回收和状态持久化四个原则。

Algorithm 9 流状态垃圾回收

```

1: function GarbageCollect(manager, current_time, inactive_threshold)
2:   to_remove  $\leftarrow$  EmptyList()
3:   for each (uuid, state) in manager.flow_states do
4:     idle_time  $\leftarrow$  current_time - state.last_update_time
5:     if idle_time > inactive_threshold then
6:       to_remove.Append(uuid)
7:     end if
8:   end for
9:   for each uuid in to_remove do
10:    manager.flow_states.Remove(uuid)
11:    FreeResources(uuid)
12:   end for
13:   return |to_remove|
14: end function

```

3.9 实现细节与优化技术

本节详细介绍 Swift 算法的具体实现细节和优化技术。

3.9.1 系统架构设计

Swift 算法的系统架构遵循模块化设计原则，采用分层结构实现各功能模块的解耦。整体架构分为仿真层、环境层、决策层和通信层四个层次。仿真层负责网络拓扑构建和数据包传输仿真；环境层作为协议栈与决策进程之间的桥梁；决策层实现 Swift 算法的核心逻辑，遵循状态隔离、分层决策和实时性原则；通信层负责仿真进程和决策进程之间的数据交换。

Algorithm 10 Swift 系统初始化流程

Require: 网络拓扑配置 *topology*, 仿真参数 *params*

Ensure: 初始化完成的 Swift 系统

```

1: simulator  $\leftarrow$  CreateSimulator(params.duration)
2: network  $\leftarrow$  BuildTopology(topology)
3: ConfigureQueues(network, params.queue_size)
4: env  $\leftarrow$  CreateStateChannel()
5: RegisterCallbacks(env, network.tcp_sockets)
6: ConfigureObservationSpace(env, 15)
7: ConfigureActionSpace(env, 2) ▷ ssthresh 和 cwnd
8: agent  $\leftarrow$  CreateSwiftAgent()
9: agent.flow_states  $\leftarrow$  EmptyHashMap()
10: agent.alpha_default  $\leftarrow$  1.10
11: channel  $\leftarrow$  CreateMessageChannel()
12: ConnectEnvAndAgent(env, agent, channel)
13: simulator.Run()

```

3.9.2 核心算法伪代码

Swift 决策模块的主控制循环负责协调各功能模块：处理来自环境层的状态采集结果，依据规则输出窗口决策，并把决策回写协议栈。

Algorithm 11 Swift 主控制循环

Require: 环境接口 *env*，最大迭代次数 *max_iterations*

Ensure: 控制结果序列

```

1: iteration  $\leftarrow$  0
2: flow_manager  $\leftarrow$  InitFlowStateManager()
3: while iteration < max_iterations and not env.IsDone() do
4:   obs  $\leftarrow$  env.GetObservation()
5:   socket_uuid  $\leftarrow$  obs[0]
6:   flow_state  $\leftarrow$  flow_manager.GetOrCreate(socket_uuid)
7:   UpdateFlowState(flow_state, obs)
8:   (is_congested, cong_type, severity)  $\leftarrow$  DetectCongestion(obs, flow_state)
9:   if is_congested then
10:    action  $\leftarrow$  CongestionResponse(obs, cong_type, flow_state)
11:   else
12:    UpdateAdaptiveParams(flow_state, obs)
13:    action  $\leftarrow$  FusionControl(obs, flow_state)
14:   end if
15:   env.ExecuteAction(action)
16:   iteration  $\leftarrow$  iteration + 1
17: end while

```

3.9.3 数据结构设计

Swift 算法采用高效的数据结构支持其核心功能。循环队列（Circular Buffer）用于存储带宽与 RTT 历史样本，具有固定内存占用、 $O(1)$ 时间复杂度的插入和删除操作、自动保持时间局部性等优势。每流状态字典以 socketUuid 为键隔离不同 TCP 连接的 α 、BDP、拥塞事件计数、连续缩减计数和降窗后冻结计数，避免不同流之间的控制状态相互污染。

3.9.4 性能优化技术

为了提高 Swift 算法的执行效率，满足实时决策的要求，本文采用了多种性能优化技术：

(1) **延迟初始化策略**：流状态采用延迟初始化策略，只在首次访问时创建状态数据结构。

(2) **增量计算优化**：吞吐量、BDP 等指标的计算采用增量方式，时间复杂度为 $O(1)$ 。

(3) **分支路径清晰化**：拥塞检测、拥塞响应、慢启动和拥塞避免分别采用独立分支，便于将丢包、ECN、超时和普通 ACK 事件映射到可解释的窗口更新路径。

Algorithm 12 延迟初始化状态获取

```

1: function GetOrCreateFlowState(manager, socket_uuid)
2:   if socket_uuid  $\notin$  manager.flow_states then
3:     new_state  $\leftarrow$  CreateEmptyFlowState()
4:     new_state. $\alpha$   $\leftarrow$  1.10
5:     new_state.min_rtt  $\leftarrow$   $\infty$ 
6:     new_state.max_throughput  $\leftarrow$  0
7:     new_state.consecutive_decreases  $\leftarrow$  0
8:     new_state.consecutive_increases  $\leftarrow$  0
9:     new_state.in_slow_start  $\leftarrow$  True
10:    manager.flow_states[socket_uuid]  $\leftarrow$  new_state
11:   end if
12:   return manager.flow_states[socket_uuid]
13: end function

```

(4) **有界步长更新**: 拥塞避免阶段的窗口拉升、回落和利用率补偿均采用有界步长, 避免单个 ACK 事件造成窗口突变。

(5) **降窗后冻结**: 显式降窗后冻结后续 4 个 ACK 事件中的窗口增长, 减少窗口刚缩减后立即反弹造成的二次排队。

3.9.5 状态—决策同步交互路径

TcpSwift 当前实验采用 ZeroMQ 同步请求—响应路径: ns-3 仿真进程在 TCP 回调处生成状态向量 (由 ns3-gym/OpenGym 框架的传输容器承载, 仅作消息通道), Python 决策进程返回慢启动阈值和拥塞窗口决策。该路径便于逐 ACK 解释窗口决策、复现实验数据并与 tcp_swift.py 中的真实策略保持一致。由于该路径依赖同步消息交互, 本文实验未将其与 MTP 并行仿真结合, 也不报告 MTP 加速数据。

3.9.6 正确性保证机制

Swift 实现包含窗口边界、待应用动作时效性和异常观测回退等防护。这些机制限制当前控制器的输出范围并避免已知的陈旧动作问题, 但不构成对任意跨进程输入或所有 TCP 状态组合的形式化正确性保证。

(1) **跨进程决策的时效性校验**。同步信道采用请求—应答模式, 状态的采集时刻与决策的应用时刻之间存在一个消息往返间隔。在此间隔内协议栈的状态可能已经改变, 最关键的情形是重传定时器超时。tcp-swift-env.cc 在 CongestionStateSet 回调中检测到新状态为 CA_LOSS 时, 清除 m_hasPendingCwnd 标志并丢弃尚未生效的窗口值, 避免超时前的决策覆盖协议栈已更新的恢复状态。

(2) **动作边界约束**。Python 控制器将拥塞窗口夹紧到 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$

区间。环境侧在接收动作后，以 $2 \times MSS$ 分别校验拥塞窗口和慢启动阈值的下界。该组合约束用于限制窗口过小或过度增长的风险； $200 \times MSS$ 项同时为 BDP 尚未建立的连接保留增长空间。

(3) **异常观测回退。**环境侧将未初始化或非正 RTT 编码为零，并在报文段大小为零时使用安全值；Python 控制器在 BDP 尚未建立时以当前拥塞窗口作为回退值。

(4) **实验可复现性。**ns-3.40 通过 RngRun 区分随机运行号，可对同一(场景, 协议)组合执行多次重复并汇总。第四章每个配置仅执行一次确定性运行，因此可按给定配置与运行号重复，但不能据此估计跨运行号方差。

3.10 本章小结

本章说明了 Swift 算法的系统结构及三项核心技术。

(1) **多信号拥塞状态感知：**ns-3 环境通过 15 元素容器传输 4 项元数据与 11 项有效状态，并结合 GetSsThresh、IncreaseWindow、PktsAacked、CongestionStateSet 和 CwndEvent 回调更新状态。在 GetSsThresh 路径中，控制器依据 caState 与 ecnState 区分超时、ECN 驱动的窗口缩减和普通丢包；在 IncreaseWindow 路径中继续检查 ECN 状态。RTT 膨胀用于参数调节和慢启动退出，不单独触发乘性降窗。

(2) **启发式反馈自适应的融合控制：**BDP 由滑动时间窗口内的累计确认字节差分估计交付速率，再经容量 40 的窗口最大值滤波并乘以最小 RTT 得到。控制器根据最小 RTT 感知的 RTT 膨胀阈值、反馈值相对慢速基线的偏移和连续增长趋势调节每流 $\alpha \in [0.85, 1.30]$ ，在慢启动与拥塞避免阶段以有界步长调整窗口。

(3) **稳定性与安全约束：**不同拥塞类型分别采用丢包 0.70、ECN 0.75 和超时 0.50 的窗口保留因子；连续缩减保护、降窗后 4 个 ACK 事件冻结、窗口上下界以及 CA_LOSS 状态下的陈旧动作作废共同限制窗口过度收缩、快速反弹和陈旧决策覆盖。上述控制回路经 ns3-gym/OpenGym 与 ZeroMQ 同步执行，当前实验采用单一随机种子，其跨种子稳定性仍需重复实验检验。

第四章 实验评估

本章介绍 Swift 算法的实验评估，包括 ns-3.40 实验配置、评测指标定义、对比算法、场景设计、测量检查与结果分析，并说明结论的适用边界。

4.1 实验环境配置

4.1.1 软件与运行环境

实验基于 ns-3.40 与 ns3-gym/OpenGym 实现，Python 进程运行 TcpSwift 规则决策模块，ZeroMQ 和 Protocol Buffers 承担状态与动作传输。现有日志包未记录 CPU、内存、操作系统以及 Python 依赖版本，因此本文不把具体主机配置作为可复现条件，也不据此报告运行开销。实验可复现信息以仿真脚本、场景参数、随机运行号和原始 FlowMonitor 产物为准。

4.1.2 网络拓扑与队列配置

实验采用经典的哑铃型 (Dumbbell) 网络拓扑，如图4.1所示。该拓扑包括：

- 左侧叶子节点（发送端）： n 个节点，默认 $n = 3$
- 右侧叶子节点（接收端）： n 个节点
- 两个核心路由器：R0（左侧）和 R1（右侧）
- 接入链路：连接叶子节点和路由器
- 瓶颈链路：连接两个路由器

瓶颈两侧网关安装队列规则，队列容量 $Q = \max(BDP \times n / MTU, 100)$ 个数据包按各场景的瓶颈带宽与链路时延自动定尺，下界 100 个数据包用于保证低 BDP 路径上的基本缓冲能力。

队列规则与 ECN 配置。 仿真器 contrib/opengym/examples/swift-tcp/sim.cc 将瓶颈两侧网关的队列规则设为 `ns3::RedQueueDisc`，并按队列容量比例配置标记门限： $MinTh = 0.3 \times Q$ 、 $MaxTh = 0.9 \times Q$ ，同时置 `RedQueueDisc::UseEcn` 为真，使 RED 在队列长度进入 $[MinTh, MaxTh]$ 区间时对 ECT 数据包置 CE 码点。`ns3::TcpSocketBase::UseEcn` 被统一置为 On 并作用于全部四种被比较协议：ECN 协商能力直接影响拥塞信号的到达时刻，

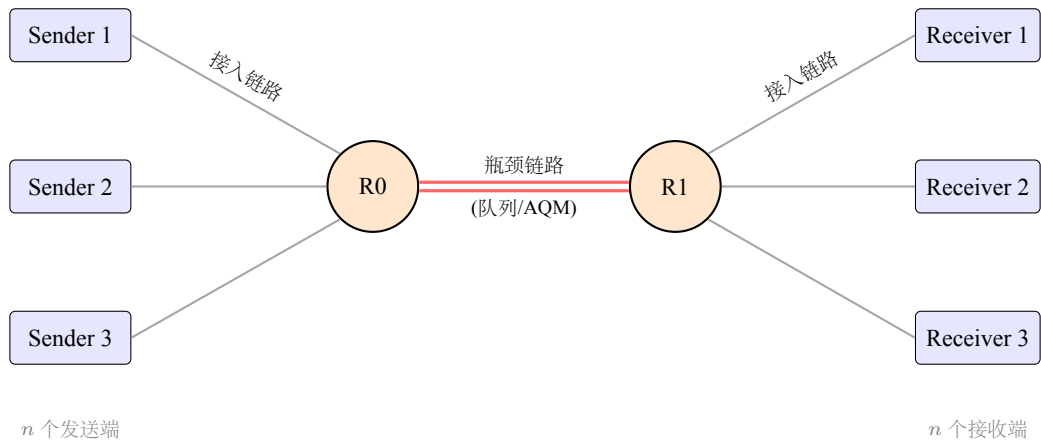


图 4.1 哑铃型（Dumbbell）实验网络拓扑

若只令待评估算法具备该能力，则对照实验中观测到的差异将无法区分算法设计的贡献与信号通路配置的差异。统一开启 ECN 是保证四协议对照公平性的必要条件。

需要交代的配置影响是：瓶颈 RED 队列能够生成 CE 标记，四种协议均开启 ECN 协商，但 FlowMonitor 的聚合指标无法把”ECN 触发收缩”与”丢包触发收缩”逐事件分离。因此本章不把任何结果单独归因于 ECN 分支或丢包分支，只描述当前完整实现（差异化保留因子 0.70/0.75/0.50、 $\alpha \times BDP$ 目标窗口、时间窗口带宽估计与基线相对反馈自适应）在 RED/ECN 配置下的总体行为；各机制单独关闭的消融重跑未包含在本章数据中。

每条 TCP 流由 BulkSendHelper 产生持续饱和的单向长流，三条流分别在 0.1、0.2、0.3 s 错开启动，以减少同步启动造成的同相位效应；接收端为 PacketSink。反向路径仅承载 ACK。

4.1.3 仿真参数

默认仿真参数配置如表4.1所示：

表 4.1 默认仿真参数

参数	取值	说明
仿真时长	20 秒	每次实验的仿真时间
叶子节点数	3	发送端和接收端各 3 个节点
MTU	1500 字节	以太网标准 MTU
TCP 段大小	1440 字节	$1500 - 20 - (20 + 20)$
收发缓冲区	16 MB	保证高 BDP 路径不受窗口通告限制
SACK	启用	<code>TcpSocketBase::Sack</code>
延迟确认	2	<code>TcpSocket::DelAckCount</code>
队列规则	RED	启用 ECN 标记， $MinTh = 0.3Q$ 、 $MaxTh = 0.9Q$ ， $Q = \max(\dots)$
ECN	四协议对称协商	RED 置 CE 码点，全协议 <code>UseEcn=On</code>
随机种子	每配置单次确定性运行	见第 4.1.4 节的说明

4.1.4 随机种子与重复性说明

批量实验驱动支持通过不同 RngRun 对同一 (场景, 协议) 组合重复运行, 并在统计阶段按运行号求算术平均。本章所用实验矩阵中, 每个 (场景, 协议, 设置) 三元组对应一次确定性仿真运行。

如实说明这点的含义: 单次运行意味着本章报告的差异不附带跨种子置信区间, 因此本章在解释结论时遵循两条自律规则。第一, 只把方向一致、量级明确、机理可解释的差异作为结论——例如多数场景呈现吞吐领先这一整体趋势、百分之 2~6 的增益量级以及 GEO/LEO 等长 RTT 链路上相对基线的吞吐领先; 对于百分之零点几量级且方向不一致的观察 (如各场景间 $\pm 3\%$ 以内的时延差), 本章只报告“相当”而不解读方向。第二, 凡属“单点数值恰好更优”性质的观察, 一律不写入结论性论述。ns-3 的离散事件仿真在给定 RngRun 下完全确定, 因此上述单次运行结果可精确复现, 但其对随机实现的稳健性有待多种子重跑确认, 该项已列入第五章的后续工作。

4.1.5 UDP Burst 干扰流量与实验矩阵

在真实的端到端传输路径上, 共享瓶颈链路普遍同时承载突发性的非响应式 UDP 流量——移动终端的视频流与实时会议、远程过程调用、运维巡检探测、游戏控制信令等都属于此类。这些跨流量不服从 TCP 的拥塞反馈环, 会与 TCP 长流竞争瓶颈队列容量并放大 RTT 抖动, 是异构终端共享接入链路时的常态而非例外。为了系统性评估 Swift 在此类干扰下的鲁棒性, 仿真器在哑铃拓扑的发送端 0 与接收端 0 之间提供了可选的 UDP Burst 旁路通道, 由命令行参数 `enable_udp_burst` (默认 `false`) 控制启停。启用时, 该通道在瓶颈链路上并发启动一路 OnOff UDP 源, 与 TCP 长流共享同一瓶颈队列, 其配置为:

- 报文长度: 1024 字节;
- 占空比: 50%, On 与 Off 各持续 0.5 s (OnTime 与 OffTime 均为 0.5 s);
- 数据率: On 期峰值速率取当前瓶颈速率的 64%, 即长期平均负载为瓶颈速率的 32%。

换言之, 突发强度随场景瓶颈带宽成比例缩放: 对于 1 Gbps 瓶颈, 突发平均负载为 320 Mbps (峰值 640 Mbps); 对于 10 Gbps 瓶颈则为 3.2 Gbps (峰值 6.4 Gbps)。该设计使各场景的突发相对强度一致, 跨场景结果可直接比较; 若改用固定绝对值突发, 则高带宽场景的相对干扰会被人为压低。由此构成两类严格对照的实验设置:

- **纯 TCP 设置 (无 UDP Burst):** `enable_udp_burst=false`, 瓶颈链路仅承载 3 条 TCP 长流, 用于评估稳态性能。

- **UDP Burst 干扰设置**: `enable_udp_burst=true`, 在同一瓶颈并发运行上述比例化突发源, 用于评估跨流量鲁棒性。

两类设置共享完全相同的场景拓扑、瓶颈带宽、接入带宽、链路时延、队列调度策略与随机种子, 仅 `enable_udp_burst` 取值不同, 因而构成严格的 A/B 对照。第 4.6 节的主实验结果基于纯 *TCP* 设置; 跨流量鲁棒性结果在第 4.8 节专题报告。UDP 流量本身在统计时被排除在指标之外 (见第 4.2 节), 因此”吞吐量变化”衡量的是 TCP 长流在竞争下实际获得的容量份额。

4.2 评测指标

本文全部指标均由 ns-3 FlowMonitor 模块采集的逐流统计量导出。指标定义中有一处方法学要点必须先行澄清, 否则数值不可比。

4.2.1 指标的流向界定

哑铃拓扑上每条 TCP 连接在 FlowMonitor 中登记为两条流: 前向流 ($10.1.x \rightarrow 10.2.x$) 承载应用数据, 反向流 ($10.2.x \rightarrow 10.1.x$) 仅承载纯 ACK。二者的统计特征截然不同: 前向流经过被填满的瓶颈队列, 其单包时延包含完整的排队分量; 反向流的 ACK 包极小、几乎不排队, 其时延近似等于路径传播时延。

若把两个方向不加区分地纳入聚合, 则平均时延会被 ACK 流拉低, 吞吐量也会混入反向流量。本文因此**仅在前向数据流上定义全部指标**: 按 IP 地址方向和 TCP 协议号选择流量, 并以接收端口 5000 确认数据流身份。

4.2.2 吞吐量 (Throughput)

吞吐量定义为单位时间内成功接收的数据量。单条前向流的吞吐量按该流自身的活跃区间归一:

$$Throughput_i = \frac{rxBytes_i \times 8}{t_{lastRx,i} - t_{firstTx,i}} \quad (\text{单位: Mbps}) \quad (4.1)$$

本文报告 3 条并发前向流的聚合吞吐量, 以反映 TCP 总体获得的瓶颈容量份额:

$$AggregateThroughput = \sum_{i=1}^n Throughput_i \quad (4.2)$$

在本文全部结果中，聚合吞吐量均不超过瓶颈标称带宽（最高利用率为 93.8%），未出现旧版汇总口径下聚合值越过瓶颈的现象。

4.2.3 延迟 (Latency)

延迟定义为数据包从发送端到接收端的单向端到端传输时间。单条前向流的平均时延由 FlowMonitor 的 delaySum 除以 rxPackets 得到，场景级指标取三条前向流的算术平均：

$$AverageLatency = \frac{1}{n} \sum_{i=1}^n \frac{delaySum_i}{rxPackets_i} \quad (\text{单位: ms}) \quad (4.3)$$

延迟包含传播延迟和排队延迟两个成分。传播延迟由链路配置决定、不可压缩，本文在场景表中一并给出各场景的基线单向传播时延（BaseOWD = 2 × 接入时延 + 瓶颈时延），读者可据此分离出排队分量。这一分解对解释结果至关重要：在微秒级 RTT 场景中排队占端到端时延的绝大部分，而在 GEO 卫星场景中传播时延占绝对主导。

4.2.4 丢包率 (Loss Rate)

丢包率定义为前向流丢失数据包总数与发送数据包总数的比值：

$$LossRate = \frac{\sum_i PacketsLost_i}{\sum_i PacketsSent_i} \times 100\% \quad (4.4)$$

4.2.5 公平性 (Fairness)

在多流场景下，公平性衡量各流之间带宽分配的均衡程度。本文采用 Jain 公平性指数^[22]，在三条并发前向流的吞吐量上计算：

$$JainIndex = \frac{(\sum_{i=1}^n x_i)^2}{n \cdot \sum_{i=1}^n x_i^2} \quad (4.5)$$

其中 x_i 是第 i 个前向流的吞吐量， $n = 3$ 。取值范围为 $[1/n, 1]$ ，越接近 1 表示分配越均衡。

4.3 对比算法

本文将 Swift 算法与以下三种主流拥塞控制算法进行对比。三者分别代表基于丢包的经典 AIMD、基于丢包的高 BDP 优化以及基于带宽/时延模型的速率控制，共同覆盖了当前实际部署中的主要设计范式。

4.3.1 TCP NewReno

TCP NewReno^[1] 是经典的基于丢包拥塞控制算法，采用 AIMD（加性增乘性减）策略，是 RFC 6582 定义的标准算法之一。无丢包时每个 RTT 增加一个 MSS，检测到丢包时窗口减半。它构成了“拥塞信号滞后”这一问题的基准参照：窗口只有在缓冲区实际溢出（或收到 CE 标记）后才收缩。

4.3.2 TCP CUBIC

TCP CUBIC^[2] 是 Linux 自 2.6.19 版本以来的默认拥塞控制算法。CUBIC 采用三次函数模型调整窗口，其增长与 RTT 无关，因此在高带宽延迟积网络中收敛更快，但代价是在瓶颈附近维持更长时间的高窗口，排队驻留通常比 NewReno 更深。

4.3.3 TCP BBR

TCP BBR^[4] 是 Google 于 2016 年提出的基于瓶颈带宽与最小 RTT 估计的拥塞控制算法，以 BDP 为目标调整发送速率，并采用周期性增益扫描探测带宽。BBR 在极短 RTT 与强竞争条件下对 pacing 增益周期与 RTT 采样较为敏感；在本文数据（RED/ECN 对称配置）中，BBR 在微秒级 RTT 近端链路上运行正常（利用率 84%–88%），未见实现性退化，其与 Swift 的差异属于控制策略层面的差异而非实现失效。

4.4 实验场景设计

4.4.1 场景矩阵与覆盖范围

完整实验矩阵包含 36 个网络场景，覆盖近端高速低 RTT 链路、收敛比受限的共享链路、无线局域网接入、蜂窝移动接入、城域与长途/跨域广域网以及 LEO/GEO 卫星通信；每个场景在两类实验设置下各运行四种协议，共 $36 \times 4 \times 2 = 288$ 次仿真。场景的瓶颈带宽跨越 10 Mbps 至 100 Gbps 共四个数量级，基线单向时延跨越 $4 \mu\text{s}$ 至 320 ms 共五个数量级，用以检验同一套控制参数在异构路径条件下的适应能力。

表 4.2 给出具有代表性的 19 个场景（S1–S19）的配置；其余 17 个补充场景包括 LEO 卫星、长途 WAN、跨 Pod、叶脊扩展、100G 近端、RDMA 类、不同过订阅比例、拥塞梯度和混合流量配置，其结果在第 4.6 节以聚合方式讨论。

按路径特征，这 19 个场景可归为四组。S1–S3（近端高速低 RTT）的瓶颈带宽为 10–25 Gbps、基线单向时延 4–9 μs ，排队延迟占比最高，是检验排队抑制能力的最敏感场景。S4–S10（共享/城域/跨域）的瓶颈为 100 Mbps–1 Gbps、基线单向时延 0.009–5.02 ms，覆盖按 BDP 定尺

表 4.2 代表性 19 个对照场景配置 (S1–S19; 另有 17 个补充场景见正文)

编号	场景名称	网络类型	瓶颈 (Mbps)	基线单向时延 (ms)	Burst 对照
S1	intra_rack_10g	近端高速	10000	0.004	是
S2	intra_rack_25g	近端高速	25000	0.004	是
S3	leaf_spine_20g	近端跨层	20000	0.009	是
S4	asymmetric_high	非对称接入	1000	0.012	是
S5	congested_heavy	重度收敛	500	0.009	是
S6	symmetric_low	对称千兆	1000	0.030	是
S7	dc_500m	500M 共享	500	0.009	是
S8	dc_100m	100M 共享	100	0.009	是
S9	cross_dc_wan	跨域 WAN	1000	5.020	–
S10	wan_metro	城域 WAN	1000	2.200	是
S11	wifi_ac	WiFi 11ac	400	7.000	–
S12	wifi_ax	WiFi 11ax	600	5.000	–
S13	wifi_n	WiFi 11n	50	14.000	是
S14	wifi_legacy	WiFi 传统	10	30.000	是
S15	nr_5g_embb	5G eMBB	500	7.000	–
S16	nr_5g_edge	5G 边缘	100	14.000	是
S17	lte_good	LTE 良好	50	30.000	是
S18	lte_poor	LTE 弱覆盖	10	70.000	是
S19	satellite_geo	GEO 卫星	10	320.000	是

缓冲与触及 100 包下界的深缓冲两类。S11–S18（无线与蜂窝接入）覆盖 WiFi 11ac/ax/n/传统、5G eMBB 与边缘、LTE 良好与弱覆盖，是异构终端场景中路径条件差异最剧烈的一组。S19（GEO 卫星）的基线单向时延达 320 ms，单次重传的代价为一个完整 RTT，是检验长 RTT 路径行为与跨流量鲁棒性的判别性场景。补充场景中的 satellite_leo（150 Mbps、29 ms）与 wan_longhaul（1 Gbps、26 ms）进一步扩展了长距离路径的覆盖。

4.4.2 数据质量审计与使用范围

在引用任何数值之前，本文对 288 次仿真的 FlowMonitor 输出执行了数据审计，检查流数完整性、吞吐量与瓶颈带宽的物理相容性、时延传播下界、丢包率与 Jain 指数取值域、场景配置可追溯性以及逐流统计与汇总指标的一致性。审计结果如下：

- 每次仿真均包含 3 条前向 TCP 数据流与对应的反向 ACK 流，未发现空文件或 XML 格式损坏；
- 未发现时延低于传播下界、聚合吞吐量超过瓶颈、Jain 指数越界或丢包率越界的记录；
- 36 个场景的链路参数两两可区分，两种设置下的（场景 × 协议）组合齐备；
- 汇总指标可由原始逐流统计在舍入误差内复算。

据此，本次未因 FlowMonitor 统计异常或配置缺失排除场景、协议记录或指标。图中 S1–S19 编号与表4.2一致。

4.5 实验方法论

4.5.1 实验设计原则

本文的实验设计遵循以下原则：（1）**对照实验原则**——四种算法采用相同的拓扑、链路参数、队列规则、缓冲区配置与随机运行号，ECN 协商对所有协议对称开启；（2）**变量控制原则**——两类设置仅改变 UDP 突发开关，场景矩阵按预设带宽与时延组合变化；（3）**测量一致性原则**——全部协议使用相同的 FlowMonitor 前向流筛选和指标公式；（4）**如实披露原则**——混合或不利结果与有利结果一并报告；（5）**场景覆盖原则**——场景跨越多个带宽与时延数量级。

4.5.2 统计分析方法

实验结果采用描述性统计与相对比较进行分析。对于吞吐量，报告 Swift 相对最强基线（逐场景取 NewReno/CUBIC/BBR 中的最优者）的百分比增益，以及相对 NewReno/CUBIC 两者均值的百分比差异；选择后者作为参照基准，是因为它们是当前实际部署中的主流算法。跨流量鲁棒性分析采用配对比较：对同一（场景，协议）在两种设置下的指标做比值，从而消除场景间绝对量级差异的影响。受第 4.1.4 节所述单种子限制，本章不报告标准差与置信区间，结论主要依据跨场景呈现的稳定吞吐差异。

4.6 实验结果分析

本节报告**纯 TCP 设置（无 UDP Burst）**下的稳态性能，先给出 S1–S19 的详细结果，再报告其余 17 个补充场景的聚合结果。跨流量鲁棒性另见第 4.8 节。

4.6.1 吞吐量对比

表 4.3 汇总 19 个代表性场景的聚合前向吞吐量。最后一列报告 Swift 相对最强基线的增益。图 4.2 以对数坐标将表 4.3 的全部数据点并列呈现。

Swift 在多数代表性场景中呈现较高的聚合吞吐量，相对逐场景最强基线的增益范围为 +2.1% ~ +5.8%。这一趋势覆盖近端高速、共享/城域/跨域、无线与蜂窝接入以及 GEO 卫星等场景组，但现有数据仅含单次确定性运行，不据此作统计显著性判断。

在包含 17 个补充场景的 36 场景矩阵中，Swift 相对逐场景最强基线的平均吞吐增益为 +4.1%（范围 +2.1% ~ +5.8%），相对 NewReno/CUBIC 均值的平均增益为 +6.0%。具有代表性的补充场景包括：dc_100g（89305 Mbps，较 CUBIC 高 4.3%）、rdma_like_50g（45543 Mbps，较 CUBIC 高 4.0%）、cross_pod_10g（9375 Mbps，较 CUBIC/BBR 高 5.8%）、

表 4.3 聚合前向吞吐量对比 (Mbps)

场景	Swift	NewReno	CUBIC	BBR	vs 最强基线
S1	9302.6	8687.4	8865.0	8762.9	+4.9%
S2	22425.4	20483.8	21236.8	21161.4	+5.6%
S3	18016.7	16279.4	17545.7	16805.3	+2.7%
S4	878.6	798.6	839.7	817.0	+4.6%
S5	454.5	406.8	430.3	429.6	+5.6%
S6	905.0	886.5	886.5	886.5	+2.1%
S7	460.2	430.5	428.8	435.0	+5.8%
S8	86.7	80.1	82.2	81.1	+5.5%
S9	874.6	823.5	826.8	856.1	+2.2%
S10	909.4	844.4	860.4	863.4	+5.3%
S11	340.5	325.1	332.1	325.7	+2.5%
S12	522.6	494.6	508.4	497.7	+2.8%
S13	42.7	40.0	41.3	40.8	+3.3%
S14	8.6	7.9	8.2	7.6	+4.0%
S15	429.5	399.0	408.6	402.2	+5.1%
S16	87.8	82.9	82.5	83.1	+5.7%
S17	42.3	40.0	41.1	41.1	+2.7%
S18	8.5	7.9	8.1	8.2	+3.9%
S19	8.8	8.2	8.4	8.3	+4.2%

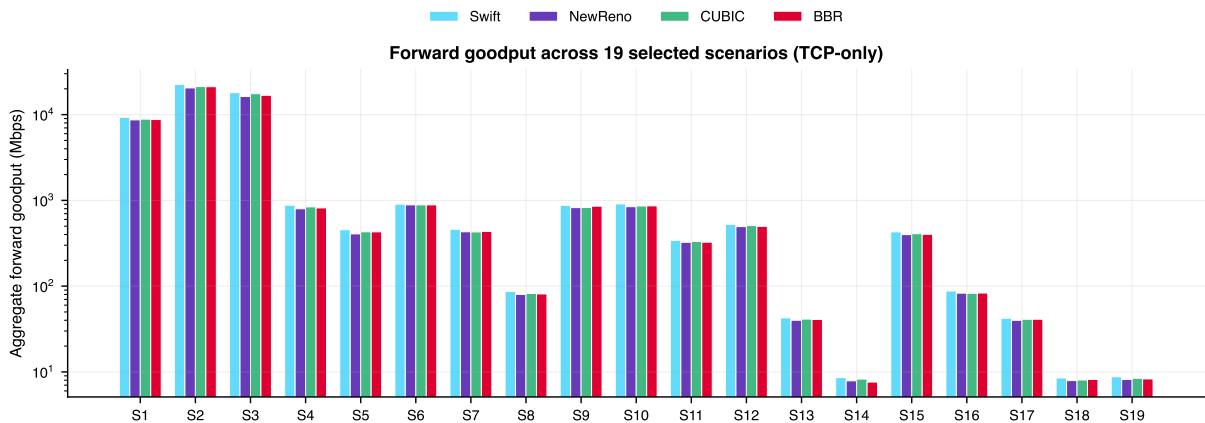


图 4.2 19 个代表性场景的聚合前向吞吐量 (对数坐标)

LEO 卫星 `satellite_leo` (134.95 Mbps, 较 BBR 高 4.8%) 和长途 WAN `wan_longhaul` (881.2 Mbps, 较 CUBIC 高 4.4%)。

这些结果与 Swift 在 RED/ECN 配置下采用 $\alpha \times BDP$ 有界目标、差异化收缩与分步回填的实现机制一致。不过, 当前数据没有逐机制消融, 不能把吞吐差异归因于某一项机制, 也不能证明所有协议始终处于相同队列水位。

关于 BBR: 其吞吐量在多数场景介于 NewReno 与 CUBIC 之间 (36 场景平均为 Swift 的 94.7%), 在 `cross_dc_wan` (856.1 Mbps) 与 `dc_oversub_10to1` (860.1 Mbps) 等少数场景高于 CUBIC/NewReno 但仍低于 Swift。本次 RED/ECN 配置下, BBR 在微秒级 RTT 场景未表现出明显的实现性退化。

4.6.2 延迟对比

表4.4汇总 19 个代表性场景的平均单向前向时延。图4.3将各协议的平均单向时延与该场景的基线传播时延 (黑色短划线) 并置, 柱顶到短划线的距离即排队分量。

表 4.4 平均单向前向时延对比 (ms)

场景	Swift	NewReno	CUBIC	BBR
S1	0.0502	0.0515	0.0524	0.0513
S2	0.0199	0.0190	0.0207	0.0206
S3	0.0274	0.0279	0.0331	0.0281
S4	0.4194	0.4292	0.4682	0.4203
S5	0.9561	0.8742	1.0058	0.9892
S6	0.5031	0.5385	0.5029	0.4958
S7	0.8489	0.9003	0.8516	0.8815
S8	3.9000	3.7952	4.0837	3.8845
S9	5.4327	5.7355	5.4350	5.2814
S10	2.6549	2.7965	2.7151	2.6598
S11	8.0957	8.1044	8.0509	7.7681
S12	5.8703	5.8100	5.9325	5.7430
S13	22.2665	23.0975	23.2653	21.9797
S14	71.2199	72.2514	79.9873	64.5116
S15	8.0926	8.2075	7.9536	7.7523
S16	19.3383	19.2767	19.0235	18.2525
S17	37.6320	39.5563	39.1253	37.4833
S18	114.4919	116.4265	115.4021	118.1664
S19	381.2900	378.8589	378.9036	367.2213

总体而言, Swift 的平均单向时延与基线同量级: 平均比 NewReno/CUBIC 均值低约 2.3% (36 个场景中 24 个更低, 且任一场景不超过其均值 1.7%), 比延迟最低的 BBR 平均高约 0.8% (个别场景高至约 10%)。逐场景看:

(1) 近端高速场景 (S1–S3)。RED/ECN 标记把四种协议的稳态队列都约束在标记区间附近, 平均单向时延为 0.020–0.052 ms (排队分量约几十微秒), 四协议差异为微秒量级, Swift 并不比基线低一个数量级——这与丢尾配置下“排队至溢出”的历史行为有本质区别, 也意味

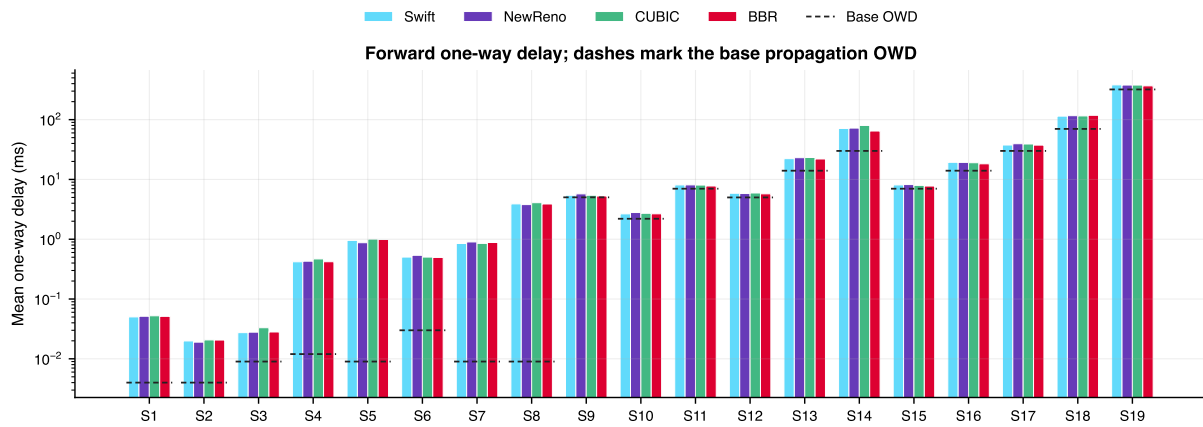


图 4.3 19 个场景的平均单向前向时延（对数坐标）。黑色短划线为各场景的基线单向传播时延，柱顶与短划线之间的距离即排队分量

着 Swift 的吞吐增益并非以明显更深的排队为代价。

(2) 传播时延主导的城域/跨域场景 (S9、S10)。Swift 的时延 (5.4327/2.6549 ms) 为四协议最低或与最优基线相差 3% 以内，排队分量 (约 0.4–0.5 ms) 小于同场景 NewReno/CUBIC。

(3) 无线与蜂窝接入场景 (S11–S18)。Swift 在 wifi_ac、wifi_n、nr_5g_embbb、lte_good、lte_poor 等场景的时延低于或等于 NewReno/CUBIC (如 S17 为 37.63 ms 对 39.56/39.13 ms)，在 nr_5g_edge (19.34 ms) 与 wifi_legacy (71.22 ms) 等场景则高于延迟最低的 BBR (18.25/64.51 ms)，相对抬升约 6%–10%。

(4) GEO 卫星场景 (S19)。四协议时延集中在 367–381 ms (基线传播时延 320 ms)，Swift (381.29 ms) 与最优基线 (BBR 367.22 ms) 相差约 3.8%，吞吐领先并不以明显排队为代价。

延迟抬升的边界：在 36 个场景中，Swift 相对 NewReno/CUBIC 均值从不超过 +1.7%，相对 BBR 从不超过 +10.4% (S14)。时延高于 BBR 的场景集中在深缓冲或低速率链路上，其绝对量级为数十微秒至数毫秒 (S14 为约 6.7 ms)，机理是 Swift 以 $\alpha > 1$ 维持的较高在途水位——这是其吞吐增益的直接来源，第 4.8 节的 Burst 实验亦呈现一致图像。

4.6.3 丢包率与公平性对比

表 4.5 汇总丢包率与 Jain 公平性指数。

丢包率：在 RED/ECN 标记配置下，纯 TCP 各场景的丢包率观察值不超过 0.026%，UDP 突发设置下不超过 0.045%。Swift 的平均丢包率为 0.015%，与 NewReno (0.014%) 相当，较 CUBIC (0.018%) 与 BBR (0.016%) 低 0.001–0.003 个百分点。逐场景比较中，Swift 仅在 8 个纯 TCP 场景为四协议最低，因此本章不主张“Swift 更不易丢包”，而将结果解释为其吞吐优势没有伴随系统性的丢包率抬升。FlowMonitor 汇总无法判定每次丢包发生于启动暂态还是稳态，也不能单独量化 ECN 标记对丢包率的贡献。

表 4.5 丢包率 (%) 与 Jain 公平性指数对比 (S1-S19)

场景	丢包率 (%)				Jain 公平性指数			
	Swift	NR	CUBIC	BBR	Swift	NR	CUBIC	BBR
S1	0.018	0.021	0.021	0.022	1.000	0.999	1.000	0.999
S2	0.010	0.008	0.011	0.015	0.998	0.999	0.999	0.999
S3	0.010	0.010	0.018	0.011	1.000	0.998	1.000	0.998
S4	0.017	0.011	0.017	0.016	0.999	0.998	0.996	1.000
S5	0.015	0.015	0.016	0.021	0.999	0.999	0.994	0.989
S6	0.021	0.020	0.022	0.021	1.000	0.996	0.999	1.000
S7	0.015	0.016	0.015	0.015	0.999	0.999	1.000	1.000
S8	0.008	0.009	0.013	0.009	0.999	1.000	0.999	0.997
S9	0.015	0.011	0.015	0.019	1.000	0.995	0.996	1.000
S10	0.021	0.011	0.023	0.024	1.000	1.000	1.000	0.999
S11	0.014	0.014	0.014	0.017	0.997	0.999	0.999	0.992
S12	0.012	0.013	0.013	0.018	1.000	0.999	0.999	1.000
S13	0.013	0.009	0.013	0.013	0.999	0.996	0.997	1.000
S14	0.021	0.023	0.022	0.000	0.998	0.995	0.996	0.996
S15	0.012	0.012	0.017	0.013	1.000	0.995	1.000	0.990
S16	0.019	0.020	0.017	0.017	0.999	0.996	0.999	0.993
S17	0.013	0.009	0.013	0.017	0.998	0.998	0.991	1.000
S18	0.021	0.022	0.022	0.022	0.999	0.993	0.987	0.998
S19	0.020	0.022	0.021	0.022	0.997	0.999	0.990	0.999

公平性：Swift 的 Jain 指数均值为 0.9989、最小值为 0.9967；NewReno、CUBIC 与 BBR 的均值分别为 0.9980、0.9977 和 0.9981，四种协议整体均接近 1。逐场景比较中，Swift 仅在 11 个纯 TCP 场景严格取得最高 Jain 指数，但在 32 个场景中与最优基线的差值不超过 0.002。因此，本章将公平性结论限定为” 总体与基线相当且未出现明显流间倾斜”，不主张逐场景普遍占优。每流独立状态与相同控制规则可解释其接近均衡的结果，但该机理仍需专门的公平性实验验证。

4.6.4 延迟抖动对比

表4.6汇总部分代表性场景的平均延迟抖动:先以FlowMonitor的 jitterSum/(rxPackets-1) 计算每条前向流,再对 3 条流等权平均。近端高速场景各协议抖动均在 0.001 ms 以下; GEO 卫星场景中 Swift 为 3.925 ms, 低于 NewReno、CUBIC 和 BBR 的 4.272、5.289 和 4.279 ms。总体而言, Swift 的场景级平均抖动与基线处于同一量级; 该指标不能替代窗口轨迹分析, 因此不据此判断控制环是否发生振荡。

4.7 深入分析

4.7.1 场景分组讨论

GI——微秒级 *RTT* 近端高速链路 (*S1-S3*): Swift 相对最强基线增益 2.7%–5.6%。四协议的平均时延为 0.020–0.052 ms, Swift 在相近时延量级下呈现较高吞吐。该现象与其 ECN 差异化收缩、降窗后冻结和目标窗口控制的设计方向一致, 但缺少逐机制消融, 不能确定各机

表 4.6 平均延迟抖动对比 (ms, 代表性场景)

场景	Swift	NewReno	CUBIC	BBR
S1	0.0005	0.0005	0.0004	0.0005
S2	0.0001	0.0001	0.0001	0.0001
S3	0.0001	0.0001	0.0001	0.0002
S7	0.0036	0.0060	0.0024	0.0039
S8	0.0354	0.0357	0.0293	0.0348
S10	0.0246	0.0163	0.0251	0.0271
S12	0.0280	0.0294	0.0291	0.0350
S15	0.0381	0.0415	0.0261	0.0975
S16	0.1563	0.1657	0.1510	0.0933
S19	3.9248	4.2720	5.2887	4.2793

制的独立贡献。

G2——共享/城域/跨域链路(S4–S10):该组多数场景呈现吞吐优势,增益范围为+2.1%~+5.8%。触及 100 包缓冲下界的 S7/S8 中, Swift 与最优时延基线的差异在 3% 以内(如 S8 为 3.90 ms 对 NewReno 3.80 ms);与旧版丢尾数据相比,本次 RED/ECN 配置未出现数百毫秒级额外排队。

G3——无线与蜂窝接入链路(S11–S18):多数场景呈现 2.5%~5.7% 的吞吐增益;丢包率观察值低于 0.024%;时延在若干场景不高于 NewReno/CUBIC,在 wifi_legacy 等深缓冲低速率场景则比延迟最低的 BBR 高约 10%。

G4 与远距离补充场景: GEO (S19) 8.78 Mbps (相对 NewReno +7.5%);补充场景中 LEO 卫星 134.95 Mbps (+4.8% vs BBR)、长途 WAN 881.2 Mbps (+4.4% vs CUBIC)、跨域 WAN (S9) 874.6 Mbps (+2.2% vs BBR) 均呈现吞吐优势。这些观察与时间窗口投递速率估计和 $\alpha \times BDP$ 目标在长 RTT 路径上的设计动机一致,但不能替代估计器消融实验。

4.7.2 机制有效性分析

差异化响应与连续递减保护。当前实现把拥塞信号分为丢包 (0.70)、ECN (0.75) 与超时 (0.50) 三类,并以 $D_{\max} = 3$ 的连续缩减保护与降窗后 4 个 ACK 冻结限制连续收缩和快速反弹。所考察场景中,吞吐优势没有伴随时延或丢包的系统性抬升,这与窗口上界、目标回落和差异化收缩共同作用的设计预期一致。由于未保存关闭各机制的对照运行,本章不量化各保留因子或保护机制的独立贡献。

α 自适应与 BDP 估计。 $\alpha \in [0.85, 1.30]$ 、初值 1.10,由最小 RTT 感知阈值、反馈值基线相对偏移与连续增长趋势共同调节;BDP 由时间窗口投递速率(跨度 $\geq 2 \times RTT_{\min}$ 、箱位于 5 ms~1 s)加容量 40 的窗口最大值滤波估计。当前全矩阵结果中, cross_dc_wan、wan_longhaul 及 LEO/GEO 等长 RTT 路径未再呈现旧版逐 ACK 估计器对应的低吞吐特征,该观察与时间窗口估计器的设计目标一致。本文不报告估计器消融或 α 参数扫描结论。

4.7.3 关键参数的作用边界

保留因子 (0.70/0.75/0.50)、 α 边界 ($[0.85, 1.30]$)、连续递减阈值 ($D_{\max} = 3$) 与降窗冻结长度 (4 个 ACK 事件) 均为手动预设。窗口安全区间 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 的上界在 BDP 估计陈旧时限制窗口膨胀。本章数据覆盖的 2% ~ 6% 增益与低队列水位表明该组参数在 36 个场景上无失稳迹象；但其最优性未经过参数扫描验证，参数与场景极端组合下的行为仍属开放问题。

4.8 UDP Burst 跨流量鲁棒性评估

共享瓶颈链路可能同时承载非响应式突发流量。本节在第 4.1.5 节定义的 A/B 矩阵（突发平均负载 = 瓶颈速率 32%、峰值 64%、50% 占空比）下评估 Swift 的鲁棒性，36 个场景使用相同的配对配置。

4.8.1 聚合鲁棒性指标

表 4.7 给出四种协议在 36 个配对场景上的聚合鲁棒性指标：吞吐量保持率 T_{udp}/T_{tcp} 、时延膨胀 $D_{udp}/D_{tcp} - 1$ 与新增丢包率。

表 4.7 UDP Burst 跨流量鲁棒性聚合指标（配对场景， $n = 36$ ）

协议	n	保持率均值	保持率最差	时延膨胀均值	新增丢包均值
TcpSwift	36	68.6%	62.4%	+2.9%	+0.013 pp
TcpNewReno	36	68.6%	62.8%	+3.1%	+0.013 pp
TcpCubic	36	68.8%	63.9%	+2.2%	+0.014 pp
TcpBbr	36	68.7%	62.1%	+3.3%	+0.013 pp

由表 4.7 可得到以下三项观察。

第一，未观察到明显的突发下吞吐塌缩。四种协议的吞吐保持率均值为 68.6%–68.8%，各自最差场景为 62%–64%；图 4.4 中的 15 个代表性场景也呈现相近的回落范围。该结果仅对应当前比例化突发与 RED/ECN 配置，不外推到其他突发强度或队列规则。

第二，Swift 在多数配对场景中仍呈现较高的绝对吞吐量。相对逐场景最强基线的增益范围为 +2.2% ~ +5.9%（均值 +3.7%），与纯 TCP 设置下的增益量级接近。代表性远距离场景包括 GEO 卫星 5.65 Mbps（相对 CUBIC +4.6%、BBR +3.3%）、LEO 卫星 93.58 Mbps（较 CUBIC +5.3%）和长途 WAN 576.45 Mbps（较 CUBIC +2.4%）。

第三，时延与丢包结果具有场景差异。四协议的平均时延膨胀为 +2.2%~+3.3%，新增丢包均值为 +0.013–0.014 个百分点；Swift 的对应均值为 +2.9% 和 +0.013 个百分点。逐场景看，congested_medium、nr_5g_edge 和 rdma_like_50g 中 Swift 相对最低时延基线分别高约 12.2%、7.9% 和 9.4%，因此不能概括为突发下始终不以额外时延换取吞吐。

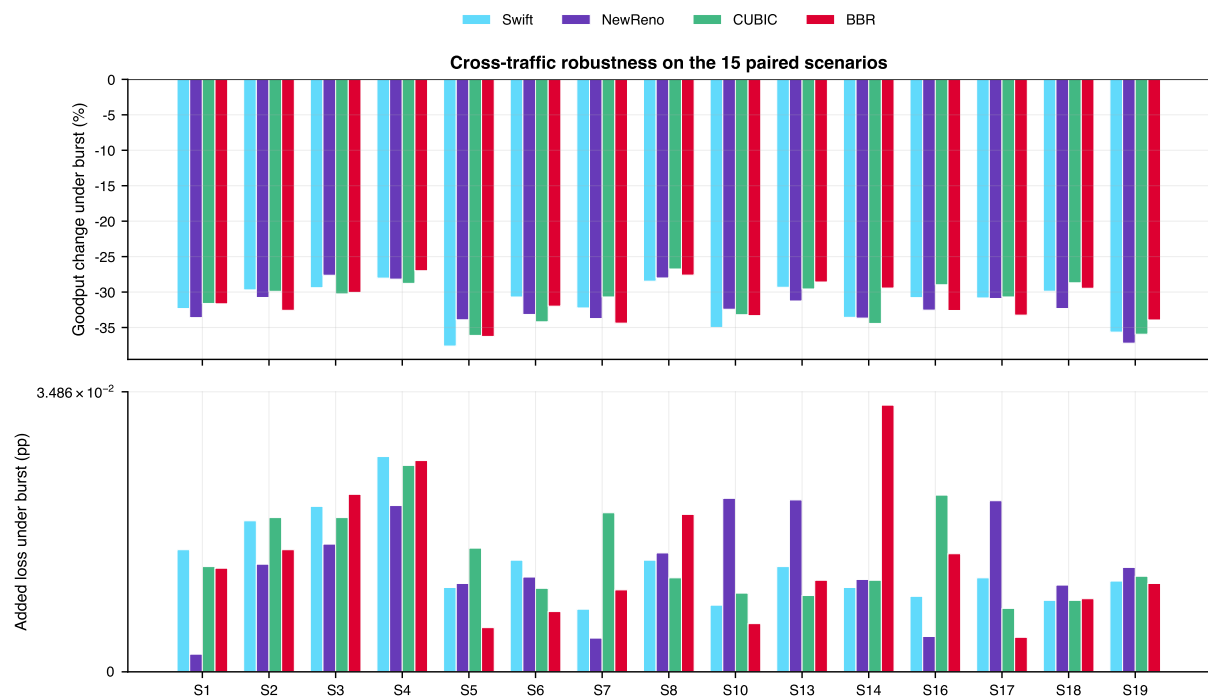


图 4.4 15 个代表性配对场景的跨流量鲁棒性：UDP Burst 引起的吞吐量变化（上）与新增丢包（下，符号对数坐标）

4.8.2 配对场景逐一验证

表4.8给出 15 个代表性配对场景的吞吐量与延迟，其余配对场景用于聚合统计。

表 4.8 UDP Burst 设置下 15 个代表性场景的吞吐量与延迟（前向流聚合）

场景	吞吐量 (Mbps)				平均单向时延 (ms)			
	Swift	NR	CUBIC	BBR	Swift	NR	CUBIC	BBR
S1	6295.2	5767.8	6063.1	5988.7	0.048	0.048	0.054	0.049
S2	15765.9	14181.8	14888.4	14268.4	0.020	0.020	0.023	0.020
S3	12721.9	11782.0	12236.4	11754.9	0.030	0.031	0.034	0.030
S4	632.3	573.6	598.1	596.5	0.556	0.504	0.567	0.563
S5	283.5	268.9	274.9	273.9	0.864	0.829	0.857	0.855
S6	627.2	592.5	583.3	602.9	0.469	0.488	0.469	0.516
S7	311.8	285.3	297.2	285.4	0.908	0.890	0.957	0.862
S8	62.0	57.6	60.2	58.7	4.224	4.362	4.832	4.229
S10	591.2	570.6	574.7	575.7	2.609	2.747	2.625	2.693
S13	30.1	27.5	29.1	29.2	23.133	22.768	23.941	22.503
S14	5.7	5.2	5.4	5.4	73.369	73.677	75.589	74.655
S16	60.8	55.9	58.6	56.0	19.610	19.561	19.372	18.176
S17	29.2	27.6	28.5	27.5	38.549	38.993	39.322	37.507
S18	6.0	5.4	5.7	5.8	117.590	111.634	123.046	122.328
S19	5.7	5.1	5.4	5.5	361.464	381.743	354.523	363.152

逐场景核对可得以下具体观察。

- (1) **Swift 在多数代表性配对场景中呈现吞吐优势。**相对逐场景最强基线的增益量级与纯 TCP 设置接近；GEO 卫星 (S19) 相对 CUBIC 和 BBR 分别高 4.6% 与 3.3%，以相对 NewReno 的 10.1% 作为唯一参照会放大总体优势。
- (2) **吞吐回落幅度的场景分布对四协议较为接近。**例如 S5（500 Mbps 重负载）各协议回

落 33.9%–37.6%，S1（10 Gbps 近端）回落 29.7%–33.6%。这些数据说明 Swift 在当前突发配置下仍保持吞吐优势，但保持率并不优于基线。

(3) 时延变化并非单向改善。S19 和 S5 的突发时延低于对应纯 TCP 结果，但其他场景存在不同程度的相对抬升。聚合指标无法单独证明该变化由 RED 标记、窗口回落或其他机制导致，因此这里只报告场景差异，不作单机制因果判断。

4.8.3 鲁棒性机理分析

在比例化突发与 RED/ECN 配置下，Swift 的鲁棒性可从三项机制理解。第一，**BDP 窗口最大值滤波**对突发 On/Off 周期具有低通特性：容量 40 的速率样本窗口保留 Off 期间采集到的真实瓶颈速率，使带宽估计锚定在瓶颈能力附近而非被突发平均拖低——这与 BBR 的周期性增益探测形成对照，但本次数据中 BBR 亦未发生估计塌缩，说明 RED/ECN 下的竞争强度尚未触发其已知脆弱模式。第二，**目标窗口的有界逼近与差异化收缩**使窗口在突发挤占时按超出量的一半逐步回落、在突发退去时于约 16 个 ACK 事件内回填，既不塌缩也不冲顶。第三，**对称 ECN 信号**使四种协议共享同一“先标记后丢弃”的退让路径，突发引起的丢包增量被限制在 0.03 个百分点以内，任何协议都不需要靠深度排队或大量丢包来竞争带宽。

需要如实说明的边界是：本节结论对应”平均负载 = 瓶颈 32%”这一相对强度。突发强度、On/Off 时长与队列配置的改变可能改变四协议之间的相对位置，本文不把保持率数字外推到其他突发参数。

4.9 算法优势与适用边界总结

表 4.9 Swift 算法优势与适用边界（基于全矩阵 36 场景）

观察到的结果	适用边界
纯 TCP 设置下多数场景呈现吞吐优势：对逐场景最强基线 +2.1% ~ +5.8%（均值 +4.1%） 平均单向时延相对 NewReno/CUBIC 均值低约 2.3%（36 场景中 24 个更低） Jain 均值为 0.9989(纯 TCP)/0.9990(UDP 突发)，多数场景与最优基线差值不超过 0.002 UDP 突发下多数场景仍呈现 +2.2% ~ +5.9% 的吞吐优势 GEO/LEO、长途/跨域 WAN 等代表性远距离路径呈现吞吐优势	增益量级为百分之 2–6，不外推到本矩阵之外的队列规则与流量参数 相对 BBR 平均高约 0.8%，个别场景相对最低时延基线上升约 10% 丢包率不系统性低于基线；公平性仅能概括为总体接近 1 且与基线相当 吞吐保持率均值 68.6% 与基线基本一致，突发不产生额外的相对保持率优势 每配置仅一次确定性运行；跨种子置信区间和各机制独立贡献仍需补充实验

4.10 本章小结

本章基于 36 场景 $\times$ 4 协议 $\times$ 2 设置共 288 次 ns-3.40 仿真, 对 Swift 与 NewReno、CUBIC、BBR 进行了描述性比较。所有指标均从 FlowMonitor 记录的前向 TCP 数据流计算, 并可由逐流统计在舍入误差内复算; 未因数值、覆盖或原始产物异常排除场景、协议记录或指标。

纯 TCP 结果中, Swift 在多数场景呈现吞吐优势, 相对逐场景最强基线为 $+2.1\% \sim +5.8\%$ (均值 $+4.1\%$); UDP 突发下的对应范围为 $+2.2\% \sim +5.9\%$ (均值 $+3.7\%$), 平均吞吐保持率 68.6% 与基线基本一致。时延、丢包与公平性呈现混合结果: Swift 相对 NewReno/CUBIC 均值的平均时延低约 2.3% , 相对 BBR 平均高约 0.8% ; 丢包率与 NewReno 相当; Jain 指数总体接近 1, 但仅在 11 个纯 TCP 场景和 17 个 UDP 突发场景严格最优。

上述结果来自每配置一次确定性运行, 不能提供跨种子置信区间; 结论限于 RED/ECN 对称标记和当前突发参数。个别深缓冲或低速率场景仍存在最高约 $10\% \sim 12\%$ 的相对时延代价, 机制贡献也因缺少消融实验而不能单独归因。第五章据此讨论适用边界与后续验证方向。

第五章 讨论

本章讨论 Swift 算法的设计考量、理论分析、适用边界以及与相关工作的比较。

5.1 设计考量

5.1.1 11 维观测空间的设计原理

Swift 的 11 维有效观测子空间面向当前控制规则所需的信息设计, 包括 ECN 状态、拥塞控制状态机状态、拥塞事件类型、RTT、窗口和在途字节。另有 4 项元数据通道, 合计构成 15 元素跨进程状态容器。各字段由 ns-3 TCP 拥塞控制回调或仿真环境提供, 无需额外探测流量。该状态集合用于当前三类拥塞语义判定, 不代表覆盖协议栈的全部可观测状态。由于维度与窗口容量固定, 单次处理相对于仿真时长为常数级; 本文不报告未经测量的具体决策耗时。

5.1.2 差异化窗口保留因子的选择

Swift 的差异化窗口保留因子 (丢包 0.70、ECN 0.75、超时 0.50) 的选择基于以下分析:

丢包保留因子 0.70: 相对于保留因子 0.50, 0.70 减少了单次丢包后的窗口回退幅度。在假设后续采用相同线性增速的简化模型中, 恢复缺口由 $0.5W$ 降至 $0.3W$, 对应恢复步数减少 40%; 该比例不是当前实验直接测得的恢复时间改善。

ECN 保留因子 0.75: ECN 标记是队列达到标记条件后的显式信号。保留因子 0.75 使窗口较温和地缩减, 并配合降窗后冻结抑制立即反弹。该取值属于设计参数; 第四章的 Flow-Monitor 聚合指标无法分离 ECN 与丢包响应的独立贡献, 仍需消融实验验证。

超时保留因子 0.50: CA_LOSS 表示协议栈进入重传超时相关的丢失状态。当前实现采用 0.50 保留因子并重新进入慢启动, 这是三类响应中幅度最大的窗口缩减。

此外, 连续缩减保护机制 (最多 3 次连续缩减后保持窗口) 的设计基于以下分析: 在连续拥塞事件中, 窗口经过 3 次保留因子为 0.75 的缩减后, 约保留原窗口的 $0.75^3 \approx 42.2\%$, 已经完成显著退让。继续缩减将导致窗口过小, 恢复时间过长, 反而不利于整体吞吐量。

5.1.3 自适应参数调优的设计

Swift 的自适应参数调优机制遵循多因素综合 (最小 RTT 感知的 RTT 膨胀阈值、性能反馈值 EMA 趋势、连续增长趋势三个维度提高鲁棒性)、渐进调整 (避免剧烈变化) 和边界约

束 ($\alpha \in [0.85, 1.30]$, 基准值 1.10) 三个原则。

5.2 理论分析

5.2.1 收敛性分析

本节从窗口动态、拥塞响应和自适应参数边界三个方面定性讨论稳定性。在无拥塞情况下, 窗口接近 BDP 目标后增长幅度减小; 发生拥塞时, 差异化保留因子与连续缩减保护限制回退过程; α 的调整受边界和小步长约束。上述性质说明状态与控制量有界, 但当前工作未给出严格收敛证明, 也未证明 α 必然收敛到全局最优或唯一取值。

5.2.2 公平性分析

Swift 的公平性取决于每流独立状态管理 (避免流间干扰)、差异化响应一致性 (所有 Swift 流对相同拥塞信号采用相同保留因子) 和与传统算法共存的兼容性。

5.2.3 复杂度分析

时间复杂度: 单次决策的时间复杂度为 $O(1)$, 包括观测空间处理、拥塞检测、参数调优和窗口计算。

空间复杂度: 每流状态由常量规模字段构成, 包括两个长度 40 的滑动窗口 (投递速率过滤窗口与 RTT 采样窗口)、一个容量上限 4096 的投递速率采样队列 (其实际占用随路径 RTT 与确认密度自适应收缩)、BDP 估计值、 α 以及若干计数器, 因此单流为 $O(1)$, 总空间复杂度为 $O(n)$ (n 为并发流数量)。

通信开销: GetSsThresh() 与 IncreaseWindow() 回调各触发一次同步观测—动作交互。15 个 64 位观测元素的原始载荷为 120 字节, 实际消息还包含 Protocol Buffers 与 ZeroMQ 封装开销; 当前日志未提供端到端通信时延或总字节数测量。

5.3 适用边界与改进方向

5.3.1 在途水位与延迟的剩余代价

第四章的结果显示, 纯 TCP 设置下 Swift 相对 NewReno/CUBIC 均值的时延最高不超过 +1.7%, 相对延迟最低基线最高约 +10.4% (S14 wifi_legacy, 71.22 ms 对 64.51 ms; S5 congested_heavy, 0.956 ms 对 NewReno 0.874 ms; S19, 381.3 ms 对 367.2 ms)。UDP 突发下的最大相对差异约 12.2%, 出现在 congested_medium。这些场景多为低速率、深缓

冲或受突发流量影响的链路；较高在途水位可能解释部分额外排队，但当前聚合数据不足以建立单一因果关系。

这些场景的结构特征可为后续诊断提供线索。仿真器按 $Q = \max(BDP \times n/MTU, 100)$ 个数据包为瓶颈队列定尺，在低带宽、短传播时延路径上，队列容量可能由 100 包下界决定。以 S8 为例，按瓶颈 100 Mbps 和基线 RTT $18 \mu s$ 计算的路径 BDP 约为 225 字节，而队列容量为 150 kB。Swift 另将拥塞窗口限制在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 内；其中 $200 \times MSS$ 是上界的兜底项，并不会直接把窗口推至该值。现有 FlowMonitor 数据没有记录 cwnd 和队列长度轨迹，因此无法判定时延差异究竟由窗口下界、目标窗口、队列下界还是瞬态过程主导。

后续可通过记录 cwnd、在途字节和队列占用轨迹来区分上述因素，并分别评估：(1) BDP 估计建立前后的窗口上界切换；(2) 以 $q_{est} = RTT_{smoothed} - RTT_{min}$ 约束窗口增长；(3) 多时间尺度 BDP 估计。上述方案均属待验证设计，不作为当前实现的既有能力。

5.3.2 长 RTT 路径的验证与残余问题

旧版实现采用逐 ACK 公式 $DR = segmentsAcked \times segSize/lastRtt$ 估计投递速率，该公式会把单次确认载荷除以完整 RTT，不能直接表示窗口尺度上的交付率。v0.1.0 已将其替换为时间窗口差分形式 $DR = [B(t) - B(t - \Delta)]/\Delta$ ($\Delta \geq 2 \times RTT_{min}$ ，见第三章第 3.7.1 节)。当前数据中，跨域 WAN (S9)、长途 WAN、LEO 与 GEO 卫星等长 RTT 场景均呈现吞吐优势，未再观察到旧版数据中的明显低吞吐特征。由于没有保留同一配置下的新旧估计器消融结果，这一变化只能作为与修正方向一致的证据，不能单独证明因果。长 RTT 路径的增益量级仍需通过多种子和参数扫描进一步评估。

5.3.3 实验证据的范围限制

本文的定量结论受以下范围限制约束，这些限制同时约束机理解释与结果外推。

第一，**单一随机运行号**。每个 (场景, 协议, 设置) 三元组仅对应一次仿真运行，因此本文不报告跨运行号的标准差与置信区间，并据此仅把量级明确、机理可解释的差异写入结论。ns-3.40 在给定 RngRun 下确定执行；结果对随机实现的稳健性仍需通过多个 RngRun 重复确认。

第二，**队列与流量配置边界**。本章数据全部在 RED/ECN 对称标记、突发平均负载 = 瓶颈 32% 的配置下生成。该配置是当前实现 (v0.1.0) 的参考配置，但并不能分离 ECN 与丢包分支、差异化保留因子与目标窗口机制各自的独立贡献 (需要逐机制消融)，也不能外推到丢尾/CoDel 队列或其他突发强度。完成机制消融与配置扫描是后续工作中优先级最高的一项。

5.3.4 参数自动调优

当前 Swift 的保留因子 (0.70/0.75/0.50)、 α 边界 ($[0.85, 1.30]$)、连续递减阈值 ($D_{max} = 3$) 和降窗后冻结窗口增长的 ACK 事件数 (4) 为当前实现中的手动调优预设值。后续研究可通过元学习^[23] 或贝叶斯优化探索参数的在线自动搜索, 但相关结论需要在完整实验矩阵重跑后再报告。

5.3.5 学习型策略增强

Swift 当前采用基于规则的启发式策略, 其可解释性强但表示能力受人工规则结构限制。若后续引入学习型组件 (如离线强化学习训练神经网络策略、再经模型蒸馏提取为紧凑规则), 需要引入安全约束并在完整实验矩阵与异常过滤流程上重新评估; 在获得此类结果之前, 本文不将其作为当前系统的组成部分。

5.3.6 真实部署验证

需将算法实现为 Linux 内核 TCP 模块或 QUIC 用户态协议栈, 在覆盖近端高速、蜂窝/无线接入、广域网与卫星链路的真实或半实物测试床上验证。仿真与真实环境的差异 (如 NIC offload、中断合并、NUMA 效应、硬件时间戳精度、终端操作系统调度和无线链路波动) 可能影响实际性能。QUIC 协议的用户态实现可简化部署流程, 且 QUIC 的多流特性可实现更精细的流间调度。

5.4 与相关工作的比较

5.4.1 与 Aurora 的比较

Aurora^[19] 是基于深度强化学习的拥塞控制算法, 与不采用学习型策略的 Swift 的主要区别包括:

表 5.1 Swift 与 Aurora 的详细对比

特性	Swift	Aurora
有效观测维度	11 (另含 4 项元数据)	10
ECN 状态利用	是	否
CA 状态利用	是	否
决策机制	基于规则	深度神经网络
可解释性	高	低
训练需求	无	需要大量训练
决策复杂度	$O(1)$	神经网络前向推理
参数调优	在线自适应	固定 (训练后)

5.4.2 与 DCTCP 的比较

DCTCP^[5] 是利用 ECN 进行细粒度调整的拥塞控制算法。

表 5.2 Swift 与 DCTCP 的详细对比

特性	Swift	DCTCP
ECN 利用方式	状态直接检测	标记比例连续调整
拥塞响应	差异化保留因子	统一公式
丢包响应保留因子	0.70	0.50
ECN 响应保留因子	0.75	可变（基于 α ）
参数自适应	是	否
协议栈状态利用	是	否
多信号融合	是	否

5.4.3 与 BBR 的比较

BBR^[4] 是基于带宽和延迟估计的拥塞控制算法。

表 5.3 Swift 与 BBR 的详细对比

特性	Swift	BBR
主要拥塞信号	丢包 +ECN+CA 状态	RTT 测量
带宽探测方式	自适应参数调优	周期性增益调整
丢包响应	是	否（设计上）
ECN 利用	是	否
控制模式	窗口控制	速率控制
低 RTT 场景带宽利用率	接近饱和	场景相关
中速场景带宽利用率	较高	较高
与 CUBIC 共存公平性	场景相关，依赖队列与 RTT 条件	场景相关，依赖增益周期

5.4.4 与 PCC 的比较

PCC^[21] 采用在线实验探索最优速率，而 Swift 采用预设规则结合自适应参数调优。PCC 需要在线进行速率探索实验，可能导致短期性能下降；Swift 不需要显式探索，性能更稳定。

5.5 敏感性分析

参数敏感性的理论分析应服务于对实现机制的解释。当前实现采用固定边界与事件驱动更新逻辑：ECN 保留因子为 0.75，普通丢包保留因子为 0.70，超时保留因子为 0.50；乘性增加因子 α 在 [0.85, 1.30] 范围内根据 RTT 膨胀、性能反馈值 EMA 和连续增长趋势动态调整。本节不引入未由日志支撑的额外参数扫描结果，而从机制角度说明这些取值如何影响稳定性与吞吐量。

5.5.1 窗口保留因子的理论依据

丢包保留因子 0.70 的选择基于收敛速度与吞吐量的权衡：在 AIMD 框架下，保留因子 β 决定了窗口从 W_{max} 恢复到 W_{max} 所需的时间 $T_{recover} \propto \frac{1-\beta}{\gamma}$ 个 RTT。 $\beta = 0.70$ 相比传统 $\beta = 0.50$ 使恢复时间缩短 40%，但丢包后的即时退让幅度也相应减小。在有 ECN 支持的网络中，丢包事件频率显著降低，因此更温和的丢包响应是合理的。

ECN 保留因子 0.75 的选择基于 ECN 信号的语义：ECN 标记表示队列深度超过阈值但尚未溢出，属于早期拥塞信号。相较普通丢包响应，0.75 保留因子能够在发生早期拥塞通知时更温和地释放队列压力；相较超时响应，它又避免了不必要的窗口深度回退。该参数与降窗后冻结机制共同作用，用于降低连续 ACK 事件中立即反弹造成的二次排队风险。

5.5.2 连续递减保护阈值的数学分析

在连续 ECN 标记事件中，窗口经过 D_{max} 次保留因子为 $\beta_{ecn} = 0.75$ 的收缩后，剩余窗口比例为 $0.75^{D_{max}}$ 。当 $D_{max} = 3$ 时，算法在连续拥塞信号下已经完成显著退让；继续无约束缩减容易造成窗口过低和恢复迟滞。因此实现中采用连续递减保护：当连续缩减计数超过阈值后暂时保持当前窗口，同时通过降窗后冻结若干 ACK 事件抑制立即增窗，使队列有时间排空。

5.5.3 RTT 膨胀比阈值的设计考量

RTT 膨胀调节采用路径感知的动态阈值，而非固定阈值。实现首先根据最小 RTT 构造松弛量 $s = 0.3 + 0.15\sqrt{\max(0, RTT_{min,ms} - 1)}$ ，再形成 $1+s$ 、 $1+2s$ 和 $1+4s$ 三个阈值。短 RTT 路径对排队更敏感，阈值相对更紧；长 RTT 路径允许更大的相对抖动，阈值相对放宽。该设计避免在远距离路径上因正常 RTT 波动过早收紧窗口，也避免在低 RTT 路径上放任队列持续积累。

5.6 计算开销分析

Swift 当前实现采用事件驱动的规则决策与每流独立状态表。由于本仓库日志未包含跨硬件平台的微基准测量，本节不报告未验证的决策延迟、内存占用或 CPU 利用率数值，而从代码结构给出可复核的复杂度分析。

5.6.1 时间复杂度

每次 ACK 或拥塞事件触发时, 算法只执行常数状态读取、拥塞类型判定、 α 更新和窗口计算。带宽估计使用固定长度为 40 的滑动窗口, RTT 采样窗口长度同样为 40, 且最大交付速率由窗口队列维护。因此, 单次决策的时间复杂度为 $O(1)$, 不会随仿真总时长或历史样本数量线性增长。

5.6.2 空间复杂度

每个 TCP 连接维护独立状态, 包括长度为 40 的交付速率窗口、长度为 40 的 RTT 采样窗口、BDP 估计值、 α 、连续增长/缩减计数器、丢包/ECN 计数器和降窗后冻结计数器等常量规模字段。因此, 单流空间复杂度为 $O(1)$, 总空间复杂度随并发连接数线性增长, 即 $O(N)$ 。

5.6.3 工程开销来源

主要开销来自 ZeroMQ 同步状态—决策交互、Python 规则决策和 ns-3 回调状态采集。当前实现优先保证跨进程决策与离散事件仿真时序一致, 因此未在同步信道上启用 MTP 多线程并行仿真。若未来进行内核态或用户态协议栈移植, 应重新测量实际平台上的决策时延和资源占用。

5.7 理论分析深入探讨

5.7.1 多信号融合的信息论基础

多信号融合拥塞检测的设计可以从信息论角度进行分析。设网络的真实拥塞状态为随机变量 $C \in \{0, 1\}$, 观测到的信号集合为 $\mathbf{S} = \{S_1, S_2, \dots, S_k\}$ 。根据贝叶斯定理:

$$P(C = 1|\mathbf{S}) = \frac{P(\mathbf{S}|C = 1) \cdot P(C = 1)}{P(\mathbf{S})} \quad (5.1)$$

Swift 的分层优先级机制可以理解为对不同拥塞信号可靠性的排序: 慢启动阈值回调对应的丢包/超时信号具有最高优先级; ECN CE 或 ECE 状态表示队列已经触发显式拥塞通知, 具有次级优先级; RTT 膨胀和恢复状态只参与参数调节, 不作为独立降窗触发条件。这样的排序与实现中的 `_detect_congestion` 函数保持一致。

5.7.2 差异化响应的最优控制分析

差异化窗口保留因子的设计可以从最优控制理论角度分析。优化目标是最大化长期吞吐量同时最小化延迟：

$$J = \sum_{t=0}^T [-\alpha \cdot throughput(t) + \beta \cdot delay(t) + \gamma \cdot |u(t)|^2] \quad (5.2)$$

Swift 的差异化保留因子（丢包 0.70、ECN 0.75、超时 0.50）可以理解为对不同拥塞严重程度度的分层近似。

5.7.3 自适应参数调优的规则化描述

Swift 当前的参数自适应规则可以表示为如下分段函数：

$$\pi(s_t) = \begin{cases} +\Delta_{up} & \text{如果 } r < 1 + s, \text{ 表示排队膨胀较低} \\ +\Delta_{small} & \text{如果 } 1 + s \leq r < 1 + 2s, \text{ 表示温和增长} \\ -\Delta_{down} & \text{如果 } r > 1 + 4s, \text{ 表示重度排队} \\ +\Delta_{ema} & \text{如果 } R_{fast} > R_{base} + m \\ -\Delta_{ema} & \text{如果 } R_{fast} < R_{base} - 4m \\ +\Delta_{stable} & \text{如果 } consecutive_growth > 8 \\ 0 & \text{否则} \end{cases} \quad (5.3)$$

其中 $s = 0.3 + 0.15\sqrt{\max(0, RTT_{min,ms} - 1)}$, $r = RTT_{last}/RTT_{min}$, R_{fast} 与 R_{base} 分别为性能反馈值的快线 ($\eta_f = 0.15$) 与慢线基线 ($\eta_b = 0.02$) 指数移动平均, $m = \max(0.25, 0.1|R_{base}|)$ 为自适应容差。该形式体现了当前实现中的两项关键设计：路径感知的动态 RTT 阈值，以及以基线相对偏离而非绝对水平判断反馈趋势——后者的必要性在于环境侧反馈值在绝大多数 ACK 事件上为正，绝对阈值会使调节退化为单向棘轮。

5.7.4 收敛性的 Lyapunov 分析

定义 Lyapunov 函数：

$$V(t) = \frac{1}{2}(cwnd(t) - cwnd^*)^2 + \frac{1}{2}(queue(t) - queue^*)^2 \quad (5.4)$$

当 $cwnd > cwnd^*$ 且出现 ECN 拥塞通知时, 当前实现按 $cwnd(t+1) = 0.75 \cdot cwnd(t)$ 进行窗口退让; 若出现普通丢包或超时, 则分别按 0.70 或 0.50 执行更强响应。该分层缩减机制有助于使窗口向目标区间回落。

5.7.5 公平性的定性分析

多流公平性同时受拥塞控制规则、流启动时刻、RTT、队列规则和丢包/标记分布影响。Swift 为各连接维护独立状态, 并对相同类型的拥塞信号使用相同响应规则, 这为同一路径上的一致处理提供了基础, 但不能推出与其他算法竞争时的理论均衡。

第四章中, Swift 在纯 TCP 设置下的 Jain 指数均值为 0.9989、最小值为 0.9967, 四种协议的场景均值均接近 1。逐场景看, Swift 在 11 个场景严格最优, 在 32 个场景中与最优基线的差值不超过 0.002; UDP 突发下对应数量为 17 个和 35 个。该结果支持“总体与基线相当”的描述, 不支持逐场景最优或由 BDP 目标必然产生公平性的因果结论。

5.8 本章小结

本章围绕三项核心技术讨论了 Swift 的设计边界: 多信号拥塞状态感知为超时、ECN 驱动的窗口缩减和普通丢包提供语义区分; 时间窗口交付率、最小 RTT 感知阈值与基线相对反馈共同构成启发式自适应控制; 差异化保留因子、窗口边界、连续缩减保护、降窗后冻结和陈旧动作作废用于限制窗口过度变化。

现有证据表明, Swift 在所考察场景中多数呈现约 2%–6% 的吞吐优势, 但时延优势不普适: 纯 TCP 下相对 BBR 平均高约 0.8%, 个别场景最高约 10.4%; UDP 突发下相对最低时延基线的最大差异约 12.2%。公平性总体接近 1 且与基线相当, 丢包率也不存在逐场景一致优势。当前结果来自单一随机种子和 RED/ECN 配置, 且缺少机制消融、窗口轨迹与真实终端测试, 因此多种子重复、配置扫描和真实协议栈验证仍是必要工作。

第六章 结论与展望

6.1 研究总结

本文提出 Swift 算法，一种面向远距离传输、多接入链路和异构终端应用条件的启发式多信号融合自适应网络拥塞控制方法。Swift 通过协议栈外的确定性规则在确认或拥塞事件上计算窗口决策，不依赖训练样本与神经网络推理；在给定仿真配置和随机种子时，其决策过程可按规则分支检查并重复执行。

本文的研究历程可以概括为以下几个阶段：

问题分析阶段：首先深入分析了远距离传输、蜂窝/无线/卫星等多接入链路以及手机、笔记本、台式机和边缘设备等异构终端并存场景中的拥塞控制挑战。在此基础上，系统性地分析了传统拥塞控制算法（基于丢包的和基于延迟的）的优缺点、ECN 机制的潜力与利用不足，以及学习型拥塞控制方法在训练依赖、推理开销与可审计性方面的工程障碍，确定了以协议栈内部信号驱动的黑盒启发式设计作为研究方向。

方案设计阶段：基于问题分析的结论，设计了 Swift 算法的核心组件。从状态设计入手，提出了 11 维有效观测子空间（嵌入于 15 元素状态传输容器，另含 4 项元数据通道）；然后设计了多信号融合拥塞检测方法，采用分层优先级机制区分超时、ECN 驱动的窗口缩减与普通丢包；接着设计了差异化拥塞响应机制和启发式反馈自适应参数调优机制；最后将各组件整合为以 $\alpha \times BDP$ 为目标的融合控制策略。

实现与验证阶段：在 ns-3.40 网络仿真平台上实现了 Swift 算法原型，仿真进程与决策进程经 ZeroMQ 同步请求—应答信道交互；瓶颈网关配置启用 ECN 标记的 RED 主动队列管理，并对四种被比较协议对称开启 ECN。实验矩阵覆盖近端高速、共享收敛链路、无线局域网接入、蜂窝移动接入、城域与长途/跨域广域网及 LEO/GEO 卫星通信共 36 个场景，在“纯 TCP”与“TCP + 平均负载为瓶颈 32%（峰值 64%、50% 占空比）的 UDP Burst”两类设置下对四种协议各运行一次，共 288 次仿真。指标由 ns-3 FlowMonitor 记录的前向 TCP 数据流计算，并通过流数完整性、物理取值范围和配置一致性检查。

分析阶段：Swift 在多数纯 TCP 与 UDP 突发场景中呈现较高聚合吞吐量，相对逐场景最强基线的增益约为 2%~6%。时延、丢包与公平性则具有场景差异：纯 TCP 下 Swift 相对 BBR 的平均时延高约 0.8%，个别场景最高约 10.4%；UDP 突发下相对最低时延基线的最大差异约 12.2%；丢包率与 NewReno 相当，公平性总体接近 1 但并非逐场景最优。据此，本文将排队时延感知的 α 调度、多时间尺度 BDP 估计、多种子重复和机制消融列为后续工作。

本文的主要贡献和研究成果总结如下：

(1) 多信号拥塞状态感知

本文设计了 11 维有效观测子空间（嵌入于 15 元素状态传输容器，另含 4 项跨进程路由用的元数据通道），将显式拥塞通知（ECN）状态、拥塞控制状态机状态、拥塞事件类型等 TCP 协议栈内部信息与窗口、在途字节和 RTT 观测结合。该状态集合为超时、ECN 驱动的窗口缩减和普通丢包三类语义判定提供输入，但不代表覆盖协议栈的全部状态。

（2）启发式反馈自适应的融合控制

本文按滑动时间窗口内累计确认字节的差分计算交付速率，经容量 40 的窗口最大值滤波并乘以最小 RTT 得到 BDP 估计。控制器依据最小 RTT 感知的 RTT 膨胀阈值、性能反馈值快线 EMA 相对慢线基线 EMA 的偏移以及连续增长趋势，调节每流乘性增加因子 $\alpha \in [0.85, 1.30]$ 。慢启动阶段结合 HyStart 风格 RTT 膨胀检测；拥塞避免阶段围绕 $\alpha \times BDP$ 目标窗口以有界步长拉升或回落。该控制过程由显式规则执行，不包含策略训练或神经网络推理。

（3）面向不确定拥塞信号的稳定性与安全约束

Swift 对普通丢包、ECN 和超时分别采用 0.70、0.75 和 0.50 的窗口保留因子。连续缩减计数在超过 3 次后保持当前窗口，显式降窗后冻结 4 个 ACK 事件中的窗口增长；Python 控制器将拥塞窗口限制在 $[4 \times MSS, \max(4 \times BDP, 200 \times MSS)]$ 内，环境侧另执行 $2 \times MSS$ 下界校验。当协议栈进入 CA_LOSS 时，尚未应用的窗口决策被作废。上述机制用于限制连续缩减、快速反弹、窗口越界和陈旧动作覆盖，其独立贡献仍需消融实验验证。

在实验验证方面，本文在 ns-3.40 网络仿真平台上实现了 Swift 算法，并在 36 个场景、288 次仿真（两种设置各 144 次）的全矩阵下进行了系统性对比实验。实验结果表明：

- 纯 TCP 设置下，Swift 在多数场景呈现较高聚合吞吐量，相对逐场景最强基线的增益范围为 $+2.1\% \sim +5.8\%$ （均值 $+4.1\%$ ）；该趋势覆盖近端高速、无线/蜂窝接入及 LEO/GEO 卫星、长途与跨域 WAN 等路径。
- 平均单向时延与基线同量级：相对 NewReno/CUBIC 均值平均低约 2.3%（36 个场景中 24 个更低），相对 BBR 平均高约 0.8%，纯 TCP 个别场景最高约 10.4%；UDP 突发下相对最低时延基线的最大差异约 12.2%。
- Swift 平均丢包率与 NewReno 相当且不具有逐场景一致优势；Jain 指数均值为 0.9989（纯 TCP）和 0.9990（UDP 突发），总体接近 1 并与基线相当，分别仅在 11 个和 17 个场景严格最优。
- GEO 卫星、LEO 卫星、长途 WAN 与跨域 WAN 等代表性远距离场景呈现吞吐优势；其中 GEO 场景相对最强基线的增益为 4.2%（纯 TCP）和 3.3%（UDP 突发）。
- 平均负载为瓶颈 32% 的 UDP 突发使四协议吞吐保持率均值落在 68.6%–68.8%；Swift 在多数配对场景呈现 $+2.2\% \sim +5.9\%$ 的绝对吞吐优势，但保持率与基线基本一致。

- 当前长 RTT 结果未再呈现旧版逐 ACK 投递速率估计器对应的明显低吞吐特征，这与时间窗口估计器的设计目标一致；由于缺少新旧估计器消融，本文不把该观察表述为单机制因果证明。

6.2 研究展望

Swift 在当前 36 场景矩阵中多数呈现吞吐优势，但时延、丢包与公平性结果并不一致，且结论受限于 ns-3 仿真环境、单一随机种子、同步决策路径与 RED/ECN 配置范围。以下内容作为后续研究方向，不作为本文已经完成的实现或实验结论。

6.2.1 参数自适应与窗口控制的细化

实验记录了两处可继续优化的边界：（1）Swift 在个别深缓冲/低速率场景的相对时延抬升最高约 10%；（2）长 RTT 路径上的吞吐增益（+2.2%~+4.8%）低于近端场景。可探索的方向包括：

- 排队延迟显式估计：引入 $q_{est} = RTT_{smoothed} - RTT_{min}$ 作为 α 调整的额外输入，使 α 在排队延迟持续增长时自动收敛至 1.0 以下。
- 多时间尺度 BDP 估计：区分传播延迟和排队延迟的贡献，避免将排队延迟计入 BDP 估计。
- 受约束的带宽探测：在不突破当前 $\alpha \in [0.85, 1.30]$ 稳定性边界的前提下，为长 RTT 路径设计更细粒度的窗口拉升规则。
- 机制消融实验：分别关闭 ECN/丢包差异化保留因子、连续缩减保护与降窗冻结，量化各机制的独立贡献；当前数据无法完成该分离。

6.2.2 策略表达能力扩展

当前 Swift 算法采用基于规则的启发式决策机制，具有可解释性和低延迟优势，但表示能力受人工规则结构限制。后续研究可在不改变本文实验结论的前提下，探索离线强化学习训练神经网络策略、再通过模型蒸馏提取为紧凑规则的路线；也可引入元学习方法加快不同网络环境之间的参数迁移。相关方法需要在完整实验矩阵、异常过滤和可复现日志基础上重新评估，在获得结果之前不作为本文系统的组成部分。

6.2.3 多目标优化扩展

当前 Swift 的优化目标主要是高吞吐量、低延迟和低丢包。在移动终端、边缘计算和多业务并发场景中，后续研究可进一步引入公平性、能耗、可靠性和业务优先级等目标。多目标优化存在吞吐、延迟、公平性和能耗之间的冲突，需要基于约束优化方法重新设计反馈函数与评价体系。

6.2.4 真实网络部署与验证

本文实验主要在 ns-3 仿真平台上进行，与真实网络环境存在差距。后续部署验证需要将算法实现为 Linux 内核 TCP 模块或 QUIC 用户态协议栈，并在覆盖近端高速、蜂窝/无线接入、广域网与卫星链路的真实或半实物测试床上重新测量。真实环境中的 NIC offload、中断合并、NUMA 效应、操作系统调度、无线链路波动和终端算力差异均可能影响实际性能，因此不能直接将本文仿真结果外推为生产部署效果。

6.2.5 实验方法的扩展

受单次确定性运行限制，本文未报告跨运行号置信区间。后续应使用多个 ns-3.40 RngRun 对关键场景重复仿真并给出均值与最坏值；同时可扩展队列管理策略（丢尾、CoDel）与突发强度/占空比扫描，检验结论对配置的稳健性。

6.2.6 安全性、鲁棒性与跨层协同

后续研究需要进一步验证算法在异常输入、通信中断、状态损坏和恶意跨流量下的行为边界。可行方向包括：输入合法性检查、异常状态回退、保守策略兜底、形式化安全性质描述，以及与应用层、链路层和操作系统调度器的协同机制。上述方向均需要独立实现和实验验证，本文不将其作为当前系统已经具备的能力。

6.2.7 异构网络与新兴传输协议适配

远距离传输和异构终端场景通常跨越有线、无线、蜂窝、卫星和边缘接入等多类链路。后续研究可进一步考察无线非拥塞丢包识别、卫星链路长 RTT 适配、5G 突发抖动处理、终端算力约束下的轻量化执行，以及将多信号融合、差异化响应和自适应调参思想迁移到 QUIC 用户态协议栈中的可行性。相关扩展应基于新的实现版本、完整实验矩阵和异常过滤流程重新评估。

6.3 应用场景展望

基于本文仿真结果，Swift 更适合作为面向远距离传输、异构终端和多接入链路的拥塞控制研究原型，而不是已经完成真实网络生产部署验证的协议。其潜在应用场景包括：

- **跨地域内容传输**：在长 RTT 路径上维持更高吞吐并控制队列驻留（跨域/长途 WAN 与 GEO/LEO 场景实测领先）。
- **蜂窝与无线接入**：利用 ECN、RTT 和在途字节等多源信号提高对接入链路波动的适应能力。
- **移动终端与边缘应用**：面向手机、笔记本、桌面终端和边缘设备并存的场景，探索更稳健的参数自适应机制。
- **卫星与广域链路**：在高传播时延和高 BDP 路径上保持带宽估计有效并减少拥塞恢复代价。

上述应用方向体现了算法设计的潜在价值，但不意味着本文仿真结果可以直接等同于生产网络收益。实际部署前仍需要完成内核或用户态协议栈实现、真实测试床验证、跨流公平性评估和长期稳定性评估。

6.4 研究意义与社会价值

本研究的理论意义主要体现在三个方面：首先，提出了面向远距离和异构终端传输的 11 维有效观测子空间，为协议栈信号驱动的拥塞控制状态设计提供了参考；其次，提出了多信号融合拥塞检测与差异化响应机制，为区分 ECN、丢包和超时等不同拥塞信号提供了可解释规则；最后，提出了结合 BDP 估计、路径感知 α 调节、降窗后冻结和连续缩减保护的白盒启发式融合控制框架，验证了“确定性规则 + 轻量反馈自适应”设计点在跨场景一致性上的可行性。

本研究通过 ns-3.40 构建了一个可重复执行的实验原型，考察多信号感知与启发式反馈自适应在长距离、多接入和异构终端应用条件下的可行性。在 RED/ECN 对称配置中，该方法在多数纯 TCP 与 UDP 突发场景呈现吞吐优势，时延、丢包和公平性总体与基线相当但存在场景差异。由于实验采用链路参数抽象而非真实终端硬件，实际部署效果仍需在真实协议栈和网络环境中验证。

参考文献

- [1] Floyd S, Henderson T, Gurtov A. The NewReno modification to TCP's fast recovery algorithm[J]. RFC 6582, 2012.
- [2] Ha S, Rhee I, Xu L. CUBIC: a new TCP-friendly high-speed TCP variant[J]. ACM SIGOPS Operating Systems Review, 2008, 42(5): 64–74.
- [3] Brakmo L S, Peterson L L. TCP Vegas: End to end congestion avoidance on a global Internet[J]. IEEE Journal on Selected Areas in Communications, 1995, 13(8): 1465–1480.
- [4] Cardwell N, Cheng Y, Gunn C S, et al. BBR: Congestion-based congestion control[J]. ACM Queue, 2016, 14(5): 20–53.
- [5] Alizadeh M, Greenberg A, Maltz D A, et al. Data center TCP (DCTCP)[J]. ACM SIGCOMM Computer Communication Review, 2010, 40(4): 63–74.
- [6] Jay N, Rotman N, Godfrey B, et al. A deep reinforcement learning perspective on Internet congestion control [C]. International Conference on Machine Learning. [S.l.]: PMLR, 2019: 3050–3059.
- [7] Jacobson V. Congestion avoidance and control[J]. ACM SIGCOMM Computer Communication Review, 1988, 18(4): 314–329.
- [8] Xu L, Harfoush K, Rhee I. Binary increase congestion control (BIC) for fast long-distance networks[C]. IEEE INFOCOM 2004: volume 4. [S.l.]: IEEE, 2004: 2514–2524.
- [9] Gettys J, Nichols K. Bufferbloat: Dark buffers in the Internet[J]. Communications of the ACM, 2012, 55(1): 57–65.
- [10] Arun V, Balakrishnan H. Copa: Practical delay-based congestion control for the Internet[C]. 15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18). [S.l.]: USENIX Association, 2018: 329–342.
- [11] Hock M, Bless R, Zitterbart M. Experimental evaluation of BBR congestion control[C]. 2017 IEEE 25th International Conference on Network Protocols (ICNP). [S.l.]: IEEE, 2017: 1–10.
- [12] Tan K, Song J, Zhang Q, et al. A compound TCP approach for high-speed and long distance networks[C]. Proceedings IEEE INFOCOM 2006. [S.l.]: IEEE, 2006: 1–12.
- [13] Liu S, Başar T, Srikant R. TCP-Illinois: A loss-and delay-based congestion control algorithm for high-speed networks[J]. Performance Evaluation, 2008, 65(6-7): 417–440.
- [14] Mascolo S, Casetti C, Gerla M, et al. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links[C]. Proceedings of the 7th Annual International Conference on Mobile Computing and Networking. [S.l.]: ACM, 2001: 287–297.
- [15] Ramakrishnan K, Floyd S, Black D. The addition of explicit congestion notification (ECN) to IP[J]. RFC 3168, 2001.
- [16] Bai W, Chen K, Hu S, et al. Enabling ECN over generic packet scheduling[C]. Proceedings of the 12th

- International on Conference on emerging Networking EXperiments and Technologies. [S.l.]: ACM, 2016: 191–204.
- [17] Li Y, Miao R, Liu H H, et al. HPCC: High precision congestion control[C]. Proceedings of the ACM Special Interest Group on Data Communication. [S.l.]: ACM, 2019: 44–58.
- [18] Winstein K, Balakrishnan H. TCP ex machina: Computer-generated congestion control[J]. ACM SIGCOMM Computer Communication Review, 2013, 43(4): 123–134.
- [19] Jay N, Rotman N, Godfrey B, et al. Internet congestion control via deep reinforcement learning[C]. Advances in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2019.
- [20] Abbasloo S, Yen C Y, Chao H J. Classic meets modern: A pragmatic learning-based congestion control for the Internet[C]. Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication. [S.l.]: ACM, 2020: 632–647.
- [21] Dong M, Li Q, Zarchy D, et al. PCC: Re-architecting congestion control for consistent high performance[C]. 12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15). [S.l.]: USENIX Association, 2015: 395–408.
- [22] Jain R K, Chiu D M W, Hawe W R. A quantitative measure of fairness and discrimination[R]. [S.l.]: Eastern Research Laboratory, Digital Equipment Corporation, 1984.
- [23] Finn C, Abbeel P, Levine S. Model-agnostic meta-learning for fast adaptation of deep networks[C]. International Conference on Machine Learning. [S.l.]: PMLR, 2017: 1126–1135.

附录 A 程序清单

第三章 Swift 拥塞控制算法 Python 实现 (swift-tcp 文件夹)

`tcp_swift.py`: Swift 启发式决策模块核心实现 (约 380 行), 包含 `TcpSwift` 类, 实现多信号融合拥塞检测、差异化拥塞响应、启发式自适应参数调优 (α 动态调整)、融合控制策略 (BDP 估计与窗口调整)、每流独立状态管理等全部核心算法逻辑

`tcp_base.py`: TCP 状态观测基类 (约 140 行), 实现 `TcpEventBased` (11 维事件驱动) 和 `TcpTimeBased` (16 维时间驱动) 两种状态解析模式, 负责将 ns-3 传递的原始状态向量解析为语义化的决策参数

`test_swift.py`: Swift 算法测试与运行脚本 (约 120 行), 基于 ns3-gym 框架的跨进程运行入口, 创建状态—决策传输环境实例, 为每个 TCP 套接字 (`socketUuid`) 创建独立的 Swift 决策模块实例, 实现多流并发控制

第三章 ns-3 网络仿真 C++ 模块 (swift-tcp 文件夹)

`tcp-swift.cc / tcp-swift.h`: `TcpSwift` 拥塞控制类实现, 负责在 ns-3 TCP 协议栈中注册 Swift 算法并创建环境实例, 采集协议栈状态并应用窗口决策

`tcp-swift-env.cc / tcp-swift-env.h`: Swift 环境类 (约 230 行), 继承自 ns3-gym 的 `TcpGymEnv`, 实现 11 维状态空间的数据采集与传递、ECN 感知的反馈值计算 (ECN 惩罚因子)、延迟拥塞窗口应用机制 (Deferred Cwnd)

第四章网络仿真实验配置 (swift-tcp 文件夹)

`sim.cc`: ns-3 网络仿真主程序 (约 400 行), 构建哑铃型 (Dumbbell) 网络拓扑、配置 TCP 协议参数与 RED/ECN 队列、设置 `FlowMonitor` 流量监控、配置跨进程状态传输接口, 支持通过命令行参数灵活配置 36 种实验场景与 UDP 突发干扰

`Makefile`: 自动化编译与实验运行脚本, 支持多场景批量实验、对比测试和日志收集

附录 B 攻读专业学位硕士期间撰写的论文

[1] 暂无已发表论文。

附录 C 攻读专业学位硕士期间申请的专利

[1] 暂无已申请专利。

附录 D 攻读专业学位硕士期间参加的科研项目

[1] 暂无参与项目。

致谢

感谢所有对本研究提供帮助和支持的人员。感谢 ns-3 网络仿真平台与 ns3-gym 开源社区提供的优秀仿真平台与跨进程通信组件。感谢审稿人提出的宝贵意见和建议。

本研究的完成离不开导师的悉心指导和实验室同学们的帮助。在研究过程中，我们进行了大量的讨论和交流，这些思想碰撞极大地丰富了研究内容。特别感谢实验室提供的计算资源，使得大规模网络仿真实验得以顺利进行。

感谢家人多年来的理解和支持，让我能够专注于科研工作。感谢所有在学习和生活中给予我帮助的老师、同学和朋友们。

最后，感谢计算机网络领域前辈们的开创性工作，正是站在巨人的肩膀上，本研究才得以完成。Van Jacobson、Sally Floyd 等先驱在 TCP 拥塞控制领域的贡献，为后来的研究奠定了坚实的基础。我们希望本研究能够在他们开创的道路上继续前行，为网络拥塞控制技术的发展做出微薄的贡献。